INDUSTRIAL AUTOMATION | TECHNICAL MANUAL

TEMPERATURE MONITORING AND DATA LOGGING SYSTEM

Installation, Configuration, Operation and Maintenance Manual

Raspberry Pi 5 node | Waveshare 2-CH RS-485 HAT | PVR SZ-7526TX-REA | Ubuntu server

Document ID	TMS-UM-001
Revision	1.0 (Draft for site review)
Date	2026-11-07
Software baseline	Repository commit 981414c (software v1.0.0)
Audience	Technicians, system administrators, maintenance engineers, developers

CONFIDENTIAL - AUTHORIZED PERSONNEL ONLY

This manual contains system access credentials. Store the PDF and the DOCX securely, share them only with authorized personnel and do not e-mail or upload them to public services. Change default passwords where appropriate (see Chapter 17).

Verified against the project source code. Items that could not be verified are marked 'To be confirmed'.

Document Control

Revision history

Rev	Date	Description	Prepared by	Status
1.0	2026-11-07	First complete issue. Written from the project source code (repository commit 981414c, software v1.0.0) and from the screenshots and photographs supplied by the project owner.	Claude Code (AI-assisted) for the project owner	Draft for site review

Purpose and scope

This manual explains how to install, configure, operate, monitor and maintain the Temperature Monitoring and Data Logging System: PVR SZ-7526TX-REA temperature controllers read over RS-485 by a Raspberry Pi 5 node, which forwards the readings to a Flask web server running on an Ubuntu virtual machine. It is written so that a technician who has never configured the system can set up the Raspberry Pi node, configure the server, start both applications, log in to the website, monitor temperatures and export historical data.

It also explains **what happens when something fails**: Chapter 8 describes the exact behaviour of the system when the server, the Raspberry Pi or both are shut down for an hour, how the Raspberry Pi buffers readings as *pending*, and what the 7-day local retention really covers.

How this manual was verified

- **Source code first.** All files under `raspberry_pi_node/`, `server/`, `scripts/`, `docs/` and `dev_tools/` were read. Register addresses, serial parameters, file paths, service names, table names, timings and limits in this manual are taken from that code and cross-checked against the controller's instruction manual and register map. The chapter outline supplied with the request was used as a checklist, not as a source of facts.
- **Supplied by the project owner.** IP addresses, host names, user names, passwords, Wi-Fi details and the hardware photographs cannot be verified from the source code. They are reproduced as supplied and labelled accordingly.
- **Not executed.** No command in this manual was run against your Raspberry Pi, virtual machine or controller while it was written. Commands come from the project's own scripts and documents or are standard Linux usage. Example outputs are typical and will differ in detail. Try new procedures on a spare unit first.
- **To be confirmed.** Anything that could not be established from the code or the supplied material is marked **To be confirmed**. Appendix D lists all of them in one place.

Conventions used in this manual

Convention	Meaning
`monospace text`	A command, file name, path, setting or value to type or look for exactly as shown.
(Technician) / **(Administrator)**	Who normally performs the procedure. Technician = on-site person working on the hardware and the Raspberry Pi. Administrator = person with administrator rights on the Ubuntu server or the website.
To be confirmed	A fact that could not be verified from the source code. Check it on site before relying on it.
Supplied by owner	A value given by the project owner (credentials, IP addresses, host names).
Example output	Typical output shown for orientation. Your values will differ.

> **NOTE**
> A note gives background or a useful tip.

IMPORTANT
An important message describes something that affects results or safety of the data.

WARNING
A warning describes an action that can cause data loss, downtime or a security problem.

TO BE CONFIRMED
A to-be-confirmed box lists something to verify on site.

VERIFIED IN SOURCE
A verified-in-source box states what the project code actually does, with the file it comes from.

Roles

Role	Typical person	What they do with this system
Technician	Electrician / instrumentation technician	Wires RS-485, sets controller parameters, powers the Raspberry Pi, connects it to the network, checks the node on site.
System administrator	IT or automation engineer	Maintains the Ubuntu server, SSH access to the Raspberry Pi, backups, upgrades; uses the website role Administrator.
Operator (web role)	Shift operator	Dashboard, Monitoring, Data Logs, Nodes (view), Reports and exports.
Auditor (web role)	Quality / compliance	Same as Operator plus Audit Trail and audit reports.
Administrator (web role)	System administrator	Everything, including node registration, users, roles and system settings.

Source material reviewed

Area	Files
Raspberry Pi node	`raspberry_pi_node/src/` (main, collector, modbus_client, registers, decoder, local_queue, uploader, server_client, health_monitor, config_manager, config_sync, commands, logging_config, validate_controller), `config.example.yaml`, `temperature-node.service`, `requirements.txt`
Server	`server/run.py`, `server/app/` (config, models, routes, services, templates), `server/migrations/`, `.env.example`, `requirements.txt`
Scripts and tools	`scripts/setup_raspberry_pi.sh`, `scripts/backup_database.py`, `dev_tools/read_temp_from_controllert.py`, `dev_tools/delete_controler_in_server_history.py`
Project documents	`README.md`, `docs/architecture.md`, `modbus-register-map.md`, `configuration-guide.md`, `installation-raspberry-pi.md`, `hardware-validation.md`, `troubleshooting.md`, `development-log.md`, `api.md`
Controller documents (project folder, beside the repository)	`SZ-7526TX_0.pdf` (instruction manual REV3), `SZ-7526TX-REA_PRM.pdf` (register map). `SZ-4042-C_PRM.pdf` is the register map of a different controller model (SZ-4042-C); this software does not implement it and it is not used in this manual.
Supplied material	10 website screenshots, 3 hardware photographs, system access details

Table of Contents

Document Control . . . 2

Revision history . . . 2
Purpose and scope . . . 2
How this manual was verified . . . 2
Conventions used in this manual . . . 2
Roles . . . 3
Source material reviewed . . . 3

System Access Credentials and Network Configuration . . . 9

Remote Ubuntu virtual machine . . . 9
Web application login . . . 9
Raspberry Pi - Node 1 . . . 9
Wi-Fi configuration . . . 10
Names you will meet in the system . . . 10
Identify the Raspberry Pi's current IP address . . . 10
Connect to the Raspberry Pi and to the server with SSH . . . 11
Security recommendations . . . 11

1 Introduction . . . 12

1.1 Purpose of the system . . . 12
1.2 Applications and use cases . . . 12
1.3 System capabilities . . . 12
1.4 Major hardware and software components . . . 13
1.5 Limitations to be aware of . . . 13

2 System Architecture . . . 14

2.1 Architecture block diagram . . . 14
2.2 Role of each component . . . 14
2.3 How the Raspberry Pi communicates with the controllers . . . 15
2.4 How temperature data is collected . . . 15
2.5 How the node transfers data to the server . . . 15
2.6 How the server processes and stores data . . . 15
2.7 How the website retrieves and displays data . . . 15
2.8 Network layout . . . 16
2.9 Hardware configuration . . . 16
2.10 Where the key information is . . . 20

3 Raspberry Pi Initial Configuration . . . 21

3.1 What you need (Technician) . . . 21
3.2 Prepare Raspberry Pi OS (Technician) . . . 21
3.3 First boot and connection (Technician) . . . 21
3.4 Log in over SSH (Technician) . . . 22
3.5 Update the operating system (Technician) . . . 22
3.6 Set and check the host name . . . 22
3.7 Set the time zone and check time synchronisation . . . 22
3.8 Enable the RS-485 HAT (Technician) . . . 22
3.9 Install Python and other packages . . . 23
3.10 Verify network connectivity and the server . . . 23

4 Raspberry Pi Network Configuration . . . 24

4.1 Wi-Fi (interface wlan0) . . . 24
4.2 Ethernet (interface eth0) . . . 26

5 Uploading and Running the Raspberry Pi Program . . . 27

5.1 Files that make up the node . . . 27
5.2 Step 1 - Connect with SSH (Technician) . . . 27
5.3 Step 2 - Create the project folder and upload the files (Technician) . . . 28
5.4 Step 3 - Register the node on the website (Administrator) . . . 28
5.5 Step 4 - Run the installer (Technician) . . . 28
5.6 Step 5 - Edit the configuration (Technician) . . . 29
5.7 Step 6 - Store the token (Technician) . . . 30
5.8 Step 7 - Validate the configuration . . . 31
5.9 Step 8 - Check the controller communication before going live (Technician) . . . 31
5.10 Step 9 - Run the program manually and watch it (Technician) . . . 31
5.11 Step 10 - Check the logs and that data reaches the server . . . 32
5.12 Upgrading the node software . . . 32
5.13 Common errors . . . 32

6 Automatic Startup Configuration . . . 33

6.1 The service file . . . 33
6.2 Commands (Technician) . . . 34
6.3 Test the automatic start . . . 34
6.4 Disable automatic startup . . . 34

7 Temperature Controller Configuration . . . 35

7.1 The controller . . . 35
7.2 Communication parameters . . . 35
7.3 Set the controller's communication parameters (Technician) . . . 36
7.4 Registers used to collect data . . . 37

7.5 The Modbus request on the wire . . . 38
7.6 Temperature data format and worked examples . . . 38
7.7 Data quality labels . . . 39
7.8 RS-485 wiring (Technician) . . . 40
7.9 Verify that the controller is responding (Technician) . . . 41

8 Data Acquisition, Transmission and Fail-Safe Operation . . . 43

8.1 Short answers . . . 43
8.2 The journey of one measurement . . . 44
8.3 What is sent to the server . . . 45
8.4 The local buffer: queue.db and the pending status . . . 45
8.5 What the server does with the data . . . 47
8.6 Outage scenarios . . . 48
8.7 A real example from your audit trail . . . 50
8.8 Fail-safe mechanisms at a glance . . . 50
8.9 What the system does not protect against . . . 51
8.10 Test the fail-safe behaviour on site (Administrator) . . . 51

9 Ubuntu Remote Server Configuration . . . 52

9.1 Layout on the server . . . 52
9.2 1-2 Access the virtual machine and connect with SSH (Administrator) . . . 52
9.3 3 Update Ubuntu . . . 52
9.4 4-5 Install Python, pip, venv support and tools . . . 53
9.5 6-7 Create the application folder and upload the server files . . . 53
9.6 8 Create the virtual environment and install the dependencies . . . 53
9.7 9-12 Configure environment variables, database, application and port . . . 53
9.8 Create the database, roles and the first administrator . . . 54
9.9 Open the firewall port . . . 54
9.10 13 Run the server manually . . . 55
9.11 14 Test the server . . . 55
9.12 15 Start the server automatically after reboot . . . 55
9.13 16 Check the server logs . . . 56
9.14 Upgrading the server software . . . 56
9.15 17 Troubleshooting the server . . . 56

10 Database Configuration and Data Storage . . . 57

10.1 Which database is used . . . 57
10.2 How the database is created . . . 57
10.3 Tables and their purpose . . . 57
10.4 How node and controller information is stored . . . 58

10.5 How timestamps are recorded . . . 58
10.6 How historical data is retrieved . . . 58
10.7 Check the database from the command line (Administrator) . . . 58
10.8 Back up the database . . . 59
10.9 Restore the database . . . 59

11 Website Access and Login . . . 60

11.1 Procedure . . . 60
11.2 Navigating the website . . . 60
11.3 Change your password . . . 62

12 Website Features and Page-by-Page Explanation . . . 63

12.1 Dashboard . . . 63
12.2 Monitoring . . . 64
12.3 Data Logs . . . 65
12.4 Nodes, node page and controller page (no screenshot supplied) . . . 66
12.5 Reports (no screenshot supplied) . . . 67
12.6 Audit Trail . . . 68
12.7 Exports (export history) . . . 69
12.8 Administration: Users . . . 69
12.9 Administration: Roles and permissions . . . 70
12.10 Administration: System settings . . . 71

13 Monitoring Temperature Data . . . 72

13.1 Open the monitoring page and identify node and controller . . . 72
13.2 Read the current temperature and its time . . . 72
13.3 Statuses: how to recognise missing, outdated and failed data . . . 72
13.4 Find communication failures and data gaps . . . 73
13.5 Verify that the Raspberry Pi is sending data correctly . . . 73
13.6 The complete workflow from controller to screen . . . 73

14 Exporting Temperature Logs . . . 74

14.1 Export the readings shown in Data Logs . . . 74
14.2 Excel workbook layout . . . 74
14.3 PDF report layout . . . 75
14.4 Verify and keep exports . . . 75

15 Troubleshooting . . . 76

15.1 Problem table . . . 76
15.2 Diagnostic command cheat sheet . . . 77

16 Maintenance and Backup . . . 79

16.1 Restarting (Technician / Administrator) . . . 79

16.2 Check resources (Administrator) . . . 79

16.3 Routine schedule . . . 79

16.4 Backing up (Administrator) . . . 80

16.5 Restoring the system after a failure (Administrator) . . . 80

17 System Access Credentials . . . 81

17.1 Consolidated credentials . . . 81

17.2 Network configuration . . . 81

17.3 Application secrets (not passwords you type) . . . 81

17.4 Changing the default passwords . . . 82

18 Final System Verification Checklist . . . 83

18.1 Raspberry Pi and controller . . . 83

18.2 Server and website . . . 83

18.3 Data protection and operation . . . 84

Appendix A Controller Parameter Registers 40031-40068 . . . 85

A.1 System status register 40010 (bits) . . . 86

A.2 Coils (function 01 read / 05 write) . . . 86

Appendix B Event, Status and Quality Glossary . . . 87

B.1 Node events . . . 87

B.2 Server-generated events . . . 87

B.3 Quality flags . . . 87

B.4 Glossary . . . 88

Appendix C Quick Command Reference . . . 89

C.1 Raspberry Pi . . . 89

C.2 Ubuntu server . . . 89

Appendix D Items To Be Confirmed . . . 90

Appendix E Source File Map . . . 91

System Access Credentials and Network Configuration

CONFIDENTIAL - AUTHORIZED PERSONNEL ONLY
CONFIDENTIAL - AUTHORIZED PERSONNEL ONLY. This section contains passwords. Store this document in a secured location, restrict who can open it, never e-mail or upload it to public services, and change the default passwords where appropriate (see Chapter 17).

SUPPLIED BY OWNER - NOT VERIFIABLE FROM CODE
All values in this section are supplied by the project owner. The source code stores only hashed web passwords, so none of the passwords can be verified from the code. Confirm each value on site.

Remote Ubuntu virtual machine

Parameter	Configuration
IP address	`10.145.15.245`
Hostname	`temp-Virtual-Machine`
Username	`temp`
Password	`Temp@098`
Operating system	`Ubuntu (version To be confirmed; the project README supports Ubuntu 22.04 and 24.04)`
Purpose	`Remote server hosting the application and the database`

NOTE
The user name `temp` stands for **Temperature Monitoring System**. It does not mean *temporary*.

Web application login

Parameter	Configuration
Website URL	`http://10.145.15.245:8000`
Username	`admin`
Password	`avo@1234`

To open the website, connect a computer to the plant network, start a web browser, type `http://10.145.15.245:8000` in the address bar, and sign in with the user name and password above. The full procedure with screenshots is in Chapter 11. The website listens on TCP port 8000 and uses plain HTTP (no certificate), as configured by `HOST` and `PORT` in the server's `.env` file.

Raspberry Pi - Node 1

Parameter	Configuration
Hostname	`temp-logging-system-node-1`
Ethernet IP	`192.168.0.100`
Username	`pi`
Password	`avo1234`
Device role	`Temperature Monitoring Node 1`

Wi-Fi configuration

Parameter	Configuration
Wi-Fi SSID	`AVO_MACHINES`
Wi-Fi password	`b1n@ry010`
Currently observed IP	`10.145.15.166`
Network interface	`wlan0`

Do not treat the Wi-Fi IP address as permanent
The Wi-Fi address `10.145.15.166` is a **DHCP address observed at the time of writing**. The router may give the Raspberry Pi a different address after a restart or lease expiry. Do not treat it as a permanent address. The Ethernet address `192.168.0.100` is the address to use when you connect with a cable to that network.

Names you will meet in the system

Several different names are used. They are **not** interchangeable:

Name	Example in this system	Where it is used
Raspberry Pi host name	temp-logging-system-node-1	The operating system name of the Pi (shown in the SSH prompt).
Node ID	temp_control_unit_1	The application name of the node. Must be identical in the Pi's `config.yaml` and in the website under Nodes. Shown in Data Logs and in the Audit Trail (Figures 12-5 and 12-3).
Controller ID	Temp_Control_1, Temp_Control_2	One per physical controller; unique within a node. Shown as *Temp Control 1* and *Temp Control 2* on the dashboard (Figure 12-1).
Slave ID (`did`)	1 and 2	The Modbus address set on each controller. It must match the controller's `slave_id` setting.
Ubuntu VM host name	temp-Virtual-Machine	The operating system name of the server.

Identify the Raspberry Pi's current IP address

The Raspberry Pi has two network interfaces with different addresses:

Interface	Address type	When to use it
`eth0` (Ethernet cable)	192.168.0.100 (supplied by owner; fixed or reserved - **To be confirmed**)	Commissioning and maintenance with a cable. The address does not depend on Wi-Fi.
`wlan0` (Wi-Fi, SSID AVO_MACHINES)	Assigned by the Wi-Fi router (DHCP); can change	Normal operation: this is the path the node uses to reach the server at 10.145.15.245.

To find the Wi-Fi address, log in to the Raspberry Pi (by cable at `192.168.0.100`, or with a keyboard and monitor) and run:

On the Raspberry Pi

```
hostname -I
ip -4 addr show wlan0
nmcli -f GENERAL.STATE,IP4.ADDRESS,IP4.GATEWAY device show wlan0
```

`hostname -I` lists all addresses of the Pi. The line starting `inet` under `wlan0` is the Wi-Fi address. You can also read the address the node last used from the website: the node's page (Nodes > the node) shows **Host / IP**, and the **Source IP** column of the Audit Trail shows the address from which the node connected (for example `10.145.15.166` in Figure 12-5). The DHCP client list in the Wi-Fi router shows it as well.

Connect to the Raspberry Pi and to the server with SSH

SSH is a secure remote login. On Windows 10/11 it is built in (Windows Terminal or PowerShell):

> **From a Windows PC (first line: Pi by Ethernet; second: Pi by Wi-Fi; third: Ubuntu server)**
> `ssh pi@192.168.0.100`
> `ssh pi@<current-Wi-Fi-IP>`
> `ssh temp@10.145.15.245`

The first time you connect, SSH shows the host's fingerprint and asks `Are you sure you want to continue connecting`. Type `yes`. Then enter the password from the tables above (nothing is shown while you type). To leave the session, type `exit`. If you reinstall a device and SSH warns that the host key has changed, remove the old entry with `ssh-keygen -R <address>` and connect again.

Security recommendations

- Change the default passwords of the `pi` and `temp` accounts and of the website `admin` account after commissioning. The website enforces a minimum of 8 characters, a mix of letters with digits or symbols, and not containing the user name; a lock-out follows 5 failed attempts for 15 minutes.
- Create a separate website user for each person (Administration > Users) instead of sharing `admin`, so that the Audit Trail shows who did what.
- Prefer SSH keys to passwords for the `pi` and `temp` accounts once access is working.
- If the Wi-Fi password is changed on the router, the Raspberry Pi will lose its network until the new password is stored on the Pi (Chapter 4). The node keeps recording during that time and uploads the backlog afterwards (Chapter 8).
- Treat the node token (shown once when a node is registered) as a password. It is stored on the Pi in `/etc/temperature-node/node.env`.

CHAPTER 1

Introduction

1.1 Purpose of the system

The Temperature Monitoring and Data Logging System continuously reads the temperature and the operating state of PVR Controls SZ-7526TX-REA temperature controllers, stores every reading, and shows live and historical data in a web browser. It is designed for sites that need a trustworthy temperature record, **including an honest record of when data is missing**, and a tamper-resistant history of who changed what.

Three design rules shape everything in this manual (see `README.md` and `docs/architecture.md`):

- **Nothing is invented.** A failed poll is stored as a record with no temperature. Missing values stay empty; they are never estimated or carried forward.
- **No data loss through outages.** The Raspberry Pi stores every reading on its own disk first and deletes it only after the server confirms that it is stored (Chapter 8).
- **Everything is traceable.** Every login, configuration change, export and communication event is written to an append-only audit trail.

1.2 Applications and use cases

The SZ-7526TX-REA is a refrigeration temperature controller: its registers report a compressor relay, an alarm relay, defrost state, high/low-temperature faults, a door-open input and probe failure (`docs/modbus-register-map.md`). Typical uses of this system are:

- Monitoring cold rooms, freezers and other refrigerated storage from a central screen.
- Keeping a searchable temperature history with set point, compressor and alarm state for each controller.
- Producing temperature-excursion, fault, downtime and missing-data reports for quality or audit purposes.
- Monitoring several controllers per Raspberry Pi and several Raspberry Pi nodes per server.

> **IMPORTANT**
> The project makes **no claim of compliance with any regulatory standard** (`README.md`). Whether the reports are acceptable for a given regulation must be decided by the site's quality department.

1.3 System capabilities

Capability	What it means in practice
Live dashboard	One card per controller with temperature, set point, compressor and alarm state, faults and a clear status label.
Trend monitoring	Chart with minimum/maximum per time bucket, set point, fault changes and failed polls for 1 h, 6 h, 24 h, 7 d, 30 d or a custom range.
Data logs	Every poll attempt, filterable by time, node, controller, site, temperature, faults, communication state, quality and more.
Exports	Excel (.xlsx) and PDF export of exactly the records shown, plus eight audit and operational reports. Every export is recorded with a SHA-256 hash.
Node management	Register Raspberry Pi nodes, edit their configuration remotely (versioned), disable them, issue a new token.
Store-and-forward	The node buffers readings during server or network outages and uploads them in order afterwards.
Health monitoring	Heartbeat every 30 s with queue size, Wi-Fi signal, CPU temperature, disk space and clock synchronisation.

Capability	What it means in practice
Audit trail	Append-only log of logins, configuration changes, communication events, faults and exports. Entries cannot be edited or deleted through the application.
Role-based access	Administrator, Operator and Auditor roles; permissions are checked on the server for every request.

1.4 Major hardware and software components

Component	Details (from the project)
Temperature controller	PVR Controls SZ-7526TX-REA, Modbus RTU slave. In this installation: slave ID 1 (*Temp Control 1*) and slave ID 2 (*Temp Control 2*). Its serial port is a 5-pin TTL header ("TTL OUT (RS485)"), connected to the bus through a TTL-to-RS-485 interface (Chapter 7).
Raspberry Pi	Raspberry Pi 5 (identified from the supplied photograph). The project supports Raspberry Pi 3B+/4/5/Zero 2 W with Raspberry Pi OS Bookworm or newer, Python 3.10 or newer.
RS-485 interface	Waveshare 2-channel RS-485 HAT (per `config.example.yaml`); serial port `/dev/ttySC1`.
Node software	Python application `src.main` (v1.0.0); pymodbus 3.11.3, pyserial 3.5, requests 2.32.3, PyYAML 6.0.2; local buffer in SQLite.
Server	Ubuntu virtual machine 10.145.15.245; Python, Flask 3.1.0 served by Waitress 3.0.2; SQLAlchemy 2.0.36 with Alembic migrations.
Database	SQLite file `instance/temperature.db` (default). PostgreSQL is supported through `DATABASE_URL` but has not been tested by the project.
Web interface	Jinja2 pages with Bootstrap 5 and Chart.js, all bundled locally (works without internet access).
Reports	openpyxl 3.1.5 (Excel), ReportLab 4.2.5 (PDF).

1.5 Limitations to be aware of

- **Hardware validation.** The project's development log states that the software was verified against the register-map documents and a simulator and not yet against a physical controller. Complete the commissioning checks in Chapter 7 and record them (Chapter 18).
- **No e-mail or SMS alerts.** Alarms and faults are visible on the dashboard and in reports only.
- **Controller parameters cannot be changed remotely.** The software only reads parameters. Remote controller commands (alarm mute, fault reset, log reset, defrost) are disabled unless enabled on both the server and the node.
- **Exports are not stored on the server.** Only the export's metadata and SHA-256 are kept; the file is delivered to your browser.
- **One server process.** The offline-detection monitor runs inside the server process, so run only one instance.
- **Large databases.** Very large SQLite databases (tens of millions of rows) make unfiltered Data Logs counts slower. Date filters keep normal use fast.

CHAPTER 2

System Architecture

2.1 Architecture block diagram

Figure 2-1 shows the complete path of a temperature reading from the controller to the person looking at the website.

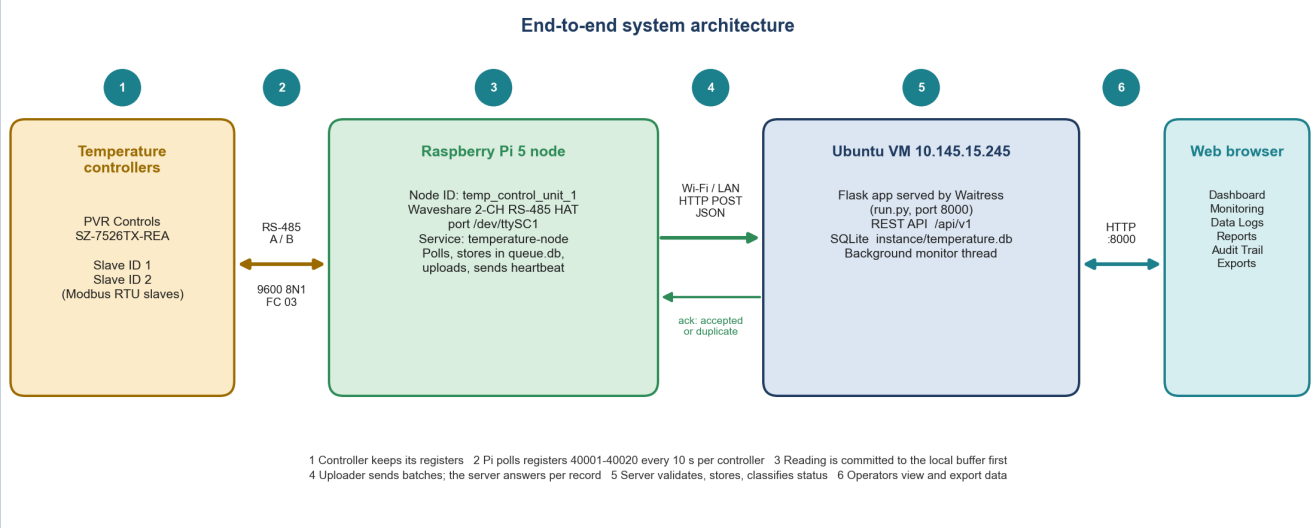


Figure 2-1 *- End-to-end architecture: Temperature controller, RS-485, Raspberry Pi node, Wi-Fi/Ethernet, Ubuntu server with database, web application and end user.*

2.2 Role of each component

Component	Role	Key facts
Temperature controller	Measures the temperature and controls the refrigeration equipment. Exposes its state in Modbus holding registers.	Slave (responds only when asked). Registers 40001-40020 are read-only status data.
RS-485 bus	Two-wire differential serial link between the Raspberry Pi and the controllers.	9600 baud, 8 data bits, no parity, 1 stop bit (project default); half duplex.
Raspberry Pi node	Field computer. Asks each controller for its registers, decodes them, stores them, uploads them, reports its own health.	Always the initiator: the server never connects to the Pi.
Wi-Fi / Ethernet	Carries HTTP requests from the node to the server.	Node uses the server URL `http://10.145.15.245:8000`.
Ubuntu server	Receives readings, validates and stores them, tracks node and controller state, serves the website and exports.	Flask app run by Waitress (`run.py`), port 8000.
Database	Permanent storage of readings, events, nodes, users, audit trail and settings.	SQLite `instance/temperature.db`; readings and audit events are never purged.
Web application	Browser interface for monitoring, history, reports, administration.	Pages refresh themselves every 10 s (setting *Dashboard refresh*).
End user	Operator, auditor or administrator.	Signs in with a user name and password; permissions depend on the role.

2.3 How the Raspberry Pi communicates with the controllers

The node runs one *bus collector* thread per RS-485 port. For this installation there is one bus, `rs485-1` on `/dev/ttySC1`, carrying both controllers. Every 10 seconds per controller the thread sends a **Modbus RTU read-holding-registers request (function code 03)** for registers 40001 to 40020 and waits up to 1 second for the reply. All traffic on a bus is serialised with a lock, because RS-485 is half duplex. Chapter 7 lists every register and Chapter 8 shows the request frame in hexadecimal.

2.4 How temperature data is collected

1. The controller answers with 20 register values. The node checks the identity registers (40001 must be 7507 and 40003 must be 115) so that a wrong address or wrong device can never produce plausible-looking data.
2. The decoder converts the raw control temperature (register 40006) using the resolution register rS (40005): rS = 0 means a scale of 0.1 °C, rS = 1 means 1 °C. The value 1001 means probe fail low and 1002 probe fail high; no temperature is reported in those cases.
3. The node builds one **reading record** with a unique ID, the acquisition time (UTC), the decoded values, the raw registers and a quality label. A failed poll becomes a record with `comm_status` other than `ok` and empty temperature.
4. The record is committed to the local buffer `queue.db` before anything else happens.

2.5 How the node transfers data to the server

A separate *uploader* thread takes the oldest pending records (up to 200 at a time) and sends them as JSON in an HTTP POST to `/api/v1/readings`, authenticated with the node ID and a secret token. The server stores the batch and answers per record *accepted*, *duplicate* or *rejected*. Only then does the node mark them *delivered*. If the server cannot be reached, the records stay *pending* and the uploader retries with a growing delay (2 s up to 300 s). A *heartbeat* every 30 s reports the node's health, and the same connection is used to fetch configuration and commands. Details: Chapter 8.

2.6 How the server processes and stores data

- Every record is validated (UUID, time, allowed values, temperature between -100 and 200 °C or empty). Invalid records are rejected individually.
- Records already stored (same UUID) are answered as *duplicate*, which makes retries harmless.
- Records are stored in `temperature_readings` and the controller's latest state is updated. Fault and alarm periods are derived from the fault register.
- A readings batch arriving more than 120 s after acquisition is flagged *backfilled* (delivered late after an outage).
- A background monitor marks a node *offline* when it has not been heard from for 120 s and records the event in the audit trail.

2.7 How the website retrieves and displays data

The website reads the database through the server. Pages such as the Dashboard and Monitoring poll the server's `/ui-api/dashboard` and `/ui-api/series` endpoints every *Dashboard refresh* interval (10 s by default) and update without reloading. Each controller is shown with an explicit status (Normal, Check data, Controller not responding, Stale data, Node offline, ...) so that an old value is never presented as live (Chapter 13).

2.8 Network layout

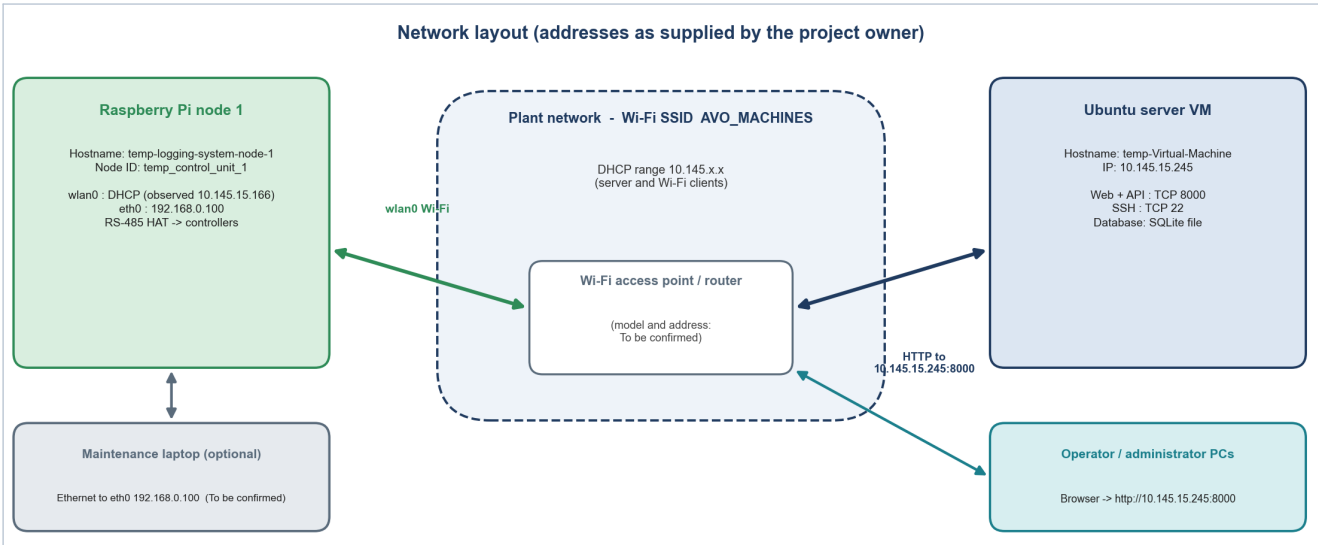


Figure 2-2 *- Network layout with the addresses supplied by the project owner. The Wi-Fi address of the Raspberry Pi is assigned by DHCP and can change.*

2.9 Hardware configuration

The photographs supplied by the project owner show the node hardware: a Raspberry Pi 5 in a black ventilated metal enclosure, with a Waveshare-style 2-channel RS-485 HAT mounted above the enclosure on brass standoffs and connected by a 40-way ribbon cable.

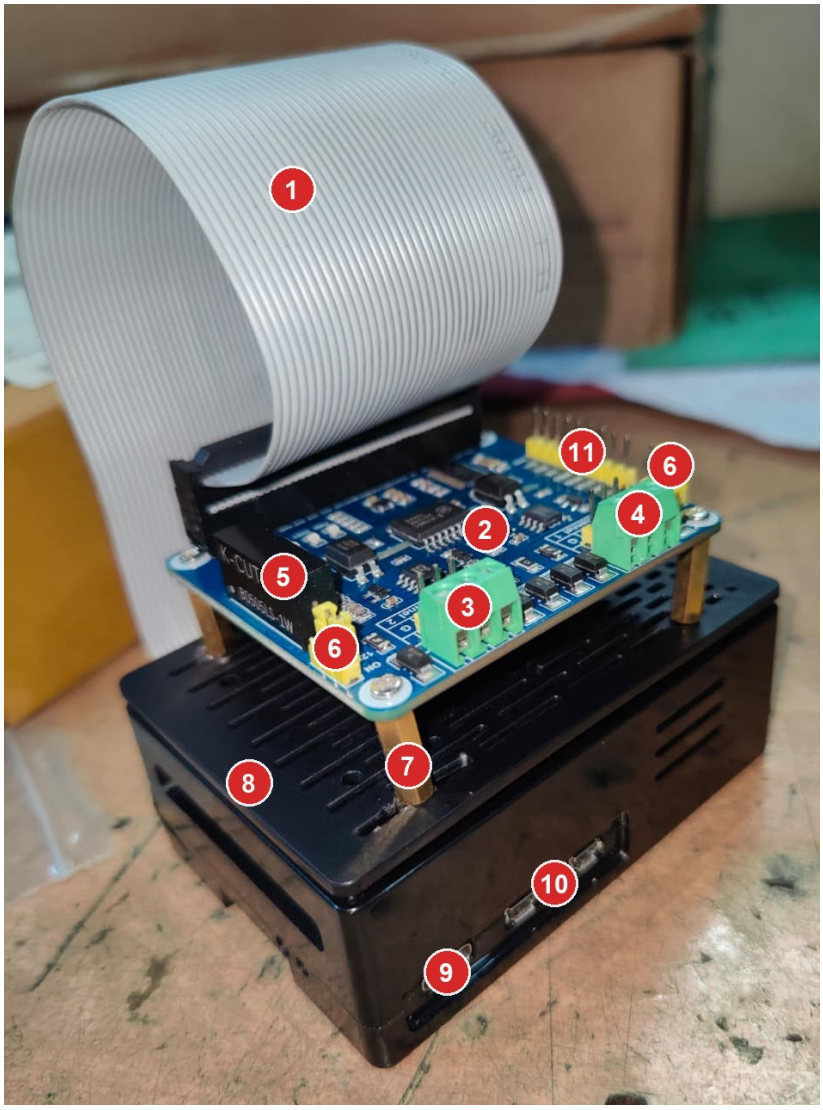


***Figure 2-3** - Node hardware overview (photograph supplied by the owner, call-outs added).*

No.	What is shown	Explanation
1	40-way ribbon cable	Carries the Raspberry Pi's 40-pin GPIO header (inside the enclosure) up to the HAT. The HAT is therefore powered and controlled through this cable.
2	Interface circuitry of the HAT	Converts between the Pi's SPI interface and the two RS-485 channels. The chip markings are not legible in the photograph. The device names `/dev/ttySC0` and `/dev/ttySC1` used by the project indicate an SPI-UART bridge driver. Exact HAT model: **To be confirmed**.
3	Terminal block, front (silkscreen *Channel 2*)	Screw terminals for one RS-485 channel. The project uses `/dev/ttySC1`, which the example configuration describes as the HAT's second channel.
4	Terminal block, rear (*Channel 1*)	Second RS-485 channel. Not used by the current configuration.
5	Black module marked `K-CUT B0505LS-1W`	A DC-DC converter module. Its presence indicates that the RS-485 side of the HAT has its own isolated supply; verify against the HAT documentation.
6	Yellow jumpers	Configuration jumpers of the HAT (one near the DC-DC module, one beside each terminal block). Their functions are **To be confirmed** from the HAT documentation. Do not move them without checking.
7	Brass standoffs	Mechanical spacers holding the HAT above the enclosure.
8	Black ventilated metal enclosure	Holds the Raspberry Pi 5.
9	USB-C connector on the enclosure front	Power input of the Raspberry Pi.

No.	What is shown	Explanation
10	Micro-HDMI connectors	Two video outputs, useful for a temporary monitor during setup (Chapter 3).
11	9-pin yellow header	Pin header on the HAT; its function is **To be confirmed** from the HAT documentation.

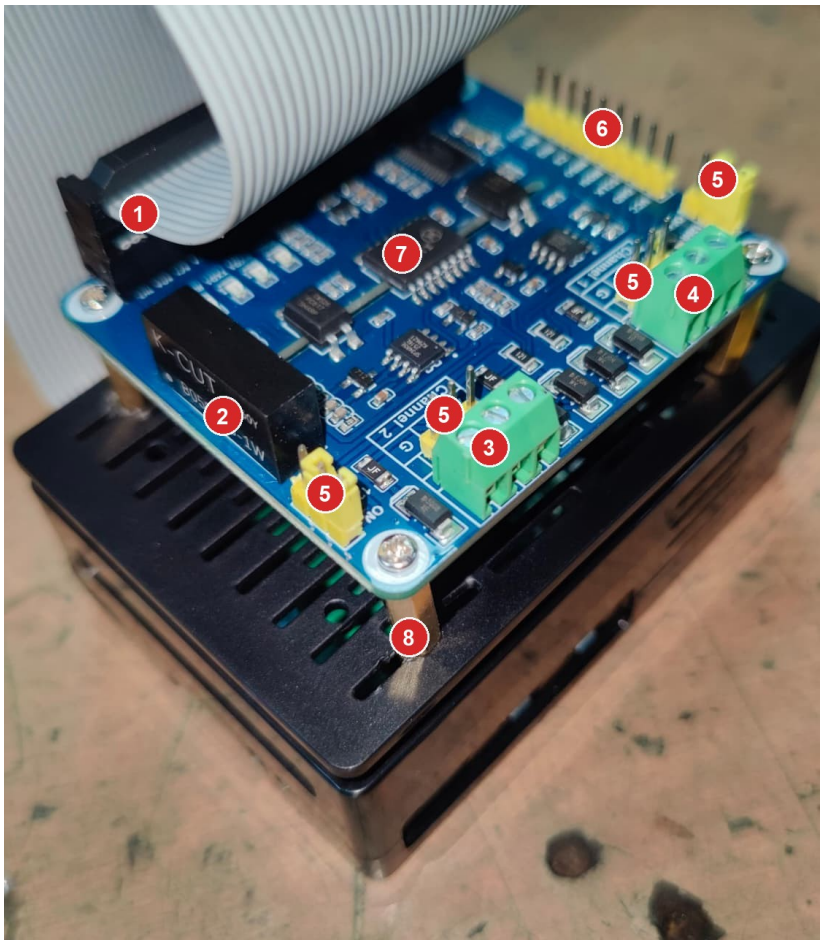


Figure 2-4 *- Close-up of the RS-485 HAT (photograph supplied by the owner, call-outs added).*

No.	What is shown	Explanation
1	Ribbon-cable socket	40-way connector on the HAT end of the ribbon cable.
2	DC-DC module	`K-CUT B0505LS-1W` (legible in the photograph).
3	Terminal block with silkscreen *Channel 2*	The silkscreen near the terminals carries letters such as G. The full order of the three screw terminals is **To be confirmed** on the board before wiring (Chapter 7).
4	Terminal block of *Channel 1*	Unused.
5	Jumper positions (four marked)	Pin headers with yellow jumpers beside the DC-DC module, beside each terminal block and at the top right edge.
6	9-pin header	Unlabelled in the photograph.
7	Largest integrated circuit	Marking not legible. Presumably the UART bridge; **To be confirmed**.
8	Standoff and mounting screw	Mechanical.



Figure 2-5 *- The Raspberry Pi 5 board inside its enclosure (photograph supplied by the owner, call-outs added).*

No.	What is shown	Explanation
1	40-pin GPIO header with the ribbon-cable connector	The HAT's signals (SPI, power, ground, interrupt) reach the Pi through this header.
2	USB-C power input (J1)	Power the Raspberry Pi with a suitable supply (the official Raspberry Pi 5 supply is 5 V / 5 A; this is general Raspberry Pi guidance, not a project requirement).
3	RTC battery connector (BAT)	Optional battery connector for the Pi 5's real-time clock. No battery is visible in the photograph. Without one the clock depends on network time (NTP); see Chapter 8.
4	Micro-HDMI 0 and 1	Video outputs.
5	Gigabit Ethernet (RJ45)	Wired network; `eth0`, address 192.168.0.100 as supplied by the owner.
6	USB ports (two stacks)	USB 3.0 (blue) and USB 2.0. Usable for a keyboard during setup.
7	PCIe connector	Not used by this project.
8	Camera / display connectors	Not used by this project.
9	PoE header (J14)	Not used by this project.
10	Fan connector	Not used by this project.
11	Enclosure frame	The Raspberry Pi is mounted in the black enclosure.

2.10 Where the key information is

NOTE

What happens when the server or the Raspberry Pi is shut down for an hour? In short: if the *server* is down, nothing is lost: the Pi keeps recording and uploads everything afterwards. If the *Raspberry Pi* is down (or both), the hour is **not recorded**, because the Pi is the only device that reads the controllers; the gap is reported and shown in the website. The full explanation with timings, screens and a real example from your audit trail is in Chapter 8.

CHAPTER 3

Raspberry Pi Initial Configuration

This chapter prepares a **new** Raspberry Pi so that it is ready to receive the node software (Chapter 5). Perform it once per node. The Raspberry Pi in this installation is *temp-logging-system-node-1*.

3.1 What you need (Technician)

- Raspberry Pi 5 in its enclosure, with a suitable USB-C power supply (the official 5 V / 5 A supply is the general recommendation for the Pi 5).
- A good-quality microSD card or USB SSD. The project notes that cheap cards and power cuts are the most common cause of Raspberry Pi failures (`docs/installation-raspberry-pi.md`).
- The 2-channel RS-485 HAT with its 40-way ribbon cable (Chapter 2, Figures 2-3 to 2-5).
- A Windows computer with the Raspberry Pi Imager program, and either the plant Wi-Fi (SSID `AVO_MACHINES`) or an Ethernet cable to a network that reaches the server.
- Optional for first setup: micro-HDMI-to-HDMI cable, monitor and USB keyboard.

3.2 Prepare Raspberry Pi OS (Technician)

1. Start **Raspberry Pi Imager** on the Windows computer.
2. Choose **Raspberry Pi 5** as the device and **Raspberry Pi OS Lite (64-bit)** (Raspberry Pi OS **Bookworm or newer**) as the operating system. The node software needs Python 3.10 or newer, and pymodbus 3.11 does not support older Python versions.
3. Choose the storage (the microSD card or SSD) and click **Next**, then **Edit settings** (OS customisation).
4. On the *General* tab set: **hostname** `temp-logging-system-node-1`; **username** `pi` and the **password** from the credentials section; **Wi-Fi** SSID `AVO_MACHINES` with its password and your Wi-Fi country; **time zone** `Asia/Kolkata`.
5. On the *Services* tab enable **SSH** with password authentication.
6. Save, confirm, and write the image. Wait until the Imager reports success, then remove the card.

> **NOTE**
> Current Raspberry Pi OS versions no longer create a default `pi` user. The user name and password you enter in the Imager become the only account, so enter them carefully.

3.3 First boot and connection (Technician)

1. With the Pi **powered off**, check that the ribbon cable and the HAT are fitted as in Figure 2-3 (the HAT is not needed for this chapter, but never connect or disconnect the ribbon cable while the Pi is powered).
2. Insert the card. Connect an Ethernet cable if you will use the cable first. Optionally connect a monitor and keyboard to the micro-HDMI and USB ports.
3. Connect power. The first boot takes one to two minutes because the system expands itself and applies your settings.
4. Find the Pi's address: the Wi-Fi router's client list, or from Windows `ping temp-logging-system-node-1.local` (this works only if name resolution is available on your network; **To be confirmed**). With an Ethernet cable the owner's address `192.168.0.100` is expected (**To be confirmed** whether it is fixed in the Pi or reserved in the router).

3.4 Log in over SSH (Technician)

On the Windows computer (use the current Wi-Fi address instead if you connect over Wi-Fi)

```
ssh pi@192.168.0.100
```

Answer `yes` to the fingerprint question, then type the password. The prompt changes to something like `pi@temp-logging-system-node-1:~ $`. All commands in the rest of this chapter are typed in this SSH session.

3.5 Update the operating system (Technician)

On the Raspberry Pi

```
sudo apt update
sudo apt full-upgrade -y
sudo reboot
```

`sudo` runs a command with administrator rights. `apt update` refreshes the list of available packages, `apt full-upgrade -y` installs all updates (this can take several minutes), and `reboot` restarts the Pi. Wait one minute and log in again.

3.6 Set and check the host name

On the Raspberry Pi

```
hostnamectl
sudo hostnamectl set-hostname temp-logging-system-node-1
```

`hostnamectl` shows the current *Static hostname*. The second command is needed only if it differs. The host name is only the operating-system name; the application name of the node (**node ID**, `temp_control_unit_1`) is set separately in the node configuration (Chapter 5).

3.7 Set the time zone and check time synchronisation

On the Raspberry Pi

```
sudo timedatectl set-timezone Asia/Kolkata
timedatectl
```

Example output - the last two lines matter

```
               Local time: Sat 2026-11-07 13:30:12 IST
           Universal time: Sat 2026-11-07 08:00:12 UTC
                 Time zone: Asia/Kolkata (IST, +0530)
System clock synchronized: yes
              NTP service: active
```

IMPORTANT

The node stores all times in UTC and flags every reading taken while the clock is not synchronised (`clock_unsynchronized`, quality *suspect*). Make sure `System clock synchronized: yes` appears after the Pi has been online for a few minutes. The setup script enables `systemd-timesyncd` for this reason (`scripts/setup_raspberry_pi.sh`).

3.8 Enable the RS-485 HAT (Technician)

The example configuration uses the serial port `/dev/ttySC1`. Such ports exist only when the HAT's driver is enabled in the Raspberry Pi boot configuration. This step is hardware-specific and is **not part of the project's source code**, so it is marked **To be confirmed**: follow the documentation of your exact HAT model and revision.

TO BE CONFIRMED - HAT driver setting
For Waveshare 2-channel RS-485 HATs based on an SPI-UART bridge, the vendor documentation generally asks for SPI to be enabled and an overlay line such as `dtoverlay=sc16is752-spi1,int_pin=24` to be added to `/boot/firmware/config.txt` on current Raspberry Pi OS, followed by a reboot. Confirm the exact line for your HAT before using it. In this installation the ports already work (`/dev/ttySC1` is used by `dev_tools/read_temp_from_controllert.py`), so normally nothing needs to be changed on a Pi that has already been commissioned.

Check on the Raspberry Pi whether the HAT ports exist

```
ls -l /dev/ttySC*
dmesg | grep -i -E 'sc16|spi' | tail
```

`ls -l /dev/ttySC*` should list `/dev/ttySC0` and `/dev/ttySC1` (one per RS-485 channel). If nothing is listed the driver is not loaded: re-check the HAT documentation, the ribbon cable seating and the boot configuration.

3.9 Install Python and other packages

On the Raspberry Pi

```
python3 --version
sudo apt install -y python3 python3-venv python3-pip rsync sqlite3 git curl
```

`python3 --version` must show 3.10 or newer. The next command installs Python, its virtual-environment and pip tools, `rsync` (used by the installer), the `sqlite3` command-line tool (used in this manual to inspect the buffer), `git` and `curl`. The node's Python libraries (`pymodbus`, `pyserial`, `requests`, `PyYAML` and their dependencies) are installed automatically into a private virtual environment by the installer in Chapter 5; you do not install them by hand.

3.10 Verify network connectivity and the server

On the Raspberry Pi

```
ping -c 4 10.145.15.245
curl -s http://10.145.15.245:8000/api/v1/ping
```

Example output

```
PING 10.145.15.245 (10.145.15.245) 56(84) bytes of data.
64 bytes from 10.145.15.245: icmp_seq=1 ttl=63 time=3.2 ms
...
4 packets transmitted, 4 received, 0% packet loss
{"server_time":"2026-11-07T08:01:30.123456Z","status":"ok","version":"1.0.0"}
```

`ping` tests that the server answers on the network (0 % packet loss expected). The `curl` command calls the server's unauthenticated health endpoint `/api/v1/ping`; a reply containing `"status":"ok"` proves that the Pi can reach the application itself, not just the machine. If `ping` works but `curl` does not, the server application is not running or a firewall blocks TCP port 8000 (Chapters 9 and 15).

VERIFIED IN SOURCE
`/api/v1/ping` is the only node-API route that does not require authentication (`server/app/routes/api.py`).

CHAPTER 4

Raspberry Pi Network Configuration

The node only ever **initiates** connections to the server (`docs/architecture.md`). It therefore needs a working route to `10.145.15.245:8000`, by Wi-Fi, by Ethernet, or both. This chapter treats the two interfaces separately. The commands use NetworkManager (`nmcli`), the default on Raspberry Pi OS Bookworm.

4.1 Wi-Fi (interface wlan0)

List the interfaces and the available Wi-Fi networks

On the Raspberry Pi

```
nmcli device status
nmcli device wifi list
```

Command	What it does	What you should see
`nmcli device status`	Lists every network interface and whether it is connected.	A table with `DEVICE TYPE STATE CONNECTION`. `wlan0 wifi connected AVO_MACHINES` and/or `eth0 ethernet connected Wired connection 1`.
`nmcli device wifi list`	Scans for Wi-Fi networks in range.	One row per access point with SSID, channel, signal strength and security. `AVO_MACHINES` must be in the list; a `*` marks the one in use. Weak signal (below about 40) causes dropouts.
`nmcli connection show`	Lists the saved connection profiles.	Profiles such as `AVO_MACHINES` (type wifi) and `Wired connection 1` (type ethernet).

Connect to a Wi-Fi network

On the Raspberry Pi (prompts for the password)

```
sudo nmcli --ask device wifi connect "AVO_MACHINES" ifname wlan0
```

`--ask` makes `nmcli` ask for the Wi-Fi password so that it does not remain in the shell history. On success it prints `Device 'wlan0' successfully activated`. The profile is saved and the Pi reconnects automatically after every reboot.

Verify the connection, address, interface and gateway

On the Raspberry Pi

```
nmcli -f GENERAL.STATE,IP4.ADDRESS,IP4.GATEWAY,IP4.DNS device show wlan0
ip -4 addr show wlan0
hostname -I
ip route
```

Example output (addresses are illustrative, apart from the observed 10.145.15.166)

```
GENERAL.STATE:                          100 (connected)
IP4.ADDRESS[1]:                         10.145.15.166/24
IP4.GATEWAY:                            10.145.15.1
default via 10.145.15.1 dev wlan0 proto dhcp src 10.145.15.166 metric 600
10.145.15.0/24 dev wlan0 proto kernel scope link src 10.145.15.166 metric 600
```

Command	What it does	What you should see
`nmcli ... device show wlan0`	Shows the state, address, gateway and DNS of the Wi-Fi interface.	`100 (connected)` and an address in the Wi-Fi network.
`ip -4 addr show wlan0`	Shows the IPv4 address of the interface.	A line `inet 10.145.x.x/24`. The address after `inet` is your Wi-Fi IP.

Command	What it does	What you should see
`hostname -I`	Lists all IP addresses of the Pi on one line.	For example `192.168.0.100 10.145.15.166` when both cable and Wi-Fi are up.
`ip route`	Shows the routing table, including the **default gateway** (`default via ...`).	One `default via <router>` line per interface that has a gateway. The route to `10.145.15.245` normally follows the Wi-Fi network.

Test connectivity to the server

On the Raspberry Pi

```
ping -c 4 10.145.15.245
curl -s http://10.145.15.245:8000/api/v1/ping
```

Turn off Wi-Fi power saving

The project recommends turning Wi-Fi power saving off because it causes dropouts (`docs/installation-raspberry-pi.md`). The project's one-line command selects the first active connection; naming the Wi-Fi profile is safer when an Ethernet cable is also connected:

On the Raspberry Pi

```
sudo nmcli connection modify "AVO_MACHINES" 802-11-wireless.powersave 2
sudo nmcli connection up "AVO_MACHINES"
```

Troubleshoot Wi-Fi connection failures

Symptom	Check	Fix
`AVO_MACHINES` not in `nmcli device wifi list`	Distance, antenna, access point switched off, 5 GHz-only network.	Move closer; confirm the access point; list again.
Connection fails with a secrets or authentication error	Wrong password; password changed on the router.	Repeat the connect command with the correct password, or delete the profile with `sudo nmcli connection delete "AVO_MACHINES"` and create it again.
Connected but no `inet` address	DHCP failure.	`sudo nmcli connection down "AVO_MACHINES" && sudo nmcli connection up "AVO_MACHINES"`; check the router's DHCP pool.
Address works, `ping 10.145.15.245` fails	Different subnet without a route, client isolation on the access point, server down.	`ip route`; ping the gateway; ask the network administrator.
`ping` works, `curl .../api/v1/ping` fails	Server application stopped, or firewall blocks port 8000.	See Chapter 9. On the server: `sudo ufw status`.
Node drops offline periodically	Weak signal or Wi-Fi power saving.	Check the signal on the node page of the website (better than -70 dBm); disable power saving (above); consider Ethernet.
Pi address keeps changing	DHCP lease changes.	Ask for a DHCP reservation in the router (preferred), or configure a static address (below).

Configure a static Wi-Fi address (only if a DHCP reservation is not possible)

On the Raspberry Pi - replace the placeholders

```
sudo nmcli connection modify "AVO_MACHINES" ipv4.method manual \
     ipv4.addresses <ip>/<prefix> ipv4.gateway <router-ip> ipv4.dns <dns-ip>
sudo nmcli connection up "AVO_MACHINES"
```

TO BE CONFIRMED

The subnet prefix, the router address and a free fixed address for the Wi-Fi network `10.145.x.x` are not known from the supplied material. Obtain them from the network administrator before using a static address. A DHCP reservation in the router avoids this.

4.2 Ethernet (interface eth0)

The Ethernet address supplied by the owner is `192.168.0.100`. Whether it is configured in the Pi itself or reserved by a router is **To be confirmed**. To check what is configured:

On the Raspberry Pi

```
nmcli -t -f NAME,DEVICE,TYPE connection show
nmcli -f ipv4.method,ipv4.addresses,ipv4.gateway connection show "Wired connection 1"
ip -4 addr show eth0
```

`ipv4.method auto` means the address comes from DHCP; `manual` means a fixed address is stored in the Pi. To fix the address at `192.168.0.100`:

On the Raspberry Pi (no gateway on the cable)

```
sudo nmcli connection modify "Wired connection 1" ipv4.method manual \
     ipv4.addresses 192.168.0.100/24 ipv4.never-default yes
sudo nmcli connection up "Wired connection 1"
```

Avoid two default routes

`ipv4.never-default yes` stops the Ethernet connection from becoming a second default route. With two default routes the Pi could send its traffic to the server over the wrong interface. Give the cable a gateway only if the server is reached through it (then the `24` prefix and a gateway must match that network - **To be confirmed**).

Difference between the two addresses. The Wi-Fi address (`10.145.x.x`, assigned by DHCP, can change) is the one the node normally uses to reach the server. The Ethernet address (`192.168.0.100`) belongs to a different network and is the stable address for commissioning and maintenance with a cable. Both can be active at the same time; SSH works on either.

CHAPTER 5

Uploading and Running the Raspberry Pi Program

This chapter installs the node software on the Raspberry Pi, configures it, runs it by hand, and proves that data reaches the server. Chapter 6 then makes it start automatically. The installer is the project's own script `scripts/setup_raspberry_pi.sh`.

> **IMPORTANT**
> Register the node on the website **before** starting it, because the one-time token that the node needs is created at registration (step 3). An unregistered node keeps recording locally but its data is not accepted by the server.

5.1 Files that make up the node

Location in the project	Purpose
`raspberry_pi_node/src/`	The Python application (entry point `python -m src.main`).
`raspberry_pi_node/config.example.yaml`	Template of the local configuration file.
`raspberry_pi_node/temperature-node.service`	The systemd unit (Chapter 6).
`raspberry_pi_node/requirements.txt`	Pinned Python dependencies.
`scripts/setup_raspberry_pi.sh`	Installer: creates user, folders, virtual environment, configuration and service.
`docs/`	Documentation (copied to the Pi by the installer).
`dev_tools/read_temp_from_controllert.py`	Stand-alone test that reads one temperature from a controller.

After installation the Raspberry Pi has this layout (created by the script):

Path on the Pi	Content
`/opt/temperature-node/raspberry_pi_node/`	The application code.
`/opt/temperature-node/venv/`	The private Python virtual environment.
`/etc/temperature-node/config.yaml`	Local configuration (edit this).
`/etc/temperature-node/node.env`	`TEMP_NODE_TOKEN=...` (secret, mode 640).
`/var/lib/temperature-node/`	`queue.db` (local buffer) and `remote_config.json` (settings pushed from the server).
`/var/log/temperature-node/node.log`	Rotating log file (5 MB x 10 files); the same log also goes to journald.
`/etc/systemd/system/temperature-node.service`	The service definition.

5.2 Step 1 - Connect with SSH (Technician)

Follow Section 3.4. All commands below are typed on the Raspberry Pi unless a heading says *On the Windows computer*.

5.3 Step 2 - Create the project folder and upload the files (Technician)

The installer copies from the folder that contains `raspberry_pi_node/`, `scripts/` and `docs/`. Upload those three folders (and `dev_tools/` for the quick controller test). Choose one method.

Method A: tar and scp from Windows (recommended)

On the Windows computer (PowerShell)

```
cd "C:\Users\gokul\Documents\projects\Avo AC Unit Logging System"
cd avo_ac_loggin_system_with_rashberrypi
tar -czf node.tgz --exclude=__pycache__ --exclude=.venv --exclude=state `
    raspberry_pi_node scripts docs dev_tools
scp node.tgz pi@192.168.0.100:/home/pi/
```

On the Raspberry Pi

```
mkdir -p ~/temperature-logging-system
tar -xzf ~/node.tgz -C ~/temperature-logging-system
ls ~/temperature-logging-system
```

`tar -czf` packs the folders into one compressed file (excluding caches and local virtual environments), `scp` copies it to the Pi, and `tar -xzf` unpacks it into `~/temperature-logging-system`. `ls` must show `docs`, `raspberry_pi_node` and `scripts`. Do **not** upload `raspberry_pi_node/config.yaml` from a development PC; the Pi has its own configuration.

Method B: git clone, or a graphical copy

If the repository is reachable from the Pi, `git clone <repository-url> ~/temperature-logging-system` gives the same result. A graphical SFTP client such as WinSCP can also copy the three folders to `/home/pi/temperature-logging-system` (protocol SFTP, port 22, user `pi`).

5.4 Step 3 - Register the node on the website (Administrator)

1. Sign in to `http://10.145.15.245:8000` as an Administrator (Chapter 11).
2. Open **Nodes** and click **+ Register node**.
3. Enter the **Node ID** `temp_control_unit_1`, a friendly name, site and equipment, the serial port `/dev/ttySC1`, baud rate `9600`, parity `N`, and one row per controller (Controller ID, name, slave ID). Give a reason (for example *Initial registration*) and save.
4. The next page shows the **token once**. Copy it immediately. It cannot be shown again; if lost, issue a new token on the node page.

5.5 Step 4 - Run the installer (Technician)

On the Raspberry Pi

```
cd ~/temperature-logging-system
sudo bash scripts/setup_raspberry_pi.sh
```

The script (`scripts/setup_raspberry_pi.sh`) does the following; it can be run again later to upgrade the software without losing configuration or data:

1. Checks that Python 3.10 or newer is installed, installs `python3-venv`, `python3-pip` and `rsync`.
2. Creates the system user `tempnode` (no login shell) and adds it to the group `dialout` so it may open the serial ports.
3. Creates `/opt/temperature-node`, `/etc/temperature-node`, `/var/lib/temperature-node`, `/var/log/temperature-node`, and copies the code and documentation.
4. Creates the virtual environment `/opt/temperature-node/venv` and installs `requirements.txt` into it.
5. Creates `/etc/temperature-node/config.yaml` from the example and an empty `node.env` if they do not exist yet.
6. Installs, reloads and **enables** the systemd service `temperature-node.service`, and enables time synchronisation.
7. Validates the configuration; if it is valid it starts the service, otherwise it tells you what to edit.

5.6 Step 5 - Edit the configuration (Technician)

On the Raspberry Pi (Ctrl+O, Enter to save; Ctrl+X to leave)

```
sudo nano /etc/temperature-node/config.yaml
```

The following is a complete configuration for this installation (one RS-485 channel with two controllers). It is based on `config.example.yaml`; the controller IDs `Temp_Control_1` and `Temp_Control_2` are the IDs that appear in the supplied Audit Trail screenshot.

/etc/temperature-node/config.yaml

```
node:
  id: temp_control_unit_1        # must match the Node ID registered on the website
  name: Temp Control Unit 1
  site: ""
  equipment: ""
server:
  url: http://10.145.15.245:8000
  token: null                    # leave null; the token goes into node.env
  verify_tls: true
  request_timeout_s: 10
  upload_interval_s: 5
  batch_size: 200
  heartbeat_interval_s: 30
  config_sync_interval_s: 60
  backoff_initial_s: 2
  backoff_max_s: 300
buses:
  - name: rs485-1
    port: /dev/ttySC1            # HAT channel used in this installation
    baudrate: 9600               # 9600 | 19200 | 38400  (controller parameter bd)
    parity: N                    # N | E | O            (controller parameter PAr)
    bytesize: 8
    stopbits: 1
    timeout_s: 1.0
    retries: 2
    inter_request_delay_ms: 50
controllers:
  - id: Temp_Control_1
    name: Temp Control 1
    model: SZ-7526TX-REA
    bus: rs485-1
    slave_id: 1                  # controller parameter did
    enabled: true
    poll_interval_s: 10
    parameter_read_interval_s: 3600
    include_raw_registers: true
    comm_loss_threshold: 3
    register_overrides: {address_offset: 0, read_strategy: auto}
  - id: Temp_Control_2
    name: Temp Control 2
    model: SZ-7526TX-REA
    bus: rs485-1
    slave_id: 2
    enabled: true
    poll_interval_s: 10
    parameter_read_interval_s: 3600
    include_raw_registers: true
    comm_loss_threshold: 3
    register_overrides: {address_offset: 0, read_strategy: auto}
buffer:
  path: /var/lib/temperature-node/queue.db
  retention_days_delivered: 7
  max_pending_records: 2000000
  full_behavior: drop_oldest
```

```
  min_free_disk_mb: 100
logging:
  level: INFO
  file: /var/log/temperature-node/node.log
  format: text
  max_bytes: 5000000
  backup_count: 10
  diagnostic_registers: false
safety:
  allow_remote_control: false
state_dir: /var/lib/temperature-node
simulation: false
```

Live values: To be confirmed
The values `/dev/ttySC1`, 9600, N, slave IDs 1 and 2 and the poll interval come from `config.example.yaml` and from the tool `dev_tools/read_temp_from_controllert.py`. The **live** configuration of your Pi is the merge of this file and the settings the server pushes (`/var/lib/temperature-node/remote_config.json`). The audit trail in Figure 8-4 shows that port and controller changes were made from the website. Check the effective values on the node page (Nodes > node) or with the command in Step 7. Real values on the Pi: **To be confirmed**.

Meaning of the important keys

Key	Meaning	Limits (checked at start-up)
`node.id`	Application name of the node; must equal the Node ID on the website.	Letters, digits, `_ . -`, up to 64 characters.
`server.url`	Address of the server. Use `https://` only behind a TLS proxy.	--
`server.request_timeout_s`	Time to wait for the server.	1 to 120 s
`server.upload_interval_s`	Upload period when there is no backlog.	1 to 3600 s
`server.heartbeat_interval_s`	Health report period.	5 to 3600 s
`buses[].port / baudrate / parity`	Serial port and line settings. Must equal the controllers' `bd` and `PAr`.	baud 9600/19200/38400; parity N/E/O; 8 data bits; 1 stop bit
`buses[].timeout_s / retries / inter_request_delay_ms`	Reply timeout, extra attempts after a failure, silent time between requests.	0.05-10 s; 0-10; 0-5000 ms
`controllers[].slave_id`	Modbus address of the controller (`did`).	1 to 250
`controllers[].poll_interval_s`	Seconds between polls of that controller.	1 to 3600 s
`controllers[].comm_loss_threshold`	Consecutive failed polls before *communication lost* is raised.	1 to 100
`buffer.retention_days_delivered`	Days that already-delivered records stay in the local buffer.	default 7
`buffer.max_pending_records` / `full_behavior`	Capacity for undelivered records and what to do when full.	default 2 000 000; `drop_oldest` or `stop_collecting`
`safety.allow_remote_control`	Allows alarm mute / fault reset / log reset / defrost commands from the server. Can only be enabled here.	keep `false` unless the site approves
`simulation`	`true` only when polling the simulator; labels every reading *simulated*.	keep `false`

5.7 Step 6 - Store the token (Technician)

On the Raspberry Pi
```
sudo nano /etc/temperature-node/node.env
```

/etc/temperature-node/node.env
```
# Node API token shown once by the server when the node was registered
TEMP_NODE_TOKEN=<paste the token here>
```

The file is readable only by root and the `tempnode` group (mode 640). Do not copy the token into `config.yaml`, e-mails or chat messages.

5.8 Step 7 - Validate the configuration

On the Raspberry Pi

```
cd /opt/temperature-node/raspberry_pi_node
sudo -u tempnode env $(sudo grep ^TEMP_NODE_TOKEN /etc/temperature-node/node.env) \
  /opt/temperature-node/venv/bin/python -m src.main \
  --config /etc/temperature-node/config.yaml --check-config
```

This starts the program only far enough to validate the settings. `configuration OK (remote version N)` means success. `INVALID configuration:` is followed by a list of the problems (for example a serial port that does not exist, or a baud rate that is not 9600/19200/38400).

To read the settings the node is really using, including those pushed from the website:

On the Raspberry Pi

```
sudo cat /var/lib/temperature-node/remote_config.json
```

5.9 Step 8 - Check the controller communication before going live (Technician)

The project provides two read-only checks. Both open the serial port, so **stop the service first**; only one program can use a port.

Full read-only validation (On the Raspberry Pi)

```
sudo systemctl stop temperature-node
cd /opt/temperature-node/raspberry_pi_node
sudo -u tempnode /opt/temperature-node/venv/bin/python -m src.validate_controller \
  --port /dev/ttySC1 --slave 1 --baud 9600 --parity N
```

Quick one-line temperature test (On the Raspberry Pi)

```
sudo systemctl stop temperature-node
cp ~/temperature-logging-system/dev_tools/read_temp_from_controllert.py ~/
/opt/temperature-node/venv/bin/python ~/read_temp_from_controllert.py
```

The quick test reads registers 40001-40006 of slave 1 on `/dev/ttySC1` at 9600 baud, no parity, checks the identity values 7507 and 115 and prints `Temperature: 18.0 °C` (or `Probe fail (low)` / `Probe fail (high)`). Edit `PORT`, `BAUD`, `PARITY` and `SLAVE_ID` at the top of the file if your values differ. If the controller's address, baud rate or parity are unknown, use `--scan` (add `--scan-ids 1-20`) to search. The full procedure and a record sheet are in `docs/hardware-validation.md`; Chapter 7 summarises them.

5.10 Step 9 - Run the program manually and watch it (Technician)

On the Raspberry Pi - stop with Ctrl+C

```
sudo systemctl stop temperature-node
cd /opt/temperature-node/raspberry_pi_node
sudo -u tempnode env $(sudo grep ^TEMP_NODE_TOKEN /etc/temperature-node/node.env) \
  /opt/temperature-node/venv/bin/python -m src.main --config /etc/temperature-node/config.yaml
```

Example start-up lines in the output

```
... temperature node temp_control_unit_1 v1.0.0 starting (config v5, simulation=False,
    remote control disabled)
... bus rs485-1 on /dev/ttySC1: polling ['Temp_Control_1', 'Temp_Control_2']
```

Use the right user

Run the program as the user `tempnode` (as shown), never as `pi` or `root`. Files created by another user in `/var/lib/temperature-node` can stop the service from starting later.

Press Ctrl+C to stop. When satisfied, start the service again: `sudo systemctl start temperature-node`.

5.11 Step 10 - Check the logs and that data reaches the server

On the Raspberry Pi (Ctrl+C to leave)

```
sudo journalctl -u temperature-node -f
sudo tail -f /var/log/temperature-node/node.log
```

- `server unreachable, buffering locally: ...` means the node cannot reach the server; the data is safe in the buffer (Chapter 8).
- `server rejected node credentials (check node id/token)` means a wrong token or node ID.
- `controller ... communication lost` or `poll failed (n in a row)` means a controller does not answer.

Count buffered readings (On the Raspberry Pi)

```
sudo sqlite3 /var/lib/temperature-node/queue.db "select status, count(*) from readings group by status;"
```

While everything is healthy, `pending` is close to 0 (a few records at most) and `delivered` grows. On the website the node must show **Online** and a recent *Last upload*, the controllers must appear on the Dashboard, and new rows must appear in **Data Logs** (Figure 12-1 and Figure 12-3).

5.12 Upgrading the node software

Copy the new project folders to the Pi (Step 2) and run `sudo bash scripts/setup_raspberry_pi.sh` again. The configuration, the token and the buffer are kept.

5.13 Common errors

Message or symptom	Cause	What to do
`port_error: cannot open serial port /dev/ttySC1`	HAT driver not loaded; wrong port; another program (for example your manual test) holds the port; `tempnode` not in `dialout`.	`ls -l /dev/ttySC*`; stop other users of the port; `id tempnode` must list `dialout`.
`INVALID configuration`	Typo or unsupported value in `config.yaml`.	Read the listed errors; fix; repeat `--check-config`.
`no server token configured: data is buffered locally until the node is registered`	`TEMP_NODE_TOKEN` is empty.	Paste the token into `node.env` and restart the service.
`server rejected node credentials`	Wrong token or node ID; token re-issued.	Install the current token (node page > Issue new token).
`server refused node: this node has been disabled`	Node disabled on the website.	Enable it on the node page. Data is still buffered on the Pi.
`timeout: no valid response`	Wrong slave ID, baud rate or parity, A/B swapped, no power on the controller.	Chapter 7; run `validate_controller --scan`.
Service not running after reboot	Service not enabled.	Chapter 6.

CHAPTER 6

Automatic Startup Configuration

The node software starts automatically through **systemd**, the Linux service manager. The installer in Chapter 5 already installs and enables the service; this chapter explains the unit file and the commands you use to control it.

VERIFIED IN SOURCE
The startup mechanism is `raspberry_pi_node/temperature-node.service`, installed as `/etc/systemd/system/temperature-node.service` and enabled by `scripts/setup_raspberry_pi.sh`. The service name is **temperature-node**.

6.1 The service file

/etc/systemd/system/temperature-node.service (as installed by the project)

```
[Unit]
Description=Temperature monitoring node (SZ-7526TX-REA Modbus RTU -> central server)
Documentation=file:///opt/temperature-node/docs/installation-raspberry-pi.md
After=network-online.target time-sync.target
Wants=network-online.target
[Service]
Type=notify
NotifyAccess=main
User=tempnode
Group=tempnode
SupplementaryGroups=dialout
WorkingDirectory=/opt/temperature-node/raspberry_pi_node
EnvironmentFile=-/etc/temperature-node/node.env
ExecStart=/opt/temperature-node/venv/bin/python -m src.main --config /etc/temperature-node/config.yaml
Restart=always
RestartSec=5
WatchdogSec=120
TimeoutStopSec=90
NoNewPrivileges=true
ProtectSystem=strict
ProtectHome=true
PrivateTmp=true
ReadWritePaths=/var/lib/temperature-node /var/log/temperature-node
ProtectKernelTunables=true
ProtectControlGroups=true
[Install]
WantedBy=multi-user.target
```

Setting	What it does
`After=network-online.target time-sync.target` / `Wants=network-online.target`	Start after the network is up (and after time synchronisation when available).
`Type=notify`, `NotifyAccess=main`	The program tells systemd when it is ready (`READY=1`) and then keeps reporting that it is alive.
`User=tempnode`, `Group=tempnode`, `SupplementaryGroups=dialout`	Run as the unprivileged service user, which may open the serial ports.
`WorkingDirectory=/opt/temperature-node/raspberry_pi_node`	Folder the program runs in (needed for `-m src.main`).
`EnvironmentFile=-/etc/temperature-node/node.env`	Loads `TEMP_NODE_TOKEN`; the leading `-` means a missing file is not an error.
`ExecStart=/opt/temperature-node/venv/bin/python -m src.main --config ...`	The command that is run: the virtual-environment Python, the application module and its configuration file.
`Restart=always`, `RestartSec=5`	If the program exits for any reason, systemd starts it again 5 seconds later.
`WatchdogSec=120`	The program signals every 15 s while all its worker threads are alive. If systemd hears nothing for 120 s it restarts the service.

Setting	What it does
`TimeoutStopSec=90`	On stop, wait up to 90 s for the program to finish cleanly before killing it.
`NoNewPrivileges`, `ProtectSystem=strict`, `ProtectHome`, `PrivateTmp`, `ReadWritePaths=...`	Hardening: the service can write only `/var/lib/temperature-node` and `/var/log/temperature-node`.
`WantedBy=multi-user.target`	When the service is *enabled*, it starts during a normal boot.

6.2 Commands (Technician)

Command	What it does
`sudo systemctl daemon-reload`	Makes systemd re-read unit files. Needed after you change `/etc/systemd/system/temperature-node.service`.
`sudo systemctl enable temperature-node`	Starts the service automatically at every boot (the installer already did this).
`sudo systemctl start temperature-node`	Starts it now.
`sudo systemctl stop temperature-node`	Stops it now (wait: up to 90 s). Required before running a controller test.
`sudo systemctl restart temperature-node`	Stops and starts it; needed after editing `config.yaml` or `node.env`.
`systemctl status temperature-node`	Shows whether it is `active (running)`, since when, and the last log lines.
`systemctl is-enabled temperature-node`	Prints `enabled` if it will start at boot.
`sudo journalctl -u temperature-node -f`	Follows the live log (Ctrl+C to leave). Use `--since "1 hour ago"` for history.
`sudo systemctl disable --now temperature-node`	Stops the service and prevents it from starting at boot.

Example: what a healthy service looks like

```
systemctl status temperature-node
```

Example output

```
● temperature-node.service - Temperature monitoring node (SZ-7526TX-REA Modbus RTU -> central server)
     Loaded: loaded (/etc/systemd/system/temperature-node.service; enabled; preset: enabled)
     Active: active (running) since Sat 2026-11-07 13:40:02 IST; 2h 3min ago
   Main PID: 812 (python)
     Status: "collecting"
```

6.3 Test the automatic start

1. Run `sudo reboot`.
2. After about two minutes, log in and run `systemctl status temperature-node`: it must say `active (running)` with a start time shortly after boot.
3. On the website the node should return to **Online** within a minute or two. A `node_started` event is written to the Audit Trail at every start (Figure 8-4).

6.4 Disable automatic startup

On the Raspberry Pi

```
sudo systemctl disable --now temperature-node
```

To bring it back: `sudo systemctl enable --now temperature-node`. A disabled node does not poll the controllers, so **no data is recorded** while it is off (Chapter 8).

CHAPTER 7

Temperature Controller Configuration

This chapter describes the controller that is read, exactly how the Raspberry Pi talks to it over RS-485, which registers are read, how the values are decoded, how to wire the bus and how to prove that the controller is responding. Every serial parameter and register in this chapter comes from `raspberry_pi_node/src/registers.py`, `modbus_client.py`, `decoder.py`, `config.example.yaml` and `docs/modbus-register-map.md`, and were checked against the two controller documents that are stored in the project folder next to the repository: `SZ-7526TX_0.pdf` (instruction manual) and `SZ-7526TX-REA_PRM.pdf` (register map).

7.1 The controller

Item	Value	Source
Make and model	PVR Controls **SZ-7526TX-REA** (software model name `SZ-7526TX-REA`)	`registers.py`, `README.md`
Documents behind the register map	*Product Register Map for SZ-7526TX-REA*, PVR/DSG/PRM Issue 01 Rev 00 (24-11-2023); *SZ-7526TX Instruction Manual* REV3 (23-11-2023)	`docs/modbus-register-map.md`; both PDFs are in the project folder beside the repository and were read for this manual
Protocol	Modbus RTU, half duplex, asynchronous	`docs/modbus-register-map.md`
Physical port	**TTL OUT (RS485)**: a **5-pin header** at the rear of the controller (instruction manual, *Suggested Wiring Diagram*); the register map calls it "TTL port compatible with RS 485 Slave". The manuals show **no RS-485 A/B screw terminals** and do not name the five pins, so a TTL-to-RS-485 interface is needed between the header and the bus. Type and pin-out of that interface in this installation: **To be confirmed**.	instruction manual p. 1; register map p. 1
Supply, outputs, probe	230 VAC ±10 %, 50-60 Hz (standard; others on request); compressor relay 8(3) A 250 VAC; alarm relay 5 A 250 VAC; NTC control probe; AUX digital input (potential free); IP65 front; operating 0 to 60 °C; accuracy ±1 °C; measuring range -45.0 to 100.0 °C	instruction manual, *Technical Data*
Number of controllers in this installation	Two on one bus: slave ID 1 (*Temp Control 1*) and slave ID 2 (*Temp Control 2*)	`config.example.yaml`; Figure 12-1

7.2 Communication parameters

Parameter	Value used by the software	Notes
Baud rate	9600 (allowed: 9600, 19200, 38400)	Controller parameter `bd`: 1 = 9600, 2 = 19200, 3 = 38400.
Data bits	8	Fixed by the controller.
Parity	None `N` (allowed: N, E, O)	Controller parameter `PAr`: 0 = none, 1 = even, 2 = odd.
Stop bits	1	Fixed by the controller.
Slave ID / Modbus address	1 and 2 in this installation (allowed 1 to 250)	Controller parameter `did`. `0` disables communication on the controller.
Serial port on the Pi	`/dev/ttySC1`	Waveshare HAT channel configured in this installation.
Function codes	03 (read holding registers) for polling; 01 (read coils) for diagnostics only; 05 (write coil) only for guarded commands	06 and 16 (write registers) are never used.

Parameter	Value used by the software	Notes
Registers per request	At most 50	Controller limit ("At a time only 50 registers are accessible"); the software refuses larger requests.
Reply timeout	1.0 s	`timeout_s`
Retries after a failure	2 (so up to 3 attempts per poll)	`retries`, with 0.2 s and 0.4 s pauses between attempts.
Silent time between requests	50 ms	`inter_request_delay_ms`
Communication interval	Every **10 s per controller**	`poll_interval_s`. The two controllers are offset by 0.5 s.
Status registers read	40001 to 40020 (20 registers) every poll	One request.
Parameter registers read	40031 to 40068 (38 registers) every hour and on request	`parameter_read_interval_s: 3600`; read-only.
"Communication lost" threshold	3 consecutive failed polls	`comm_loss_threshold`

Controller communication settings have no factory default
The instruction manual lists `did`, `bd` and `PAr` (index entries 25, 26 and 27) with **no factory value** ("-") and states for each that it "will not change on factory set", so restoring the factory defaults does not reset them. The values above are only the software's starting point. The real values must be read from the controller's Program Mode or found with `validate_controller --scan`. The software cannot write these parameters.

7.3 Set the controller's communication parameters (Technician)

`did`, `bd` and `PAr` are changed on the controller's own keypad in Program Mode. The procedure below follows the SZ-7526TX instruction manual (*User Interface*, *Index* and parameter descriptions 2 and 25-27):

1. If the keypad password `PA` (parameter 36, factory value 0) has been changed to something other than 0, you must enter it first; with `PA` = 0 Program Mode opens directly. If the keypad is locked (`LP` = 1) parameters can be viewed but not changed.
2. Touch and hold the **DOWN / PROGRAM** key for 2 seconds to enter Program Mode. The display flashes the first parameter, `P1`.
3. Scroll with the **UP** and **DOWN** keys to the parameter: `did` (Device ID), `bd` (baud rate) or `PAr` (parity).
4. Change the value with UP / DOWN and touch the **SET** key to store it. Values: `did` 1 to 250 (`0` disables communication); `bd` 1 = 9600, 2 = 19200, 3 = 38400; `PAr` 0 = none, 1 = even, 2 = odd.
5. Touch and hold the **DEFROST / RESET** key for 2 seconds to leave Program Mode. After 30 s without a key press the controller returns to the parameter view by itself and after a further 30 s to the main screen.

Rules:

- Every controller on the same RS-485 bus must have a **different** `did` and the **same** `bd` and `PAr`.
- After changing a controller's `did`, `bd` or `PAr`, update `slave_id`, `baudrate` and `parity` in `/etc/temperature-node/config.yaml` (or on the node's configuration page of the website) and restart the node service. The server records parameter changes it detects on the device (Audit Trail, `controller_parameters_changed`).
- Never set `did` to 0 on a controller that must be monitored: that switches its communication off.

7.4 Registers used to collect data

The Raspberry Pi reads registers 40001 to 40020 in **one** request every poll. The register map uses Modicon reference numbers (4xxxx); the Modbus frame carries the zero-based protocol address, which is the reference minus 40001 (register 40006 is sent as address 5).

Register	Name	Type	How the software uses it
40001	PVR Key	uint16	Identity check: must equal **7507**.
40002	Software revision	uint16	Reported as is.
40003	Controller identification code	uint16	Identity check: must equal **115**.
40004	Device (slave) ID	int16	Cross-check against the polled slave ID (`device_id_mismatch` otherwise).
40005	rS (resolution)	int16	0 → scale 0.1 °C; 1 → scale 1 °C; anything else → temperatures withheld (`unknown_resolution`).
40006	**Control temperature (display temperature)**	int16, signed	**The measured temperature**, multiplied by the rS scale. 1001 = probe fail low, 1002 = probe fail high (compared on the raw value).
40007, 40009, 40013, 40015	not used (NA)	--	Read inside the block, never interpreted.
40008	Actual temperature	not specified	Decoded like 40006 but only stored separately; never used as the primary temperature.
40010	System status / delay status	bit field	Defrost (bit 5), keypad lock (bit 1), resolution (bit 2), delays (bits 8-10).
40011	Relay status	bit field	Bit 0 = compressor relay, bit 1 = alarm relay (1 = ON).
40012	Fault status	bit field	Bits 0-6 = faults (table below).
40014	Compressor run hours	uint16	Hours.
40016, 40017	Balanced defrost time (P8H hours, P8m minutes)	uint16	As is.
40018	**Set point (read only)**	int16	Set point, multiplied by the rS scale.
40019, 40020	Temperature log high / low	int16	Multiplied by the rS scale; 1007 = log reset (no value).

40012 bit	Fault code in the software	Meaning
0	`probe_fail_low`	Control probe fail low (PFL)
1	`probe_fail_high`	Control probe fail high (PFH)
2	`high_temp`	High temperature (Ht)
3	`low_temp`	Low temperature (Lt)
4	`door_open`	Door open (do)
5	`compressor_fault`	Compressor fault (CF)
6	`manual_compressor_fault`	Compressor fault awaiting manual reset
7-15	`undocumented_fault_bits` (flag)	Not documented by the manufacturer; the raw value is kept.

The measuring range from the manual is -45.0 °C to 100.0 °C; a control temperature outside it is withheld (quality *invalid*, flag `out_of_range`). The programmable parameters in 40031-40068 (set point SP, high/low limits P2/P3, defrost settings, `did`, `bd`, `PAr`, `rS`, ...) are listed in Appendix A. They are only **read** (hourly) so that changes made on the keypad appear in the audit trail. The keypad password `PA` (40065) is never transmitted, stored or logged.

7.5 The Modbus request on the wire

A poll of controller 1 is the following RTU frame (hexadecimal). The CRC bytes are sent low byte first.

Request for registers 40001-40020 (CRC computed with the standard Modbus CRC-16)

```
Request  01 03 00 00 00 14 45 C5
         |  |  |---|  |---|  |---|
         |  |    |      |      +-- CRC-16 (low, high)
         |  |    |      +--------- quantity = 0x0014 = 20 registers (40001..40020)
         |  |    +---------------- start address = 0x0000 (= register 40001)
         |  +--------------------- function 03 = read holding registers
         +------------------------ slave ID 1
Controller 2:  02 03 00 00 00 14 45 F6
Response       01 03 28 <40 data bytes: 20 registers, high byte first> CRC
```

Registers are 16-bit big-endian values. The reply contains 40 data bytes (`0x28`). The parameter block is requested the same way with start address `0x001E` (= 40031) and quantity `0x0026` (= 38): `01 03 00 1E 00 26 A4 16`.

Addressing check

The register map does not say whether the controller counts addresses from 0 or 1. The node therefore **verifies the convention on every poll**: if register 40001 does not read 7507 or 40003 does not read 115, the poll is reported as `identity_mismatch` and no temperature is published. A per-controller `address_offset` exists (-1, 0, +1) for the case where commissioning proves an off-by-one; change it only on that evidence.

7.6 Temperature data format and worked examples

40005 (rS)	40006 raw	Calculation	Result
0 (0.1 °C)	180	180 × 0.1	18.0 °C
0 (0.1 °C)	65526 (0xFFF6 = -10)	-10 × 0.1	-1.0 °C
1 (1 °C)	18	18 × 1	18 °C
1 (1 °C)	65526 (0xFFF6 = -10)	-10 × 1	-10 °C
any	1001	probe fail low	no temperature; quality *probe_fault*
any	1002	probe fail high	no temperature; quality *probe_fault*

Illustrative register values for a healthy controller that agree with the dashboard in Figure 12-1 (18.0 °C, set point 15.0 °C, compressor OFF, alarm OFF, no faults), assuming rS = 0:

Register	Raw value	Meaning
40001	7507	PVR key (identity OK)
40003	115	Controller code (identity OK)
40004	1	Device ID = slave 1
40005	0	Resolution 0.1 °C
40006	180	18.0 °C
40018	150	Set point 15.0 °C
40011	0	Compressor OFF, alarm OFF
40012	0	No faults

7.7 Data quality labels

Situation	comm_status	quality	Temperature
Normal	ok	good	value
rS and 40010 bit 2 disagree; device ID differs from slave ID; or the Pi clock is not synchronised	ok	suspect	value (scaled per rS)
40006 = 1001/1002 or fault bit 0/1 set	ok	probe_fault	empty
Out of measuring range, unknown rS, or identity check failed	ok / identity_mismatch	invalid	empty
No or invalid reply, Modbus exception, port error	timeout / exception / invalid_response / port_error / error	comm_error	empty

IMPORTANT

Values are **never** estimated or carried forward. A failed poll is stored as a record with an empty temperature.

7.8 RS-485 wiring (Technician)

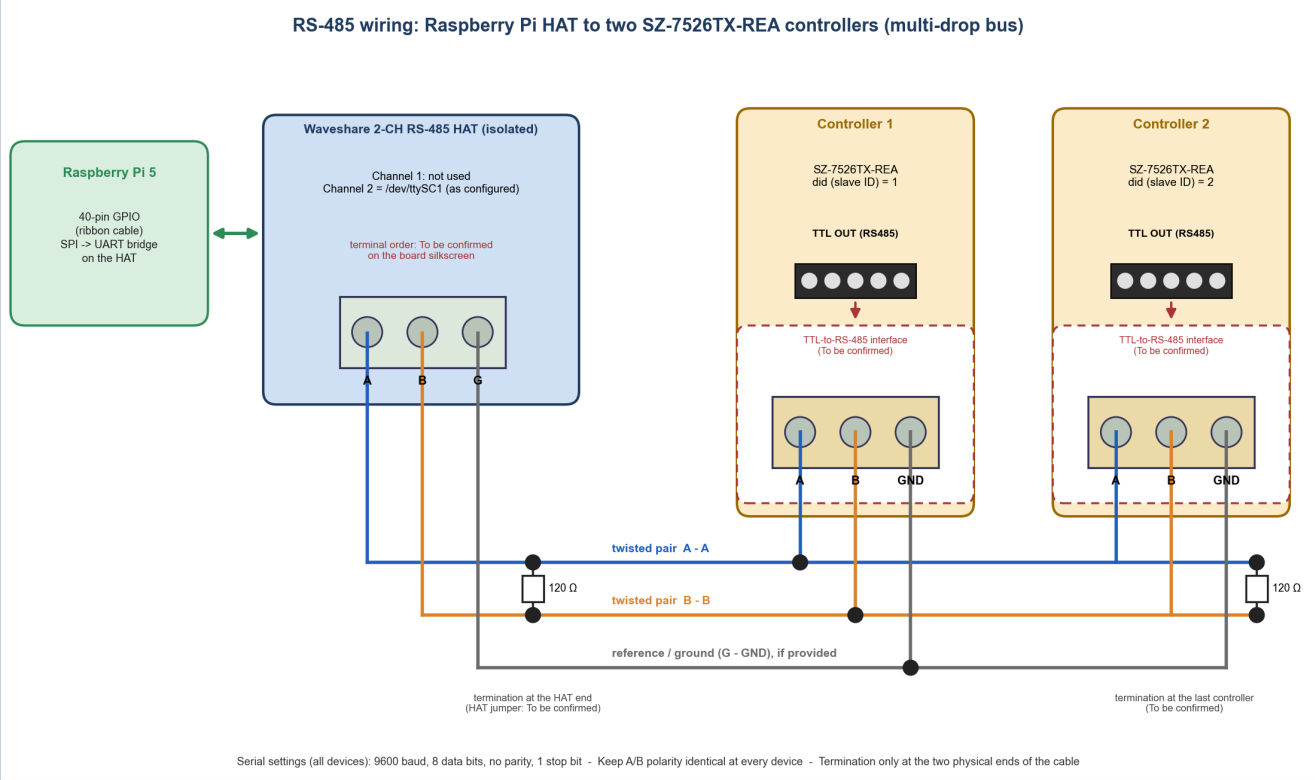


***Figure 7-1** - RS-485 wiring between the HAT's second channel and two controllers on one multi-drop bus. Each controller's 5-pin TTL OUT (RS485) header is connected to the bus through a TTL-to-RS-485 interface. Items marked "**To be confirmed**" must be taken from the HAT silkscreen and the interface documentation.*

Controller terminals (instruction manual, Suggested Wiring Diagram)

Terminal	Function
1, 2, 3	Alarm relay: NO (1), C (2), NC (3); 5 A 250 VAC maximum.
4, 5, 6	Compressor relay: NC (4), NO (5), C (6); 8(3) A 250 VAC maximum.
7, 8	Supply 230 VAC 50-60 Hz: 7 = phase, 8 = neutral (as drawn in the diagram).
10	AUX digital input (potential free): door-open or compressor-fault contact.
11 (marked *)	Common (ground) for the AUX input and the probe.
12	NTC control probe (connects to the probe assembly, drawn as *TO SZ-T75*).
TTL OUT (RS485)	5-pin header: the serial port read by the Raspberry Pi. The pins are not named in the manuals.

Mains voltage - qualified personnel only
Terminals 1 to 8 carry **mains voltage** (230 VAC). The manual states that improper wiring can cause irreparable damage and personal injury and that wiring must be done by qualified personnel, with the supply circuit connected through a protection switch. Terminals take wires up to 2.5 mm². Keep the probe wires out of any conduit that carries control or power lines. Do not touch the TTL header or its interface while the controller is powered.

Connections

- **A and B.** Connect HAT terminal A to terminal A and HAT terminal B to terminal B of the TTL-to-RS-485 interface of the first controller, using one **twisted pair**. Continue A-A, B-B to the interface of the second controller (daisy chain from device to device; avoid star wiring). The naming of A and B is not standardised between manufacturers, so labels may be inverted: **if nothing answers, swap A and B first**. This is the most common fault (`docs/hardware-validation.md`).
- **TTL-to-RS-485 interface.** The controller's own serial port is a TTL-level 5-pin header, not an RS-485 connector. Fit the interface that is intended for it, connect it to the header exactly as its documentation shows, and power it as it

requires. Which interface is used here and its pin-out: **To be confirmed**.

- **Ground / common.** The HAT shows a terminal marked **G**. The module `K-CUT B0505LS-1W` on the HAT indicates an isolated RS-485 side, so a common reference between the HAT and the interfaces is generally recommended. The project says: *common/GND connected if the adapter provides it*. Whether your interface provides one: **To be confirmed**.
- **Termination.** A 120 Ω terminator is used only at the two physical ends of the cable. Short single-drop benches usually work without it. Whether termination is a jumper on the HAT and on the interface is **To be confirmed**.
- **Power.** The Raspberry Pi is powered through its USB-C input. A HAT normally receives its supply through the 40-way cable from the Pi's GPIO header (**To be confirmed** from the HAT documentation). Each controller is supplied with **230 VAC** on terminals 7 and 8 (see the table above); the interface's own supply is **To be confirmed**. Power everything down before changing wiring.
- **Cable routing.** Keep the cable away from mains and contactor wiring (manual: *Cautions - wiring*).

Wiring procedure

1. Switch off the Raspberry Pi and isolate the controllers from the mains supply.
2. Identify the HAT's Channel 2 terminals (Figure 2-4) and the A / B terminals of each controller's TTL-to-RS-485 interface.
3. Connect A-A, B-B and (if provided) the common/ground with a twisted pair, daisy-chaining to the second controller.
4. Set `did` on each controller (1 and 2), `bd` and `PAr` identically on both.
5. Power on, then follow the next section to verify communication.

7.9 Verify that the controller is responding (Technician)

Level	How	What "working" looks like
1 Website	Dashboard card of the controller	Status **Normal** (or *Check data*) with a temperature and "Reading: n s ago". *Controller not responding* means communication failure.
2 Website	Data Logs, column **Comm**	`ok` on every row; `timeout` (red) on failed polls.
3 Node log	`sudo journalctl -u temperature-node -f`	No repeated `poll failed` lines; one `communication restored` after a fault.
4 Quick test	`read_temp_from_controllert.py` (Chapter 5, Step 8)	Prints `Temperature: xx.x °C`.
5 Validation tool	`python -m src.validate_controller --port /dev/ttySC1 --slave 1`	`[PASS]` lines for addressing, device ID, baud, parity, `did`, resolution; the printed control temperature equals the controller display.

Communication failure types

comm_status	Meaning	Typical cause
`timeout`	No (valid) reply. A CRC error is also reported as timeout.	Wrong slave ID, baud rate or parity; A/B swapped; no power; cable fault; noise; `did` = 0.
`exception`	The controller replied with a Modbus exception code.	Illegal address (code 2) once is normal: the node then switches to reading only the documented registers.
`invalid_response`	Reply of the wrong shape.	Interference; wrong device.
`port_error`	The serial port cannot be opened or failed.	HAT driver not loaded, wrong port name, another program using the port, missing `dialout` membership.
`identity_mismatch`	A device answered but 40001/40003 are not 7507/115.	Another device at that address, wrong model, or an address offset.
`error`	Anything else.	See the node log.

What the supplied screenshots show

TO BE CONFIRMED - observations from the supplied screenshots

In the supplied Dashboard and Data Logs screenshots (Figures 12-1 and 12-3), **Temp Control 2** shows *Controller not responding*: every poll is a `timeout` with an empty temperature, while **Temp Control 1** reads 18.0 °C with set point 15.0 °C. Because both are configured on one bus, this means that slave ID 2 is not answering (not connected or not powered, `did` not set to 2, or a wiring/parity problem). Confirm on site. The failed polls are recorded exactly as designed and do not affect Temp Control 1.

Temp Control 1 is labelled *Check data* (quality *suspect*) with the flag `resolution_mismatch`: the resolution register rS (40005) and bit 2 of status register 40010 disagree, a known ambiguity of the register map (A5 in `docs/modbus-register-map.md`). The software scales by rS, as the register map specifies. Compare the website value with the controller's display; if they differ by a factor of 10, report it to the controller vendor. All temperatures in the supplied screenshots are whole degrees.

CHAPTER 8

Data Acquisition, Transmission and Fail-Safe Operation

This chapter explains, from the code, how one measurement travels from the controller to the database, what the Raspberry Pi does with it while the server is unreachable, and what happens when the **server is shut down for an hour**, when the **Raspberry Pi is shut down for an hour**, and when **both** are. It also explains the **pending** status and the **7-day** retention that the node maintains.

8.1 Short answers

Question	Answer
Server (VM) down for 1 hour, Raspberry Pi running	**No data is lost.** The Pi keeps polling the controllers every 10 s and stores every reading as *pending* on its own disk (about 360 readings per controller per hour). When the server is back, the Pi uploads the backlog automatically, normally within about 6 minutes. The website is unreachable while the server is down.
Raspberry Pi down for 1 hour, server running	**The hour is not recorded.** Only the Pi reads the controllers, so no readings exist for that time. The server marks the node *offline* after 120 s and, when the Pi restarts, the node reports a data gap. The gap is visible in Monitoring and in the reports. Readings the Pi had stored before the power loss are still on its disk and are uploaded after it restarts.
Both down for the same hour	Same data result as the Pi alone: a one-hour gap. The order in which they start again does not matter; the Pi uploads when the server becomes reachable.
"Pending" status	A reading in the Pi's buffer `queue.db` that the server has **not yet confirmed**. It is kept on disk, survives restarts and power cuts, and is re-sent until the server confirms it. Pending readings are **never deleted by age**.
"7-day backup"	Readings the server **has confirmed** (*delivered*) are kept in the Pi's `queue.db` for **7 days** (`buffer.retention_days_delivered: 7`) and then removed. It is a local safety copy, not a backup of the server and not an archive. The permanent record is the server database, which must be backed up (Chapter 16).

8.2 The journey of one measurement

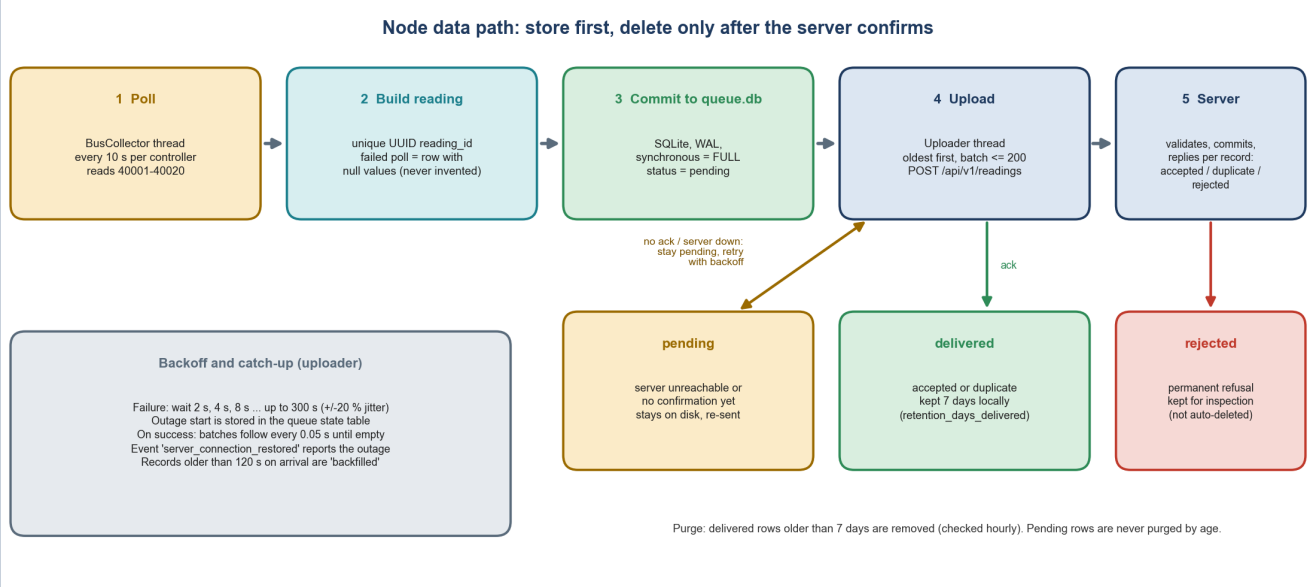


Figure 8-1 *- Data path on the Raspberry Pi: poll, build, commit to the local buffer, upload, server confirmation, and the resulting record states.*

1. **Poll.** Every 10 s the bus thread sends the read request for registers 40001-40020 to the controller over RS-485 (Chapter 7) and waits up to 1 s (up to 3 attempts).
2. **Decode and build.** The reply is checked (identity registers) and decoded. A *reading record* is created with a new UUID (`reading_id`), the acquisition time in UTC, all decoded values, the raw register values and a quality label. A failed poll also produces a record (empty temperature, `comm_status` other than `ok`).
3. **Commit.** The record is inserted into the local SQLite buffer with `journal_mode=WAL` and `synchronous=FULL` and gets the status **pending**. This is the first thing that happens; nothing depends on the network.
4. **Upload.** The uploader thread selects the oldest pending records (batch of up to 200) and sends them with HTTP POST to `http://10.145.15.245:8000/api/v1/readings` with the headers `X-Node-ID` and `Authorization: Bearer <token>`.
5. **Server commit.** The server validates each record, stores new ones, answers per record in three lists: `accepted`, `duplicates`, `rejected`. The answer is sent only after the database transaction is committed.
6. **Mark delivered.** The node changes `accepted` and `duplicates` records to **delivered**. `rejected` records (permanently refused) become **rejected** and are kept for inspection. Anything not mentioned in the answer stays pending and is retried.
7. **Purge.** Delivered records older than 7 days are deleted (checked at most once an hour, after a successful upload).

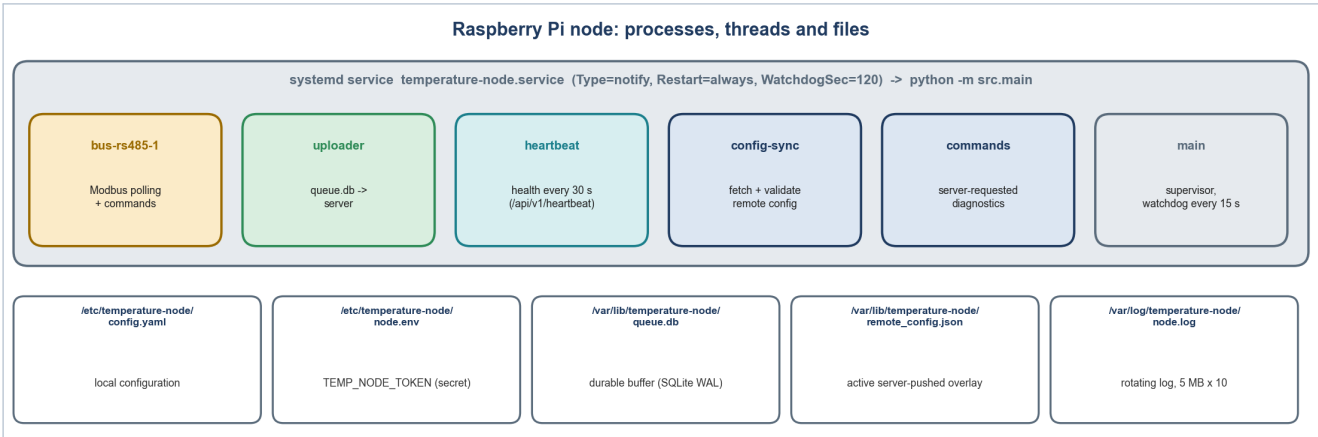


Figure 8-2 *- The node's service, its threads and the files it uses.*

8.3 What is sent to the server

Request	Purpose	When
`POST /api/v1/readings`	Batch of reading records; reply lists accepted / duplicates / rejected.	Every 5 s when idle; back-to-back (0.05 s apart) while a backlog exists. At most 200 per request by default (limit 1000).
`POST /api/v1/events`	Node events: `comm_lost`, `comm_restored`, `modbus_error`, `data_gap`, `node_started`, `server_connection_restored`, `parameter_snapshot`, `controller_command`.	Sent before readings in each upload cycle.
`POST /api/v1/heartbeat`	Health: software version, uptime, queue size, upload status, controller states, CPU temperature, free disk, Wi-Fi signal, clock status, recent errors.	Every 30 s.
`GET /api/v1/config?current_version=N`	Fetch a newer node configuration from the server.	Every 60 s, and immediately when a heartbeat reply announces a new version.
`POST /api/v1/config/ack`	Report that a configuration version was *applied* or *rejected*.	After each configuration.
`GET /api/v1/commands`, `POST /api/v1/commands/<id>/result`	Controller commands (disabled by default).	Checked every 60 s, and immediately when a heartbeat reply reports pending commands.
`GET /api/v1/ping`	Health check without authentication.	Manual test (Chapter 3).

Every request carries `X-Node-ID: temp_control_unit_1` and `Authorization: Bearer <token>`. The server stores only a SHA-256 hash of the token. An unknown node ID is recorded as *discovered* (Nodes page, *Unregistered devices*) and refused with HTTP 401; a wrong token gives 401; a disabled node gets 403. Time stamps are ISO-8601 in UTC.

One reading record (abbreviated; illustrative values; field names from build_reading in collector.py)


```
{
  "reading_id": "9b1c0f3e-2f6a-4a58-8d0e-5a0c1b6d7e11",
  "node_id": "temp_control_unit_1", "controller_id": "Temp_Control_1", "slave_id": 1,
  "model": "SZ-7526TX-REA", "source": "controller",
  "acquired_at": "2026-11-07T07:44:24.512Z",
  "comm_status": "ok", "comm_error": null, "quality": "suspect",
  "quality_flags": ["resolution_mismatch"],
  "temperature_c": 18.0, "setpoint_c": 15.0, "resolution_rs": 0,
  "compressor_on": false, "alarm_on": false, "defrost_active": false,
  "fault_flags": 0, "faults": [], "compressor_run_hours": 1234,
  "raw_registers": {"40001": 7507, "40003": 115, "40004": 1, "40005": 0, "40006": 180},
  "config_version": 5, "poll_duration_ms": 62
}
```


The server's reply to a batch


```
{"accepted": ["9b1c0f3e-2f6a-4a58-8d0e-5a0c1b6d7e11"], "duplicates": [], "rejected": []}
```


A failed poll is also a record (abbreviated)


```
{"reading_id": "...", "controller_id": "Temp_Control_2", "comm_status": "timeout",
 "comm_error": "no valid response: Modbus Error: [Input/Output] ERROR: No response received",
 "temperature_c": null, "quality": "comm_error", "raw_registers": null, ...}
```


8.4 The local buffer: queue.db and the pending status

Item	Detail
File	`/var/lib/temperature-node/queue.db` (plus `queue.db-wal` and `queue.db-shm` while running)
Tables	`readings` and `events` (identical layout) and `state` (key/value).
Columns of `readings` / `events`	`seq` (auto number), `item_id` (the UUID, unique), `controller_id`, `occurred_at`, `payload` (JSON), **`status`**, `attempts`, `last_error`, `created_at`, `delivered_at`
Status values	**`pending`** (not yet confirmed), **`delivered`** (confirmed; kept 7 days), **`rejected`** (permanently refused; kept)

Item	Detail
Durability	`PRAGMA journal_mode=WAL`, `PRAGMA synchronous=FULL`: a record that was acknowledged to the poller survives a sudden power cut. Raspberry Pis often lose power without a clean shutdown.
Upload order	Oldest first (`ORDER BY seq`), so the server sees each controller's history in order.
Persistent state	`outage_started_at`, `outage_failures`, `outage_last_error`, `last_alive_at`, `software_version`, `dropped_total`, `refused_since`, `refused_count`

What is kept, and for how long

Data	Where	Retention	Notes
Pending readings and events	Pi: `queue.db`	Until the server confirms them. **No age limit.**	Capped at `max_pending_records` = 2 000 000 readings or until free disk falls below `min_free_disk_mb` = 100 MB (see below).
Delivered readings and events	Pi: `queue.db`	**7 days** (`retention_days_delivered`), then deleted	Purged at most hourly, and only after a successful upload cycle. Local safety copy.
Rejected records	Pi: `queue.db`	Not deleted automatically	For inspection (status `rejected`, reason in `last_error`).
Node log file	Pi: `/var/log/temperature-node/node.log`	Rotating: 5 MB per file, 10 backups	At most about 55 MB. The same messages go to journald.
Readings, events, audit trail	Server: `temperature.db`	**Never purged**	The permanent record. Back it up.
Heartbeat history	Server: `temperature.db`	90 days (setting *Heartbeat history retention*)	Not measurement data.

Capacity. With two controllers polled every 10 s the node creates 12 readings a minute, 17 280 a day. The default capacity of 2 000 000 pending readings is therefore about **115 days** of backlog for two controllers (about 231 days for one). Seven days of delivered records are about 121 000 rows; at roughly 1 to 2 KB per row (an estimate, not measured) that is some 120-240 MB. Measure the real size with `ls -lh /var/lib/temperature-node/queue.db*`.

When the buffer is full

- `full_behavior: drop_oldest` (default): the oldest 1 % of pending readings are deleted to make room and a `data_gap` event states exactly how many were lost and the period they covered.
- `full_behavior: stop_collecting`: new readings are refused (and counted) until space is available; a `data_gap` event records the loss when collection resumes.
- Either way the loss is reported. With a healthy network this limit is never reached.

Inspect the buffer (Administrator)

On the Raspberry Pi

```
sudo sqlite3 /var/lib/temperature-node/queue.db \
  "select status, count(*), min(occurred_at), max(occurred_at) from readings group by status;"
sudo sqlite3 /var/lib/temperature-node/queue.db "select key, value from state;"
ls -lh /var/lib/temperature-node/
```

Example output (healthy node)

```
pending|1|2026-11-07T08:41:14.103Z|2026-11-07T08:41:14.103Z
delivered|120531|2026-10-31T08:44:21.220Z|2026-11-07T08:41:04.101Z
```

The website shows the same number in the **Buffered** column of the Dashboard's Nodes table (the count of pending readings reported in the node's last heartbeat). It is normally 0 to a few; a growing number means the server is not accepting data.

8.5 What the server does with the data

- **Validates** every record (UUID, time after year 2000, allowed `comm_status`/`quality`/`source` values, temperatures between -100 and 200 °C or empty, a temperature must be empty when `comm_status` is not `ok`). One invalid record is rejected without affecting the others in the batch.
- **De-duplicates** on the node-generated UUID: a record that was stored earlier is answered `duplicate`. This is what makes every retry safe.
- **Flags late arrivals**: a record received more than `BACKFILL_THRESHOLD_S` = 120 s after it was acquired is marked *backfilled*. Its stored time is still the original acquisition time.
- **Updates the latest state** of the controller (dashboard) only if the record is newer than what is already shown, so a late backlog never overwrites live data.
- **Tracks faults and alarms** as periods (started, cleared) from the fault register and the alarm relay.
- **Commits before answering**: the node is told a record is stored only after the database transaction has been committed (`synchronous=FULL`, WAL).
- **Watches the node**: a background thread checks every 30 s and marks a node *offline* when it has had no contact for 120 s (setting *Node offline after*), writing a `node_offline` event and an Audit Trail entry; the next contact marks it *online* again.

8.6 Outage scenarios

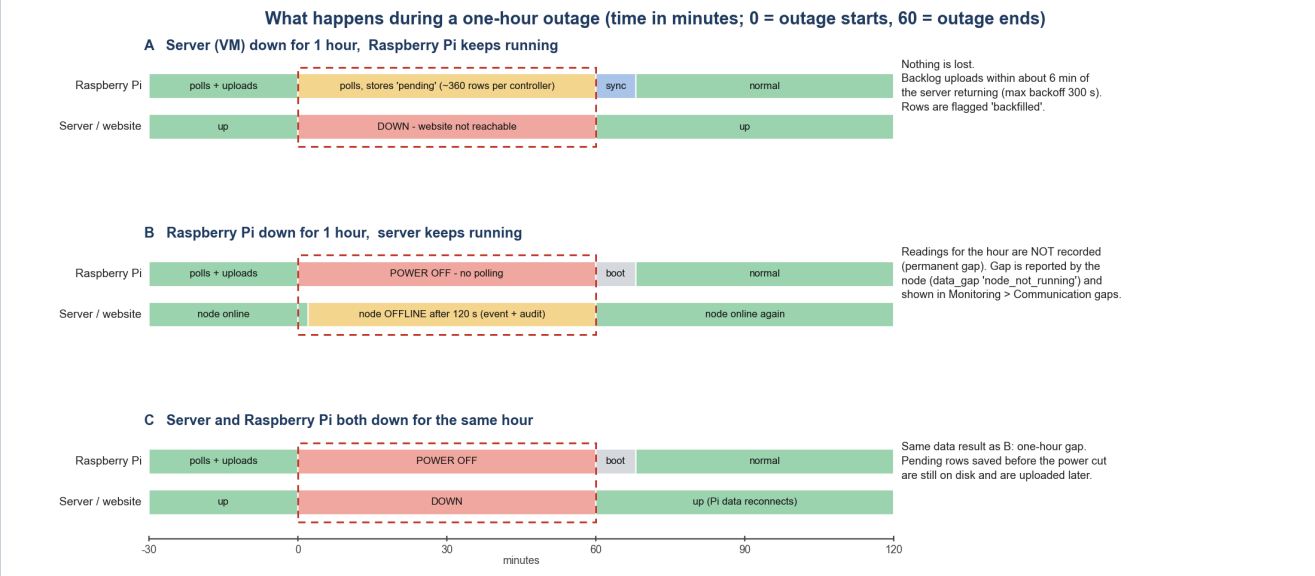


Figure 8-3 *- Behaviour during a one-hour outage of the server (A), of the Raspberry Pi (B) and of both (C).*

Scenario A - the server (Ubuntu VM) is shut down for one hour; the Raspberry Pi keeps running

When	What happens	Where you can see it
Minute 0	The server stops. The node's next upload fails (connection refused or timeout after 10 s). The node stores `outage_started_at` and logs `server unreachable, buffering locally`.	Pi log
Minutes 0 to 60	The node **keeps polling** every 10 s per controller and commits each reading as *pending*: about **360 readings per controller** (about 720 for two controllers). The uploader retries after 2 s, 4 s, 8 s ... 256 s and then every 300 s (each delay varies by ±20 %), about 19 attempts in the hour. Heartbeats fail every 30 s and are logged as warnings.	`sqlite3` count of `pending`; Pi log
Minute 60	The server starts again. Its start-up applies database migrations (`AUTO_MIGRATE`) and writes `server_started` to the audit trail. The first monitor check runs 30 s later.	Audit Trail, server log
Minute 60 to about 66	The node's next retry succeeds (at the latest about 300 s plus 20 % after the server is reachable). It sends events, then readings in batches of 200 with only 0.05 s between them: 720 readings take about four batches, a few seconds. It then records `server_connection_restored` with the outage start, end, failed attempts and backlog size.	Audit Trail: `node_reconnected`
Afterwards	Normal operation. The backlog rows are stored with their **original timestamps** and marked *backfilled*; the Pi's copies become *delivered* and are purged after 7 days.	Data Logs

What you see on the website after recovery: the Data Logs and Monitoring pages show a continuous series with **no gap** for the outage period. Filter *Ingestion* = *Backfilled* in Data Logs to list the rows that arrived late; the Excel export has a *Backfilled* column. Because the Data Logs view is frozen at the time it was opened, press **Refresh data** to see rows that arrived afterwards. During the outage the website itself cannot be opened.

> **Brief offline/online flicker after a server restart**
> Because the server's monitor thread only runs while the server is running, and the node's last-contact time is one hour old when the server restarts, the node may briefly show **Offline** and then **Online** again within the first minute after a server restart, before its first heartbeat arrives. This follows from the code (`monitor.py`, `ingestion.py`) and has not been tested on your system.

Scenario B - the Raspberry Pi is shut down for one hour; the server keeps running

When	What happens	Where you can see it
Minute 0	Power is lost (or the Pi is shut down). Nothing is polled. Records that were already in `queue.db` stay on disk. The node's `last_alive_at` was refreshed at most 15 s earlier.	--
About minute 2 to 2.5	The server sees no contact for more than 120 s and, at its next 30-second check, marks the node **offline**: event `node_offline` (error) and an Audit Trail entry with outcome *failure*. The Dashboard shows the node as **Offline** (Nodes online = 0) and the controller cards as *Node offline* (the values shown are the last ones received, not live).	Dashboard, Nodes, Audit Trail
Minutes 2 to 60	No new readings arrive. The controllers keep running by themselves; their own control is unaffected.	Monitoring: line break and gap
Minute 60	Power returns and the Pi boots. systemd starts `temperature-node` after the network is up. At start the node compares `last_alive_at` with the clock: if the gap is longer than max(60 s, 3 × the shortest poll interval) it queues a `data_gap` event "Node was not running from ... to ..." (reason `node_not_running`, with the duration), then a `node_started` event.	Pi log
Minute 60 onward	Polling resumes immediately. The first heartbeat marks the node **online** (`node_online`). The queued events reach the server and appear as `node_started` and `data_gap_reported` in the Audit Trail. Any pending readings saved before the power loss are uploaded and flagged *backfilled*.	Audit Trail, Monitoring

Data impact: a permanent one-hour gap. The controller does not keep a history that the system could read afterwards (registers 40019/40020 hold only the most recent high and low values), so the missing hour cannot be reconstructed. The system does not invent values: Monitoring lists the period under *Communication gaps (calculated)* and breaks the chart line, and the reports *Missing data periods* and *Node downtime* list it (downtime is *inferred* from missing heartbeats, and labelled as such).

Scenario C - the server and the Raspberry Pi are both shut down for one hour

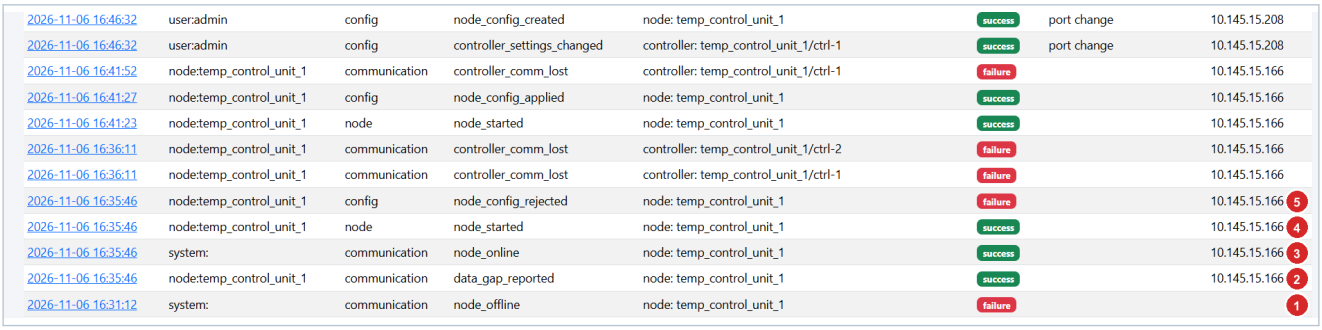
- The data result is the same as in Scenario B: **no readings exist for the hour** (the Pi was off). Records the Pi had stored before the power loss are still on its disk.
- The start order does not matter. If the Pi boots first it buffers (events `node_started` and `data_gap` and new readings are *pending*) and uploads as soon as the server is reachable, within the 300 s retry limit. If the server starts first, the node is marked offline until the Pi sends its first heartbeat.
- A server that is switched off normally writes `server_stopped` to the audit trail when its process exits; after a sudden power loss that entry is **not** written (the audit trail then shows `server_started` without a preceding `server_stopped`). The database itself is protected by WAL and `synchronous=FULL`.

Other situations

Situation	Behaviour
Wi-Fi or network fails, Pi and server both running	Same as Scenario A from the Pi's point of view (buffer and retry); the server also marks the node offline after 120 s because heartbeats stop. After the network returns: `node_online`, backlog uploaded, rows flagged *backfilled*.
RS-485 or a controller fails, Pi and server running	Failed polls are stored (empty temperature, `comm_status timeout`, quality `comm_error`). After 3 consecutive failures (about 30 s) a `comm_lost` event is sent; the Dashboard shows *Controller not responding*. When the controller answers again a `comm_restored` event is sent. Nothing is estimated.
Raspberry Pi's clock wrong after boot (no network time yet)	Readings are flagged `clock_unsynchronized` and quality *suspect* until the clock is synchronised. The server also calculates the node's clock skew from each heartbeat.
Raspberry Pi disk full or buffer full	Policy `drop_oldest` or `stop_collecting` with a `data_gap` event that gives the number of lost readings and the period.
The service crashes or hangs	systemd restarts it after 5 s (`Restart=always`); a hung process that stops signalling is restarted after 120 s (`WatchdogSec`). A dead worker thread is restarted within 15 s by the supervisor.

8.7 A real example from your audit trail

Figure 8-4 is a section of the supplied Audit Trail (2026-11-06, displayed in Asia/Kolkata). It is a real record of the behaviour described in Scenario B.



2026-11-06 16:46:32	user:admin	config	node_config_created	node: temp_control_unit_1	success	port change	10.145.15.208
2026-11-06 16:46:32	user:admin	config	controller_settings_changed	controller: temp_control_unit_1/ctrl-1	success	port change	10.145.15.208
2026-11-06 16:41:52	node:temp_control_unit_1	communication	controller_comm_lost	controller: temp_control_unit_1/ctrl-1	failure		10.145.15.166
2026-11-06 16:41:27	node:temp_control_unit_1	config	node_config_applied	node: temp_control_unit_1	success		10.145.15.166
2026-11-06 16:41:23	node:temp_control_unit_1	node	node_started	node: temp_control_unit_1	success		10.145.15.166
2026-11-06 16:36:11	node:temp_control_unit_1	communication	controller_comm_lost	controller: temp_control_unit_1/ctrl-2	failure		10.145.15.166
2026-11-06 16:36:11	node:temp_control_unit_1	communication	controller_comm_lost	controller: temp_control_unit_1/ctrl-1	failure		10.145.15.166
2026-11-06 16:35:46	node:temp_control_unit_1	config	node_config_rejected	node: temp_control_unit_1	failure		10.145.15.166 5
2026-11-06 16:35:46	node:temp_control_unit_1	node	node_started	node: temp_control_unit_1	success		10.145.15.166 4
2026-11-06 16:35:46	system:	communication	node_online	node: temp_control_unit_1	success		10.145.15.166 3
2026-11-06 16:35:46	node:temp_control_unit_1	communication	data_gap_reported	node: temp_control_unit_1	success		10.145.15.166 2
2026-11-06 16:31:12	system:	communication	node_offline	node: temp_control_unit_1	failure		1

***Figure 8-4** - Audit Trail excerpt showing a node outage and recovery (call-outs added).*

No.	Time	Entry	Explanation
1	16:31:12	`node_offline` (actor *system*, outcome failure)	The server's monitor marked `temp_control_unit_1` offline after 120 s without contact. The server wrote it, so there is no source IP.
2	16:35:46	`data_gap_reported` (actor node)	After restarting, the node reported a data gap. The details of this entry (click the time) give the period, duration and reason.
3	16:35:46	`node_online` (actor *system*)	The first contact from the node: it is online again.
4	16:35:46	`node_started`	The node software started (a power cycle or a service restart).
5	16:35:46	`node_config_rejected` (failure)	The node rejected a configuration version pushed from the server. Open the entry to read the reason; the project's troubleshooting guide names a serial port that does not exist on the Pi as the typical cause. The previous configuration stayed in force. A later entry at 16:41:27 shows `node_config_applied`.

From 16:31:12 to 16:35:46 is **4 minutes 34 seconds** between the offline mark and the node's return. The Audit Trail alone cannot tell whether the Pi lost power or only its service restarted; the details of the `data_gap_reported` entry state the period the node was not running. Further examples are in Figure 12-2: the *Communication gaps (calculated)* table lists four gaps for Temp Control 1 (10 min and 1 min on 2026-11-06, 5 min and 8 min on 2026-11-07), 24 minutes in total.

8.8 Fail-safe mechanisms at a glance

Mechanism	Where	Protects against
Store first: durable local queue (WAL, `synchronous=FULL`)	Node `local_queue.py`	Power cuts, crashes: acknowledged readings survive.
Delete only after server confirmation; UUID per reading; server de-duplication	Node `uploader.py`, server `ingestion.py`	Lost or repeated batches; every retry is safe.
Exponential backoff 2-300 s with jitter; outage start persisted	Node `uploader.py`	Server or network outage; hammering a recovering server; outage reported after a node restart.
Batch halving on HTTP 400/413; per-record rejection and quarantine	Node `uploader.py`, server	A single bad record blocking the queue.
Failed polls stored as records with empty values	Node `collector.py`	Silent gaps; invented numbers.
Identity check on every poll (7507 / 115)	Node `modbus_client.py`, `decoder.py`	Wrong device, wrong address convention.
`comm_lost` / `comm_restored` events after 3 failures	Node `collector.py`	Unnoticed controller failure.
Supervisor restarts dead threads; systemd `Restart=always`, `WatchdogSec=120`	Node `main.py`, service file	Thread crash, process crash or hang.
Start-up `data_gap` (`node_not_running`) and `node_started` events	Node `main.py`	Unexplained holes after a power loss.

Mechanism	Where	Protects against
`clock_unsynchronized` flag	Node `health_monitor.py`	Wrong time stamps after boot without network time.
Buffer capacity and free-disk guard with `data_gap` event	Node `local_queue.py`	Full disk, runaway backlog.
Remote configuration validated before use; last good version kept; new server URL tested first	Node `config_manager.py`, `main.py`	A bad configuration disabling the node.
Server answers only after commit; duplicate handling; older batches never overwrite newer state	Server `ingestion.py`	Acknowledging unsaved data; concurrent retries; backfill corrupting live values.
Offline detection after 120 s; online/offline events	Server `monitor.py`	A silent node failure.
WAL, `synchronous=FULL`, `busy_timeout` 10 s	Server `app/__init__.py`	Power loss on the server; short database locks.
Append-only audit trail (ORM guard and database triggers)	Server `models/audit.py`	Tampering with history.
No writes to controllers during polling; remote commands off by default (four gates)	Node `modbus_client.py`	Accidental control actions.
Server restart on failure (`Restart=on-failure`, see Chapter 9)	Server systemd unit	Server process crash.

8.9 What the system does not protect against

- **Raspberry Pi powered off:** nothing reads the controllers, so no data exists for that time. There is no second reader and no redundancy.
- **Failure or corruption of the Pi's storage card:** pending (unsent) readings on it would be lost.
- **A very long outage** that exceeds the buffer capacity (default 2 000 000 readings, about 115 days for two controllers) or fills the disk: the oldest pending readings are dropped, with a `data_gap` event.
- **Loss of the server's database file** without a backup. The server has no replication; `scripts/backup_database.py` or the procedure in Chapter 16 is needed.
- **No notifications:** nobody is alerted by e-mail or SMS; alarms are visible on the website only.
- **Controller-internal logs:** the system does not read any event history from the controller.
- **Wrong time on the Pi** is flagged but not corrected by the application.

8.10 Test the fail-safe behaviour on site (Administrator)

The project's commissioning procedure (`docs/hardware-validation.md`) includes these tests. Run them in a maintenance window and keep the results.

Test	How	Expected result
Network or server outage, 5 minutes	Unplug the Pi's network (or `sudo systemctl stop temperature-server` on the server) for about 5 minutes, then restore it.	No gap in Data Logs; the outage rows are marked *buffered/backfilled*; the node page shows the reconnection event; *Buffered* returns to about 0.
Controller communication loss	Disconnect the RS-485 cable for about one minute and reconnect.	*Controller not responding*, `comm_lost`, failed polls with no temperature; then `comm_restored`.
Pi restart	`sudo reboot` on the Pi.	`node_offline`, then `node_online`, `node_started` and `data_gap_reported` in the Audit Trail; a gap in Monitoring.
Watch the buffer	On the Pi: `watch -n 5 'sudo sqlite3 /var/lib/temperature-node/queue.db "select status,count(*) from readings group by status"'`	`pending` rises during the outage and returns to about 0 afterwards.

CHAPTER 9

Ubuntu Remote Server Configuration

This chapter builds the server from scratch on the Ubuntu virtual machine (`10.145.15.245`). The steps follow the "Ubuntu server" section of the project's `README.md`. The server is a **Flask** web application served by **Waitress** (`server/run.py`) with an **SQLite** database; it listens on **TCP port 8000**.

> **IMPORTANT**
> The project's other documents (`docs/installation-windows.md`, parts of `docs/troubleshooting.md`) describe a Windows server. This installation uses Ubuntu, so follow this chapter and the Ubuntu section of `README.md`. The same code runs on both.

9.1 Layout on the server

The README and the helper script `dev_tools/delete_controler_in_server_history.py` use the following layout under the user's home folder. The project's `server/` folder is copied to the VM as `temperature-logging-system` (folder name **To be confirmed** on the VM):

Path on the VM	Content
`/home/temp/temperature-logging-system/`	The server code (contents of the project's `server/` folder): `run.py`, `app/`, `migrations/`, `requirements.txt`, `.env.example`
`.../.venv/`	Python virtual environment
`.../.env`	Server settings and the secret key (mode 600)
`.../instance/temperature.db`	The SQLite database (plus `temperature.db-wal` and `temperature.db-shm` while running)
`.../instance/secret_key`	Generated session key, only used if `SECRET_KEY` is empty in `.env`
`.../logs/server.log`	Rotating log (10 MB x 10 files by default)
`/etc/systemd/system/temperature-server.service`	The service that starts the server at boot (Section 9.12)

9.2 1-2 Access the virtual machine and connect with SSH (Administrator)

Use the hypervisor's console, or connect from a Windows computer on the network:

On the Windows computer

```
ssh temp@10.145.15.245
```

Type `yes` for the fingerprint question and enter the password from the credentials section. The prompt becomes `temp@temp-Virtual-Machine:~$`.

9.3 3 Update Ubuntu

On the server

```
sudo apt update
sudo apt upgrade -y
cat /etc/os-release | head -3
```

The last command shows the Ubuntu version (the README supports 22.04 and 24.04; the installed version is **To be confirmed**). Reboot with `sudo reboot` if the upgrade asks for it.

9.4 4-5 Install Python, pip, venv support and tools

On the server

```
sudo apt install -y python3 python3-venv python3-pip git sqlite3 curl
python3 --version
```

`python3-venv` provides virtual environments and `python3-pip` the package installer. `sqlite3` is the command-line tool used later to inspect the database; `curl` is for tests.

9.5 6-7 Create the application folder and upload the server files

On the Windows computer (PowerShell; the backtick continues the line)

```
cd "C:\Users\gokul\Documents\projects\Avo AC Unit Logging System"
cd avo_ac_loggin_system_with_rashberrypi
tar -czf server.tgz --exclude=.venv --exclude=instance --exclude=logs --exclude=.env `
    --exclude=__pycache__ -C server .
scp server.tgz temp@10.145.15.245:/tmp/
```

On the server

```
mkdir -p ~/temperature-logging-system
tar -xzf /tmp/server.tgz -C ~/temperature-logging-system
ls ~/temperature-logging-system
```

The exclusions keep your development PC's virtual environment, database, logs and secrets out of the transfer. `ls` must show `run.py`, `app`, `migrations`, `requirements.txt` and `.env.example` (use `ls -a` to see `.env.example`).

9.6 8 Create the virtual environment and install the dependencies

On the server

```
cd ~
python3 -m venv temperature-logging-system/.venv
temperature-logging-system/.venv/bin/pip install --upgrade pip
temperature-logging-system/.venv/bin/pip install -r temperature-logging-system/requirements.txt
```

The dependencies are pinned in `requirements.txt`: Flask 3.1.0, Werkzeug 3.1.3, Flask-SQLAlchemy 3.1.1, SQLAlchemy 2.0.36, Flask-Migrate 4.0.7, Alembic 1.14.0, Flask-Login 0.6.3, Flask-WTF 1.2.2, WTForms 3.2.1, Waitress 3.0.2, openpyxl 3.1.5, ReportLab 4.2.5, python-dotenv 1.0.1, tzdata 2024.2. The installation needs internet access for `pip` (or an internal package mirror).

9.7 9-12 Configure environment variables, database, application and port

On the server

```
cd ~/temperature-logging-system
cp .env.example .env
SECRET=$(.venv/bin/python -c "import secrets; print(secrets.token_urlsafe(48))")
sed -i "s|^SECRET_KEY=.*|SECRET_KEY=$SECRET|; s|^HOST=.*|HOST=0.0.0.0|; s|^PORT=.*|PORT=8000|" .env
chmod 600 .env
nano .env
```

The commands copy the template, generate a random secret key for session signing, make the server listen on all interfaces (`HOST=0.0.0.0`, needed so that the Raspberry Pi and browsers can reach it) on port 8000, and restrict the file to your user. The remaining entries of `.env` are shown below with the values for this installation.

Setting	Value	Meaning
`HOST`	`0.0.0.0`	Bind address. `127.0.0.1` would accept only local connections.
`PORT`	`8000`	TCP port of the website and node API.
`WAITRESS_THREADS`	`8`	Worker threads of the Waitress server.

Setting	Value	Meaning
SECRET_KEY	(random, generated above)	Signs session cookies. Keep it secret.
DATABASE_URL	(empty / commented out)	Default: SQLite file instance/temperature.db. PostgreSQL is possible (postgresql+psycopg://...) but untested by the project.
DISPLAY_TIMEZONE	Asia/Kolkata	Initial display time zone. Administrators can change it later in System settings. Data is always stored in UTC.
SESSION_TIMEOUT_MINUTES	30	Idle time after which a user must sign in again.
LOGIN_MAX_FAILURES / LOGIN_LOCKOUT_MINUTES	5 / 15	Account lock-out.
PASSWORD_MIN_LENGTH	8	Minimum password length (mixed with digits or symbols).
ENABLE_REMOTE_CONTROL	false	Initial value of the controller-command master switch.
MAX_REQUEST_BYTES, API_MAX_BATCH	4194304, 1000	Request and batch limits for the node API.
EXPORT_MAX_ROWS_XLSX / EXPORT_MAX_ROWS_PDF	500000 / 20000	Larger exports are refused with a message, never truncated.
BACKFILL_THRESHOLD_S	120	Readings arriving later than this are flagged backfilled.
LOG_LEVEL, LOG_FORMAT, LOG_DIR	INFO, text, (default logs/)	Logging.
AUTO_MIGRATE	true	Apply database upgrades automatically at start-up.

VERIFIED IN SOURCE
Real environment variables take precedence over values in .env (server/app/config.py). If SECRET_KEY is empty a random key is created in instance/secret_key, so sessions survive restarts either way.

9.8 Create the database, roles and the first administrator

On the server

```
cd ~/temperature-logging-system
.venv/bin/python -m flask --app run db upgrade
.venv/bin/python -m flask --app run seed
.venv/bin/python -m flask --app run create-admin --username admin
```

Command	What it does
flask --app run db upgrade	Creates instance/temperature.db and all tables (Alembic migration 0001_initial_schema), including the triggers that make the audit trail append-only.
flask --app run seed	Creates the permissions and the three default roles (Administrator, Operator, Auditor).
flask --app run create-admin --username admin	Creates the user admin with the role Administrator. It asks for the password twice (hidden). The password must satisfy the policy: at least 8 characters, letters mixed with digits or symbols, not containing the user name.

Use the password from the credentials section. The program prints user admin created with role Administrator.

9.9 Open the firewall port

On the server

```
sudo ufw status
sudo ufw allow OpenSSH
sudo ufw allow 8000/tcp
sudo ufw status
```

If ufw is *inactive*, nothing blocks port 8000 and you may skip the allow rules. If it is active, TCP 8000 must be allowed so that the Raspberry Pi and the browsers can connect. Do **not** enable ufw without first allowing SSH (second command), or you lock yourself out. The current state of the firewall is **To be confirmed**.

9.10 13 Run the server manually

On the server (stop with Ctrl+C)

```
cd ~/temperature-logging-system
.venv/bin/python run.py
```

Expected log line

```
... Temperature server 1.0.0 listening on http://0.0.0.0:8000
```

At start-up the program applies database migrations, seeds the roles, writes a `server_started` entry to the audit trail and starts the monitor thread that detects offline nodes. Run **only one** server process; the monitor runs inside it.

9.11 14 Test the server

On the server, with run.py running in another session

```
curl -s http://localhost:8000/api/v1/ping
curl -sI http://localhost:8000/login | head -1
```

Expected output

```
{"server_time":"2026-11-07T08:20:00.000000Z","status":"ok","version":"1.0.0"}
HTTP/1.1 200 OK
```

Then test from another machine: open `http://10.145.15.245:8000` in a browser on the plant network, or run `curl -s http://10.145.15.245:8000/api/v1/ping` on the Raspberry Pi. If it works locally but not remotely, check `HOST=0.0.0.0`, `ufw` and the network.

9.12 15 Start the server automatically after reboot

On the server (from README.md)

```
sudo tee /etc/systemd/system/temperature-server.service >/dev/null <<'EOF'
[Unit]
Description=Temperature Monitoring Server
After=network-online.target
Wants=network-online.target
[Service]
User=temp
WorkingDirectory=/home/temp/temperature-logging-system
ExecStart=/home/temp/temperature-logging-system/.venv/bin/python run.py
Restart=on-failure
RestartSec=5
[Install]
WantedBy=multi-user.target
EOF
sudo systemctl daemon-reload
sudo systemctl enable --now temperature-server
sudo systemctl status temperature-server
```

Line	Meaning
`After/Wants=network-online.target`	Start once the network is up.
`User=temp`	Run as the user `temp`, not as root.
`WorkingDirectory=...` and `ExecStart=...`	Run `run.py` with the virtual-environment Python from the application folder (so `.env` and `instance/` are found).
`Restart=on-failure`, `RestartSec=5`	Restart 5 s after a crash. A clean `systemctl stop` is not restarted.
`WantedBy=multi-user.target`	Started at boot when enabled.

Command	Use
`sudo systemctl start|stop|restart temperature-server`	Start, stop or restart the website and API. Restart after changing `.env`.
`systemctl status temperature-server`	State and last log lines.
`sudo systemctl disable --now temperature-server`	Stop and prevent start at boot.
`sudo journalctl -u temperature-server -f`	Follow the live log.

TO BE CONFIRMED
The unit name `temperature-server` and its content come from the README. Confirm the name used on your VM with `systemctl list-units --type=service | grep -i temperature`.

9.13 16 Check the server logs

On the server
```
sudo journalctl -u temperature-server --since "1 hour ago"
tail -f ~/temperature-logging-system/logs/server.log
```

Pages that fail show "Internal error (reference ...)" in the browser; search `server.log` for that reference to find the cause.

9.14 Upgrading the server software

1. Stop: `sudo systemctl stop temperature-server`.
2. Back up the database (Chapter 16).
3. Upload the new code as in step 6-7, which does not touch `.env`, `.venv` or `instance/`.
4. Update dependencies: `~/temperature-logging-system/.venv/bin/pip install -r ~/temperature-logging-system/requirements.txt`.
5. Start: `sudo systemctl start temperature-server`. Database upgrades are applied automatically (`AUTO_MIGRATE`).

9.15 17 Troubleshooting the server

Symptom	Likely cause	Fix
Browser cannot connect	Service stopped, `HOST=127.0.0.1`, firewall, wrong IP.	`systemctl status temperature-server`; check `.env`; `sudo ufw status`; `ss -ltnp | grep 8000`.
`Address already in use`	Another process (or a second copy of the server) uses port 8000.	`sudo ss -ltnp | grep 8000`; stop it or change `PORT`.
Service fails right after start	Syntax or configuration error, missing module.	`sudo journalctl -u temperature-server -n 50`; run `.venv/bin/python run.py` by hand to see the error.
`database is locked` warnings	Many simultaneous large exports on SQLite.	They retry for up to 10 s. For heavy use consider PostgreSQL.
Pages show "Internal error (reference ...)"	Application error.	Find the reference in `logs/server.log`.
Times look shifted	Display time zone setting.	Administration > System settings > Display time zone. Data is stored in UTC.
Node data refused (401/403)	Wrong token, unknown node ID, disabled node.	Nodes page; issue a new token; check the node's `node.env`.

CHAPTER 10

Database Configuration and Data Storage

10.1 Which database is used

The server uses **SQLite**, a single-file database, through SQLAlchemy and Alembic migrations (`server/migrations/`). SQLite needs no database server, no user and no password. The file is `instance/temperature.db` below the application folder (`/home/temp/temperature-logging-system/instance/temperature.db`). Connections use `PRAGMA journal_mode=WAL`, `PRAGMA synchronous=FULL`, `PRAGMA foreign_keys=ON` and `PRAGMA busy_timeout=10000` (`server/app/__init__.py`), so an acknowledged reading survives a power loss. PostgreSQL can be selected with `DATABASE_URL`; the project documents it but has not tested it.

10.2 How the database is created

`flask --app run db upgrade` (Chapter 9) applies the migration `0001_initial_schema` and creates every table and the audit-trail triggers. The server also runs the upgrade at every start-up when `AUTO_MIGRATE=true`. Nothing else has to be created by hand.

10.3 Tables and their purpose

Table	Purpose
`temperature_readings`	**One row per poll attempt of a controller**, exactly as reported by the node; never updated. Temperature is empty when the controller could not be read.
`nodes`	One row per Raspberry Pi node: identity, token hash, latest health values.
`controllers`	One row per controller per node, with its latest state (for the dashboard) and its latest parameter snapshot.
`communication_events`	Node-reported and server-generated events (comm lost/restored, node online/offline, data gaps, ...).
`node_heartbeats`	Heartbeat history (purged after the configured retention, default 90 days).
`fault_events`	Fault and alarm periods derived from the fault register (start, clear, code).
`node_config_versions`	History of node configurations pushed from the server, with delivery and acknowledgement status.
`controller_config_history`	Server-side controller settings and parameter changes detected on the device.
`controller_commands`	Requested controller commands and their results.
`discovered_nodes`	Unregistered devices that tried to connect (never trusted).
`audit_events`	The append-only audit trail (updates and deletes are blocked by database triggers).
`export_records`	Every export or report: who, when, filters, record count, file name, SHA-256.
`users, roles, permissions, role_permissions`	Accounts (password hashes only) and role-based access.
`system_settings`	Administrator-editable settings (time zone, offline limit, ...).

Main fields of temperature_readings

Field	Description
`reading_uid`	UUID generated on the node (unique; used for de-duplication).
`node_id`, `controller_id`	Links to `nodes` and `controllers`.
`acquired_at`	Time the node polled the controller (node clock, UTC).

Field	Description
received_at	Time the server stored the record (server clock, UTC).
backfilled	True when the record arrived more than 120 s after acquired_at.
source	controller or simulated.
comm_status, comm_error	ok, timeout, exception, invalid_response, port_error, identity_mismatch, error and the error text.
quality, quality_flags	good, suspect, probe_fault, invalid, comm_error and the reasons (for example resolution_mismatch).
temperature_c	Control (display) temperature from register 40006. NULL if there is no valid value.
actual_temperature_c, setpoint_c, temp_log_high_c, temp_log_low_c, resolution_rs	Registers 40008, 40018, 40019, 40020, 40005.
compressor_on, alarm_on, defrost_active, keypad_locked	Decoded status bits.
fault_flags, fault_codes	Raw fault register 40012 and the decoded codes.
system_status, relay_status, compressor_run_hours	Registers 40010, 40011, 40014.
raw_registers, extra_json	The registers as received (JSON) and further values (delays, defrost time, warnings).

10.4 How node and controller information is stored

nodes.uid holds the Node ID (temp_control_unit_1); controllers.uid holds the Controller ID (Temp_Control_1, Temp_Control_2) and is unique within a node. The tables keep the node's last contact, heartbeat, IP address (last_ip), software version, running configuration version, queue size and clock state, and for each controller the last reading, last good reading, last temperature, set point, quality and communication status.

10.5 How timestamps are recorded

All timestamps are stored as UTC. The node stamps each reading when it polls (ISO-8601 with Z), and the server adds received_at. The website converts to the display time zone (Asia/Kolkata, setting *Display time zone*) when it shows or exports data; date filters typed on the website are interpreted in that time zone. India Standard Time is UTC+05:30, so 13:14 IST is 07:44 UTC. The footer of every page repeats this: "all data stored in UTC, displayed in Asia/Kolkata".

10.6 How historical data is retrieved

Data Logs, exports and charts all use the same filter (server/app/services/queries.py): time range on acquired_at, node, controller, site, equipment, temperature limits, fault, communication state, compressor, alarm, quality, source and ingestion type. Indexes on (controller, time), (node, time) and the time column keep queries fast. The default window is the last 24 hours.

10.7 Check the database from the command line (Administrator)

On the server

```
cd ~/temperature-logging-system
sqlite3 instance/temperature.db ".tables"
sqlite3 instance/temperature.db "PRAGMA integrity_check;"
sqlite3 instance/temperature.db "select count(*), max(acquired_at) from temperature_readings;"
sqlite3 instance/temperature.db "select c.uid, count(*), max(r.acquired_at) from temperature_readings r \
  join controllers c on c.id = r.controller_id group by c.uid;"
.venv/bin/python -m flask --app run db current
```

.tables lists the tables; integrity_check must print ok; the counts prove that readings are being stored and when the latest one arrived (in UTC); the last command shows the migration revision the database is at. Use read-only queries on a running server; never edit rows by hand.

Destructive developer tool
`dev_tools/delete_controler_in_server_history.py` **deletes** the readings, events, fault events, configuration history and commands of the listed controllers (`ctrl-1`, `ctrl-2` of node `temp_control_unit_1`) and the controller rows themselves. It asks for the word `yes`. It exists to clear test controllers; it removes data from the permanent record and should never be run on real data. Stop the server and make a backup first.

10.8 Back up the database

The project's script `scripts/backup_database.py` uses SQLite's online backup API, verifies the copy with `PRAGMA integrity_check` and keeps the newest 30 copies. It expects the repository layout (a `server/` folder next to `scripts/`), which differs from the VM layout. The following stand-alone script does the same on the VM:

/home/temp/backup_db.py (create with nano)
```
#!/usr/bin/env python3
"""Consistent online backup of the temperature database (adapted from scripts/backup_database.py)."""
import datetime, pathlib, sqlite3, sys
SRC = pathlib.Path("/home/temp/temperature-logging-system/instance/temperature.db")
DEST = pathlib.Path("/home/temp/backups")
KEEP = 30
DEST.mkdir(parents=True, exist_ok=True)
target = DEST / f"temperature-{datetime.datetime.now():%Y%m%d-%H%M%S}.db"
with sqlite3.connect(f"file:{SRC}?mode=ro", uri=True) as src, sqlite3.connect(target) as dst:
    src.backup(dst)
    result = dst.execute("PRAGMA integrity_check").fetchone()[0]
if result != "ok":
    sys.exit(f"integrity check FAILED on {target}: {result}")
for old in sorted(DEST.glob("temperature-*.db"))[:-KEEP]:
    old.unlink()
print(f"backup written: {target}")
```

On the server
```
python3 /home/temp/backup_db.py
crontab -e
```

Line to add in crontab: backup every night at 02:30
```
30 2 * * * /usr/bin/python3 /home/temp/backup_db.py >> /home/temp/backups/backup.log 2>&1
```

Keep a copy elsewhere
A backup on the same machine does not protect against loss of the machine. Copy the files in `/home/temp/backups/` to another computer regularly (for example `scp temp@10.145.15.245:/home/temp/backups/*.db D:\TemperatureBackups\`).

10.9 Restore the database

On the server (replace the file name with the backup you want)
```
sudo systemctl stop temperature-server
cd ~/temperature-logging-system/instance
mv temperature.db temperature.db.broken
rm -f temperature.db-wal temperature.db-shm
cp /home/temp/backups/temperature-YYYYMMDD-HHMMSS.db temperature.db
sqlite3 temperature.db "PRAGMA integrity_check;"
sudo systemctl start temperature-server
```

Stop the server first so that nothing is writing. Moving the old file aside (instead of deleting it) keeps it available for analysis. After the restart, readings that the Raspberry Pi still holds as *pending* or as *delivered* within the last 7 days are **not** re-sent automatically after a restore: a record the node marked delivered is not sent again. Data between the backup time and the failure can therefore only be recovered from the Pi's local copy (Chapter 8) by a developer.

CHAPTER 11

Website Access and Login

The website is the Flask application on the Ubuntu server. It is reached with an ordinary web browser; nothing has to be installed on the computer.

Item	Value (supplied by owner)
Website URL	`http://10.145.15.245:8000`
Username	`admin`
Password	`avo@1234`

11.1 Procedure

1. **Connect the computer to the appropriate network.** The computer must be able to reach `10.145.15.245` on TCP port 8000 (the plant network, for example Wi-Fi `AVO_MACHINES` or a cable on the same network, or a VPN into it).
2. **Check the connection (optional).** In PowerShell: `Test-NetConnection 10.145.15.245 -Port 8000`. `TcpTestSucceeded : True` means the server is reachable.
3. **Open a web browser** (Edge, Chrome or Firefox).
4. **Type the address** `http://10.145.15.245:8000` in the address bar and press Enter. Use `http://` (not `https://`): the server has no certificate.
5. **The sign-in page appears.** It has a *Sign in* box with **Username** and **Password** fields, a **Sign in** button, and the note "Access is logged. Accounts lock temporarily after repeated failures." No screenshot of this page was supplied.
6. **Enter the username and password** and click **Sign in**. A wrong entry shows "Invalid username or password."; after 5 failures the account is locked for 15 minutes ("Account temporarily locked after repeated failures. Try again later.").
7. **The Dashboard opens** (Figure 12-1). Your name and role are shown at the top right, for example *admin (Administrator)*.

> **NOTE**
> After 30 minutes without activity the session ends and the sign-in page appears again (`SESSION_TIMEOUT_MINUTES`). Every sign-in, failed sign-in and sign-out is recorded in the Audit Trail. In the supplied Audit Trail (Figure 12-5) the entry at 08:56:38 is a failed login for the user name "admini" (a typing error).

11.2 Navigating the website

The dark bar at the top is on every page. The page you are on is underlined. Which items you see depends on your role.

Menu item	Opens	Permission needed
Dashboard	Live status of all controllers and nodes	`view_dashboard`
Monitoring	Trend chart and statistics for one controller	`view_data`
Data Logs	Every stored reading, with filters and export	`view_data`
Nodes	Raspberry Pi nodes, controllers and their health	`view_nodes`
Reports	Excursion, fault, downtime, missing-data and audit reports	`export_data`
Audit Trail	Append-only history of events	`view_audit`
Exports	History of every export and report	`export_data`

Menu item	Opens	Permission needed
Administration > Users, Roles & permissions	User accounts and the permission matrix	`manage_users`
Administration > System settings	Time zone and monitoring limits	`manage_settings`
User menu (top right)	**Change password** and **Sign out**	any signed-in user

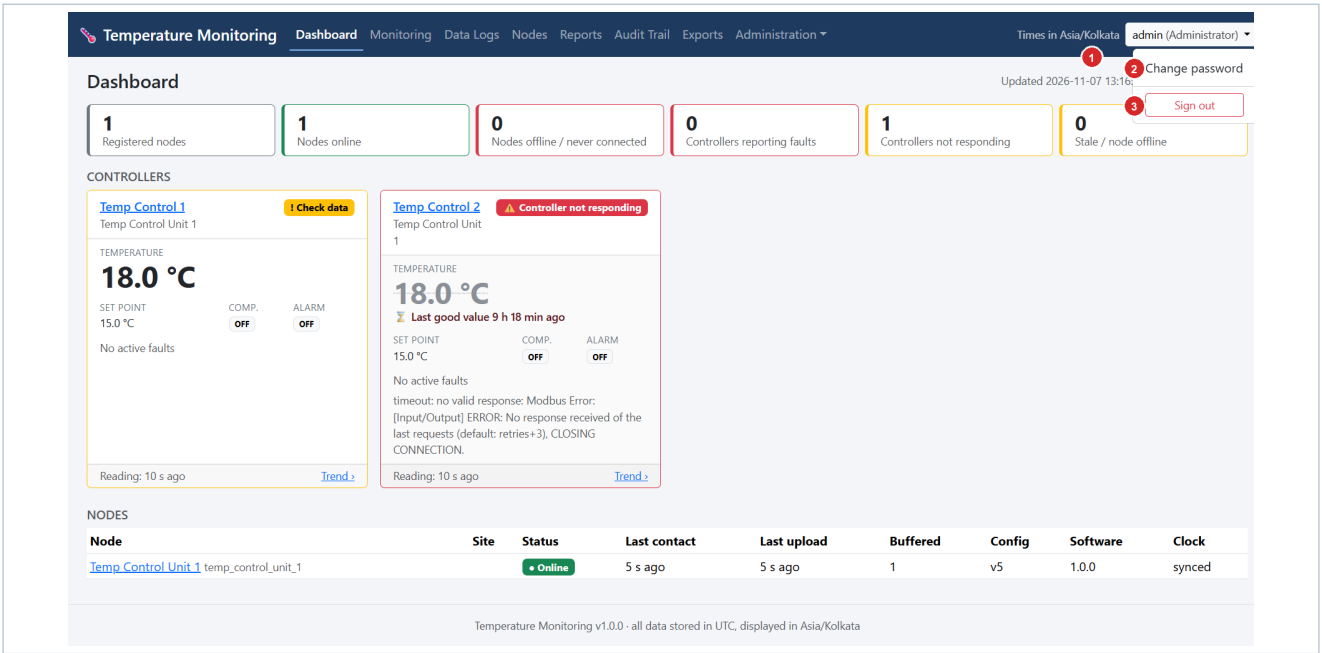


***Figure 11-1** - The user menu at the top right of any page (call-outs added). The screenshot was captured with the menu open.*

No.	Element	How to use it
1	User button, e.g. *admin (Administrator)*	Click to open the menu. It shows the signed-in user and the role.
2	Change password	Opens the password page (Figure 11-2).
3	Sign out	Ends the session. Always sign out on a shared computer.

11.3 Change your password

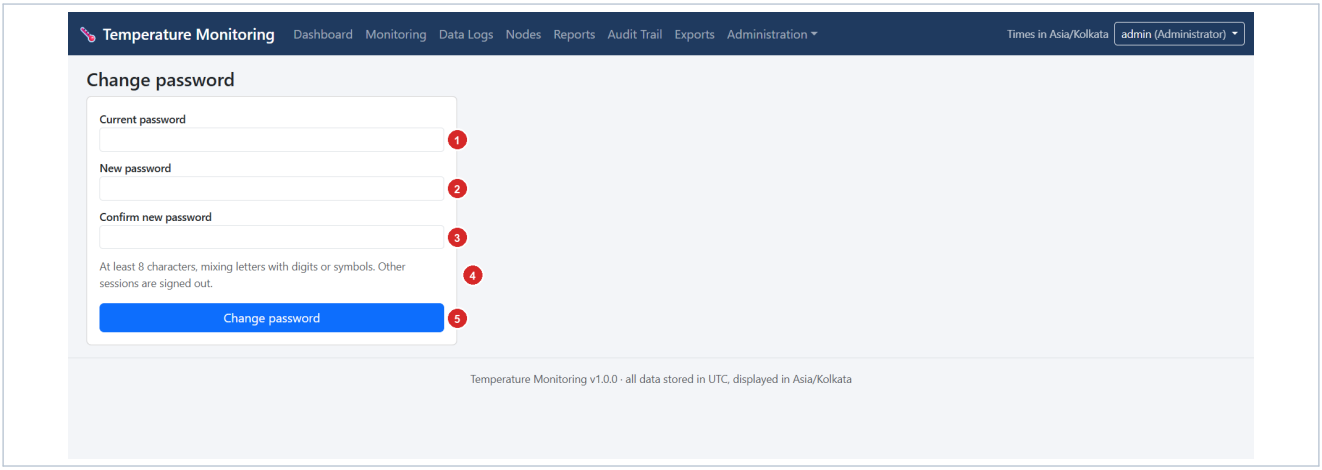


***Figure 11-2** - Change password page (call-outs added).*

No.	Field or control	Explanation
1	Current password	Your present password. If it is wrong the page says "Current password is incorrect."
2	New password	At least 8 characters, mixing letters with digits or symbols, and not containing your user name.
3	Confirm new password	Type the same new password again.
4	Hint text	"At least 8 characters, mixing letters with digits or symbols. Other sessions are signed out." Changing the password signs you out on all other browsers.
5	Change password	Saves the change (recorded in the Audit Trail). Expected result: "Password changed." and the Dashboard.

CHAPTER 12

Website Features and Page-by-Page Explanation

This chapter explains every page for which a screenshot was supplied, and the pages that exist in the source code but were not captured (Nodes, node and controller pages, Reports, sign-in). Every figure has numbered call-outs that are explained in the table below it.

12.1 Dashboard

Purpose. The live overview: how many nodes and controllers are healthy, and the current temperature, set point and alarm state of every controller. It refreshes itself every 10 seconds.

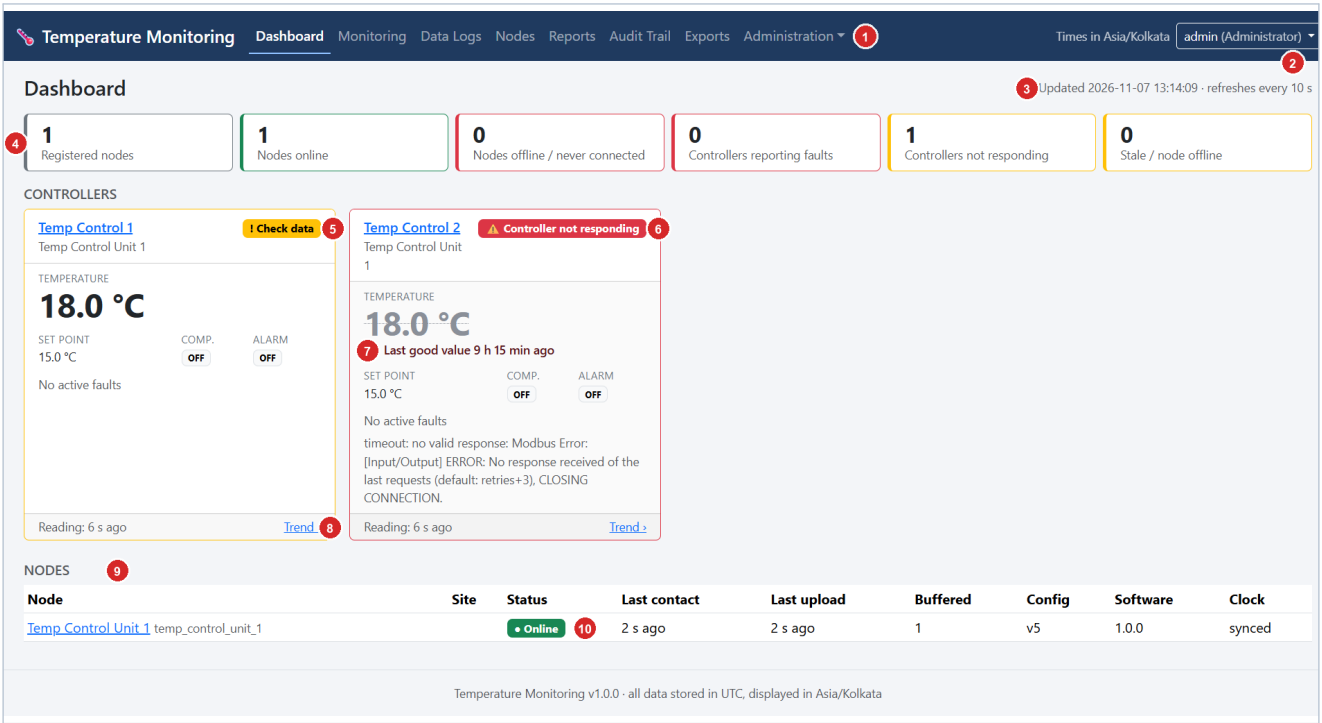


***Figure 12-1** - Dashboard (screenshot supplied by the owner, 2026-11-07 13:14, call-outs added).*

No.	Element	Explanation and how to use it
1	Navigation bar	Switch pages. The active page (Dashboard) is underlined.
2	User menu	Change password, sign out.
3	"Updated ... refreshes every 10 s"	Time of the last refresh (display time zone) and the refresh interval (setting *Dashboard refresh*). No reload is needed.
4	Summary tiles	Left to right: **Registered nodes** (1), **Nodes online** (1), **Nodes offline / never connected** (0, red when above 0), **Controllers reporting faults** (0, red), **Controllers not responding** (1, amber) and **Stale / node offline** (0, amber). A non-zero amber or red tile needs attention.
5	Controller card *Temp Control 1*	Name (click it for the controller page), node name, status badge **! Check data** (the reading is valid but a consistency check flagged it; see Chapter 7), **Temperature** 18.0 °C, **Set point** 15.0 °C, **Comp.** (compressor) OFF, **Alarm** OFF, "No active faults", "Reading: 6 s ago" and a **Trend ›** link. A card border in yellow/red repeats the status.
6	Controller card *Temp Control 2*	Status badge **Controller not responding** and the error text from the node ("timeout: no valid response ..."). The large temperature is greyed and struck through because it is **not live**.
7	"Last good value 9 h 15 min ago"	How old the last valid temperature is. Never read the greyed value as current.
8	Trend ›	Opens the Monitoring page for that controller (Figure 12-2).

No.	Element	Explanation and how to use it
9	NODES table	One row per Raspberry Pi: **Node** (name and Node ID), **Site**, **Status**, **Last contact**, **Last upload**, **Buffered** (readings waiting on the Pi; normally 0 to a few), **Config** (running configuration version, v5), **Software** (1.0.0) and **Clock** (*synced* when the Pi's clock is NTP-synchronised).
10	Status badge *Online*	Green = heard from within the last 120 s. *Offline* (red), *Never connected* and *Disabled* (grey) are the other states.

Expected result. Healthy installation: *Nodes online* equals *Registered nodes*, every controller card says **Normal**, *Reading* ages stay below about 15 s and *Buffered* is near 0. Chapter 13 explains each status.

12.2 Monitoring

Purpose. Trend and statistics for one controller over a chosen period, with missing data made visible.

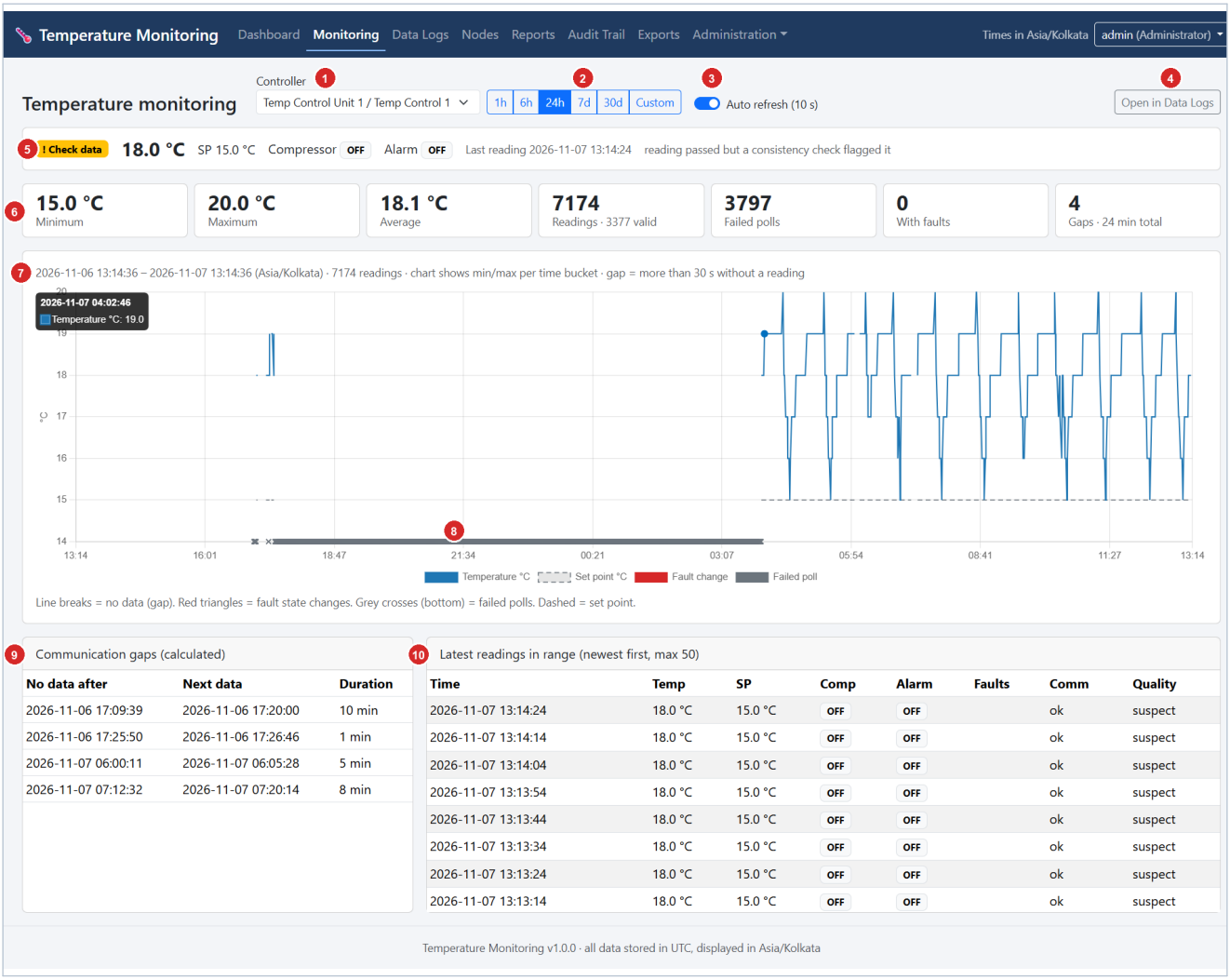


***Figure 12-2** - Temperature monitoring page for Temp Control Unit 1 / Temp Control 1 over 24 hours (screenshot supplied by the owner, call-outs added).*

No.	Element	Explanation and how to use it
1	Controller drop-down	Choose the controller (shown as *node / controller*).
2	Range buttons	**1h, 6h, 24h, 7d, 30d** or **Custom** (enter a from/to date). The selected range (24h) is highlighted.
3	Auto refresh (10 s)	When on, the page updates itself every 10 s. Turn it off to study the chart without it moving.
4	Open in Data Logs	Opens Data Logs with the same controller and period so you can see the individual rows or export them.

No.	Element	Explanation and how to use it
5	Status strip	Current status badge (**! Check data**), the latest temperature 18.0 °C, set point (SP) 15.0 °C, **Compressor** and **Alarm** state, the time of the last reading and an explanation of the status.
6	Statistic tiles	**Minimum**, **Maximum**, **Average** (of valid readings), **Readings** (7174, of which 3377 valid), **Failed polls** (3797), **With faults** (0) and **Gaps** (4, 24 min in total). Statistics use recorded readings only.
7	Chart	Blue line = temperature (minimum/maximum per time bucket). Dashed line = set point. Red triangles = fault-state changes. Grey crosses along the bottom = failed polls. **A break in the line means no data** (a gap is more than 30 s without a reading). Hover over a point for its time and value.
8	Failed-poll marks	The grey band along the bottom shows a long period of failed polls (a continuous run of crosses). Failed polls carry no temperature, so the line is absent there.
9	Communication gaps (calculated)	Each gap: *No data after*, *Next data*, *Duration*. Here four gaps (10 min, 1 min, 5 min, 8 min). They are calculated from missing readings, never estimated.
10	Latest readings in range	The newest 50 readings: Time, Temp, SP, Comp, Alarm, Faults, Comm, Quality. Here every row has Comm *ok* and Quality *suspect*.

12.3 Data Logs

Purpose. Every stored poll attempt, with powerful filters, paging and export. This is the page to use for investigations and for exporting.

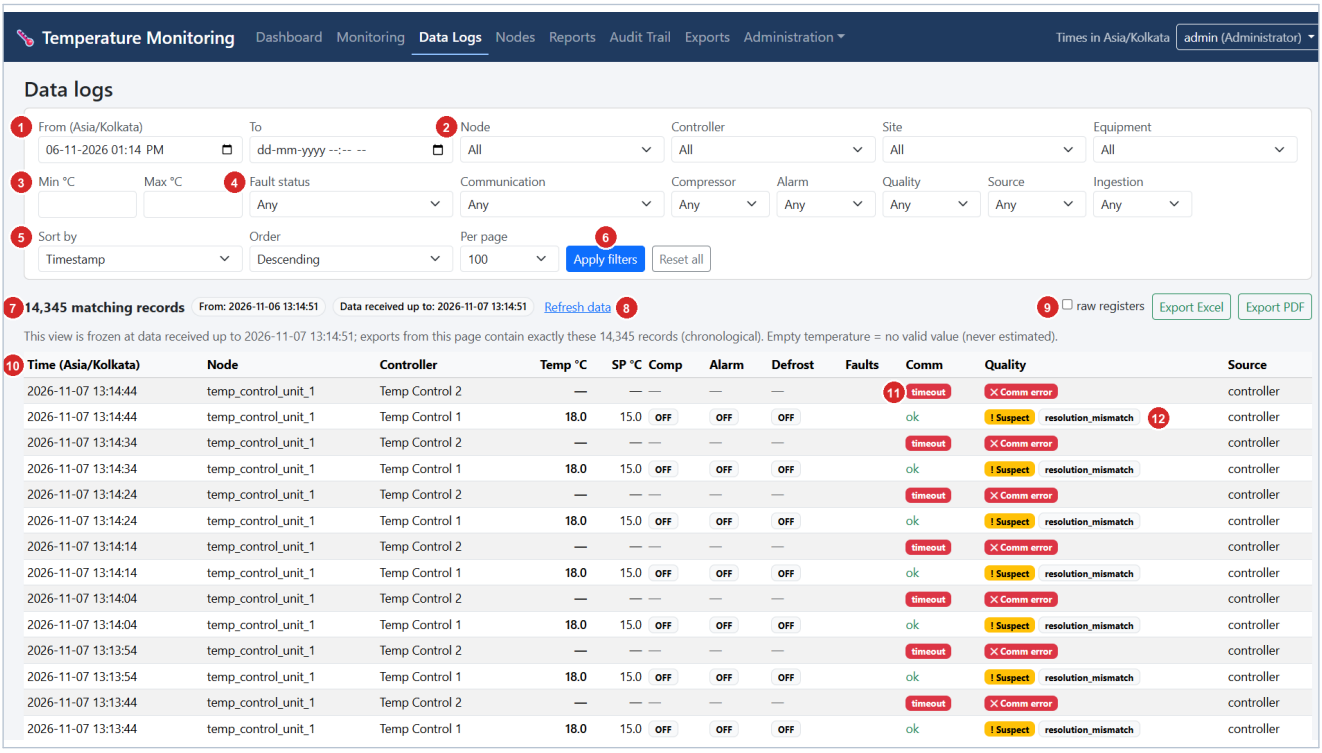


***Figure 12-3** - Data Logs page, upper part (screenshot supplied by the owner, call-outs added).*

No.	Element	Explanation and how to use it
1	From / To	Start and end of the period in the display time zone (Asia/Kolkata). Leave *To* empty for "until now". The default is the last 24 hours.
2	Node, Controller, Site, Equipment	Restrict to one node, controller, site or equipment.
3	Min °C / Max °C	Show only readings whose temperature is in this range.
4	Fault status, Communication, Compressor, Alarm, Quality, Source, Ingestion	**Fault status** (any/none/a specific fault), **Communication** (OK, any failure, timeout, exception, ...), **Compressor** and **Alarm** (on/off), **Quality** (good, suspect, probe fault, invalid, communication error), **Source** (controller or simulated), **Ingestion** (*live* or *backfilled* = delivered late after an outage).
5	Sort by, Order, Per page	Sort by timestamp, temperature, node, controller or fault status, ascending or descending; 50, 100, 250 or 500 rows per page.

No.	Element	Explanation and how to use it
6	Apply filters / Reset all	**Apply filters** runs the query; **Reset all** returns to the default (last 24 hours).
7	"14,345 matching records"	Number of rows matching the filters, with the data-freeze time: *Data received up to 2026-11-07 13:14:51*. The note under it explains that this view is frozen so that paging and exports contain exactly the same records.
8	Refresh data	Re-runs the query including data that arrived after the page was opened.
9	raw registers, Export Excel, Export PDF	Exports exactly the records of this view (Chapter 14). Tick **raw registers** to add the raw register values to the Excel file.
10	Table header	**Time** (display time zone), **Node**, **Controller**, **Temp °C** and **SP °C** (set point), **Comp**, **Alarm**, **Defrost**, **Faults**, **Comm**, **Quality**, **Source**.
11	Comm column	`ok`, or a red `timeout` and similar. A failed poll shows "-" for temperature and set point.
12	Quality column	A yellow **! Suspect** with the reason (here `resolution_mismatch`), or a red **Comm error** for failed polls.

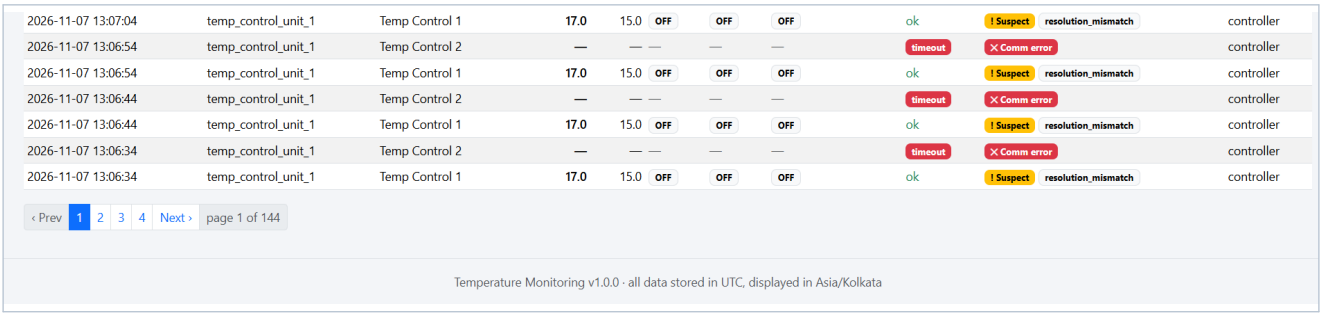


***Figure 12-4** - Data Logs, bottom of the page: paging (Prev, page numbers, Next, "page 1 of 144").*

Paging. Use *Prev*, the page numbers and *Next*. 14,345 records at 100 per page are 144 pages. A temperature shown as "-" means *no valid value*; it is never estimated.

12.4 Nodes, node page and controller page (no screenshot supplied)

DESCRIBED FROM SOURCE, NOT FROM A SCREENSHOT
No screenshots of these pages were supplied. The description below comes from the page templates in `server/app/templates/nodes/` and `controllers/`.

Nodes list

Columns: **Node ID**, **Name**, **Site / location**, **Equipment**, **Status**, **Last contact**, **Controllers**, **Config (running / latest)** (a *pending*, *delivered* or *rejected* badge shows a configuration still on its way or refused), **Software**. Buttons: **Communication events** (a log of all node events) and, for administrators, **+ Register node**. Below the table, administrators see **Unregistered devices that tried to connect** (Node ID, host name, IP, attempts) with a **Register...** shortcut; their data is not accepted until they are registered.

Node page

Area	Content
Header buttons (administrators)	**Edit configuration**, **Test connectivity** (queues a read-only identity check of registers 40001-40005 for every enabled controller), **Issue new token** (the old token stops working immediately; the new one is shown once), **Disable** / **Enable** (a disabled node's submissions are refused; the Pi keeps buffering).
Details	Site, location, equipment, registration date, token hint (last 4 characters), authentication failures.
Health (last heartbeat)	Last contact, last heartbeat, last data upload, last good poll, **Readings buffered**, upload failures in a row, software version, uptime, **Host / IP**, clock state (synchronised or skew), CPU temperature, free disk, Wi-Fi signal.
Configuration	Running version on the node versus latest on the server, a list of versions with status (applied, rejected, pending, delivered) and any error the node reported.
Controllers	Per controller: slave ID, status, temperature, set point, compressor, alarm, last reading and the node's own view of communication ("fails in a row", last error).

Area	Content
Recent communication events, Commands, Heartbeats	The latest events, queued controller commands and their results, and the latest heartbeats (config version, buffered, upload failures).

Edit configuration (administrators)

Opens a form with the node's description (friendly name, description, site, location, equipment), **Server URL override**, upload and heartbeat intervals, batch size, the serial bus (bus name, **Serial port**, **Baud rate**, **Parity**, timeout, retries, inter-request delay), one row per controller (**Controller ID**, **Name**, **Slave ID**, poll interval, **Parameter read (s)**, **Address offset**, **Read strategy**, **Enabled**), the buffer limits (retention days, **Max buffered records**, **When buffer is full**, **Min free disk**), **Log level**, **Diagnostic register logging**, an **Advanced JSON** box for several buses, and a mandatory **Reason for change**. Saving creates a new numbered version. The node fetches it within about a minute, validates it, and either applies it (and acknowledges *applied*) or keeps its current version (*rejected*, with the errors shown on the node page).

Controller page

Opened by clicking a controller name. Buttons: **Trend**, **Data logs**, **Node**. Cards: **Latest reading** (control and actual temperature, set point, resolution rS, compressor/alarm/defrost, faults, keypad lock, log high/low, compressor run hours, communication, quality and decoder warnings, and the **raw registers as received** with their signed values), **Configuration (server)**, **Device parameters 40031-40068** (read-only snapshot with the time it was read), **Configuration history**, **Fault and alarm periods**, and, for administrators, **Controller commands**. Write commands (alarm mute, fault reset, log reset, defrost) are available only when the `control_controllers` permission, the system setting *Allow controller commands* and `safety.allow_remote_control: true` on the node are all in place, and you type the controller ID and a reason to confirm. Read-only checks (identity check, read parameters) are always available.

12.5 Reports (no screenshot supplied)

The Reports page generates reports as Excel or PDF. Choose the **Report**, the period (**From**, **To**), optionally a **Node** and **Controller**, and the extra fields that appear for some reports, then click **Generate Excel** or **Generate PDF**.

Report	Content	Permission
Temperature history	Opens Data Logs with your period and filters, where you export the exact records.	`export_data`
High / low temperature excursions	Periods above or below limits you enter (**High limit °C**, **Low limit °C**). Gaps inside an excursion are flagged.	`export_data`
Controller faults and alarms	Fault and alarm periods from the fault register (Ht, Lt, door, compressor, probe).	`export_data`
Node downtime and communication interruptions	Node downtime *inferred* from missing heartbeats plus recorded communication events.	`export_data`
Missing data periods	Gaps between readings and periods without a valid temperature.	`export_data`
Configuration changes	Changes to nodes, controllers, settings (filter by event category).	`view_audit`
User activity	Logins and actions by user (filter by user and outcome).	`view_audit`
Data export activity	Who exported what and when.	`view_audit`
Audit trail	The audit trail for a period (filters: category, user, outcome).	`view_audit`

Every section of a report is labelled **RECORDED** (stored as it happened), **CALCULATED** (derived from recorded readings) or **INFERRED** (estimated from missing data). Every generated report is entered in the export history.

12.6 Audit Trail

Purpose. An append-only history of what happened: logins, configuration changes, communication events, faults, exports. Entries cannot be edited or deleted through the application, which enforces this in the program and with database triggers. Secrets are never recorded.

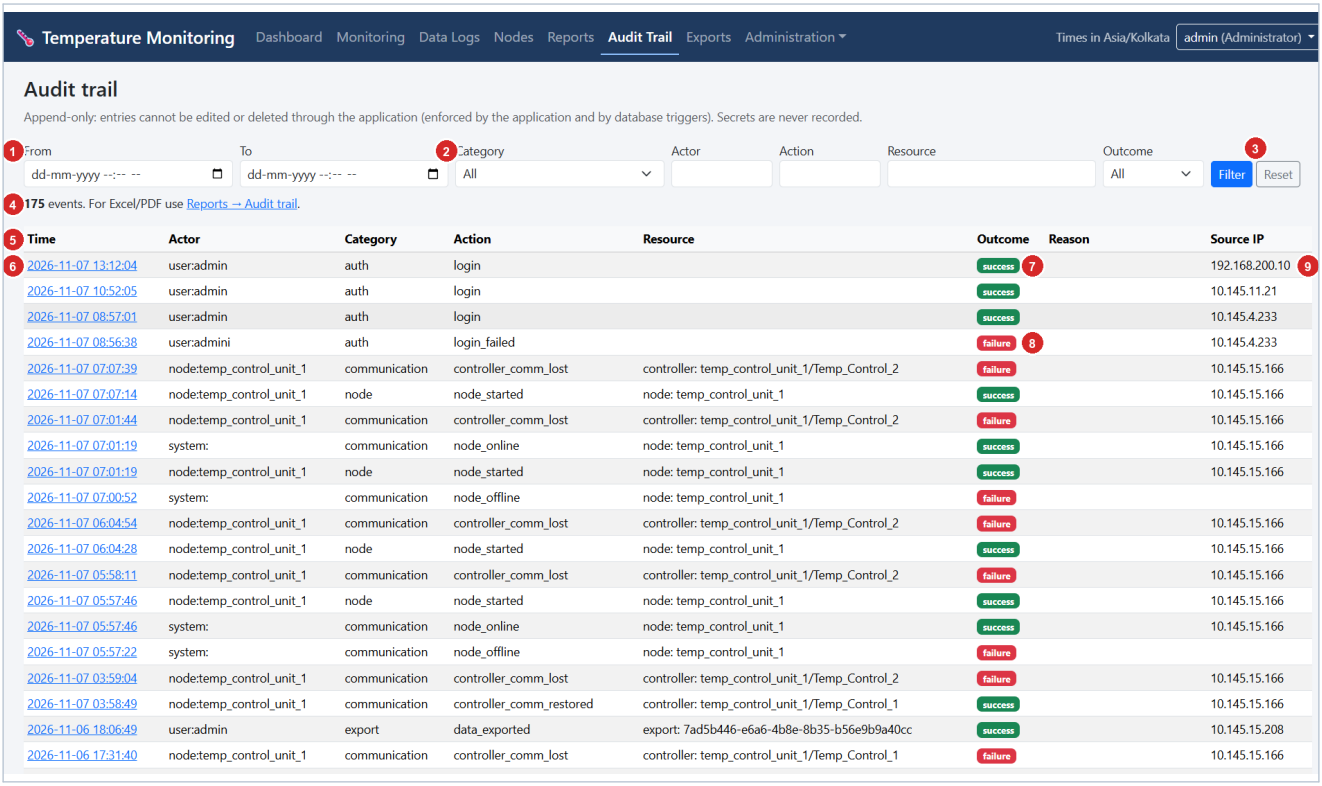


***Figure 12-5** - Audit Trail, top of the page (screenshot supplied by the owner, call-outs added).*

No.	Element	Explanation and how to use it
1	From / To	Restrict to a period (date and time in the display time zone).
2	Category, Actor, Action, Resource	Filter by category (auth, communication, node, config, command, export, system, user_admin, controller, fault), by who did it (e.g. `user:admin`, `node:temp_control_unit_1`, `system:`), by action name or by resource text.
3	Outcome, Filter, Reset	Outcome: success, failure or denied. **Filter** applies; **Reset** clears.
4	"175 events" line	Number of matching entries, and a pointer: for Excel/PDF use *Reports* > *Audit trail*.
5	Table header	**Time**, **Actor**, **Category**, **Action**, **Resource**, **Outcome**, **Reason**, **Source IP**.
6	Time (link)	Click the time to open the entry with all its details (before/after values, request ID, extra information).
7	Outcome badge	Green *success*, red *failure*.
8	A failure row	Here a failed login (`login_failed`) and `controller_comm_lost` events, in red.
9	Source IP	Address the request came from, for example the node `10.145.15.166` or an operator's PC. Entries created by the server itself (for example `node_offline`) have no source IP.

Typical actions: `login`, `login_failed`, `node_started`, `node_online`, `node_offline`, `controller_comm_lost`, `controller_comm_restored`, `data_gap_reported`, `node_reconnected`, `node_config_created`, `node_config_applied`, `node_config_rejected`, `controller_settings_changed`, `command_requested`, `data_exported`, `server_started`, `server_stopped`. The page shows 100 entries per page (here "page 1 of 2"); use *Prev / Next*.

12.7 Exports (export history)

Purpose. A record of every file generated: who, when, what, how many records, and a SHA-256 fingerprint of the file. The files themselves are not stored on the server.

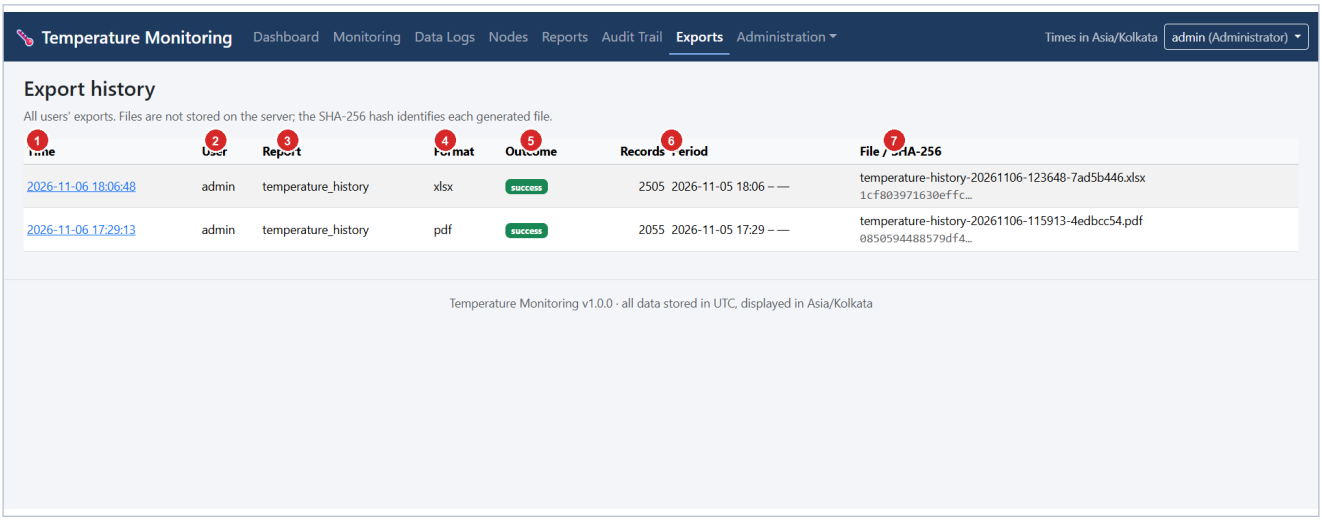


***Figure 12-6** - Export history (screenshot supplied by the owner, call-outs added).*

No.	Column	Explanation
1	Time (link)	When the export was generated (display time zone). Click for the detail page with the filters used and the full hash.
2	User	The user who generated it (here `admin`).
3	Report	The report type, here `temperature_history`.
4	Format	`xlsx` (Excel) or `pdf`.
5	Outcome	`success`, or a failure reason such as a rejected oversized export.
6	Records / Period	Number of records (2505 and 2055 in the screenshot) and the period start. "-" for an open end.
7	File / SHA-256	The generated file name (`temperature-history-20261106-123648-7ad5b446.xlsx`) and the start of the SHA-256 hash.

Check a file against its fingerprint. On the Windows computer run `Get-FileHash <file> -Algorithm SHA256` and compare the result with the hash on the export's detail page. They match only if the file is unchanged.

12.8 Administration: Users

Purpose. Manage the people who can sign in. Requires the `manage_users` permission.

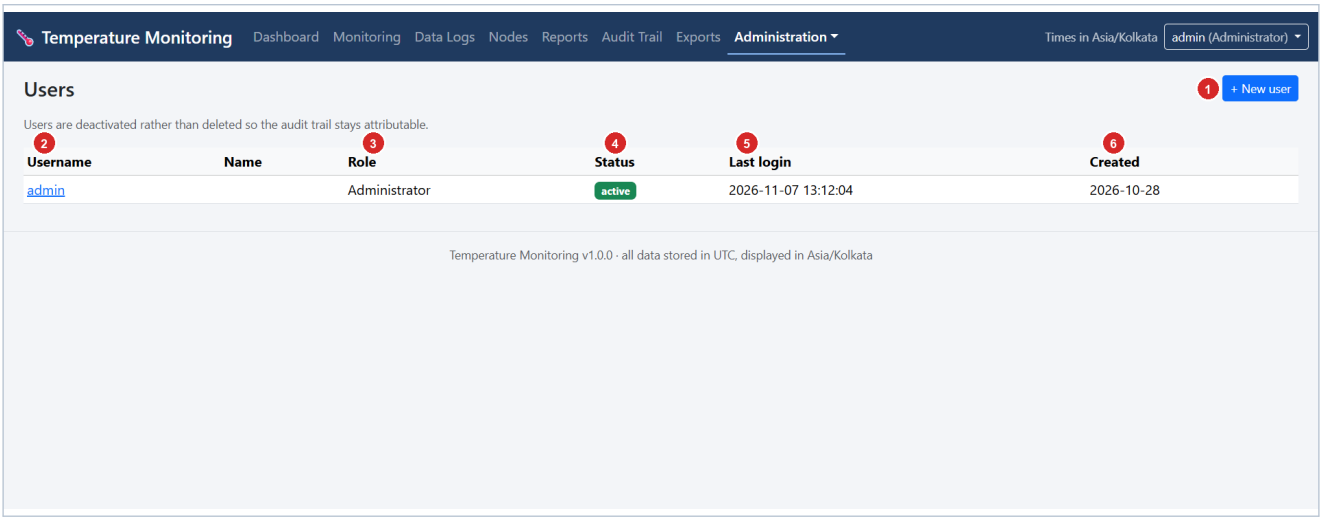


***Figure 12-7** - Users page (screenshot supplied by the owner, call-outs added).*

No.	Element	Explanation and how to use it
1	+ New user	Opens the form **Username**, **Full name**, **Email**, **Role**, **Active**, **Password**, **Reason for change**. The password must meet the policy.
2	Username (link)	Click to edit the user: change name, e-mail, role, active state or password. Changing role, status or password signs that user out everywhere.
3	Role	Administrator, Operator or Auditor.
4	Status	`active` (green) or inactive. Users are **deactivated rather than deleted** so that the audit trail stays attributable.
5	Last login	Time of the last successful sign-in.
6	Created	Date the account was created.

12.9 Administration: Roles and permissions

Purpose. Shows and edits which role may do what. Permissions are enforced on the server for every request; changes are audited and take effect on the next request.

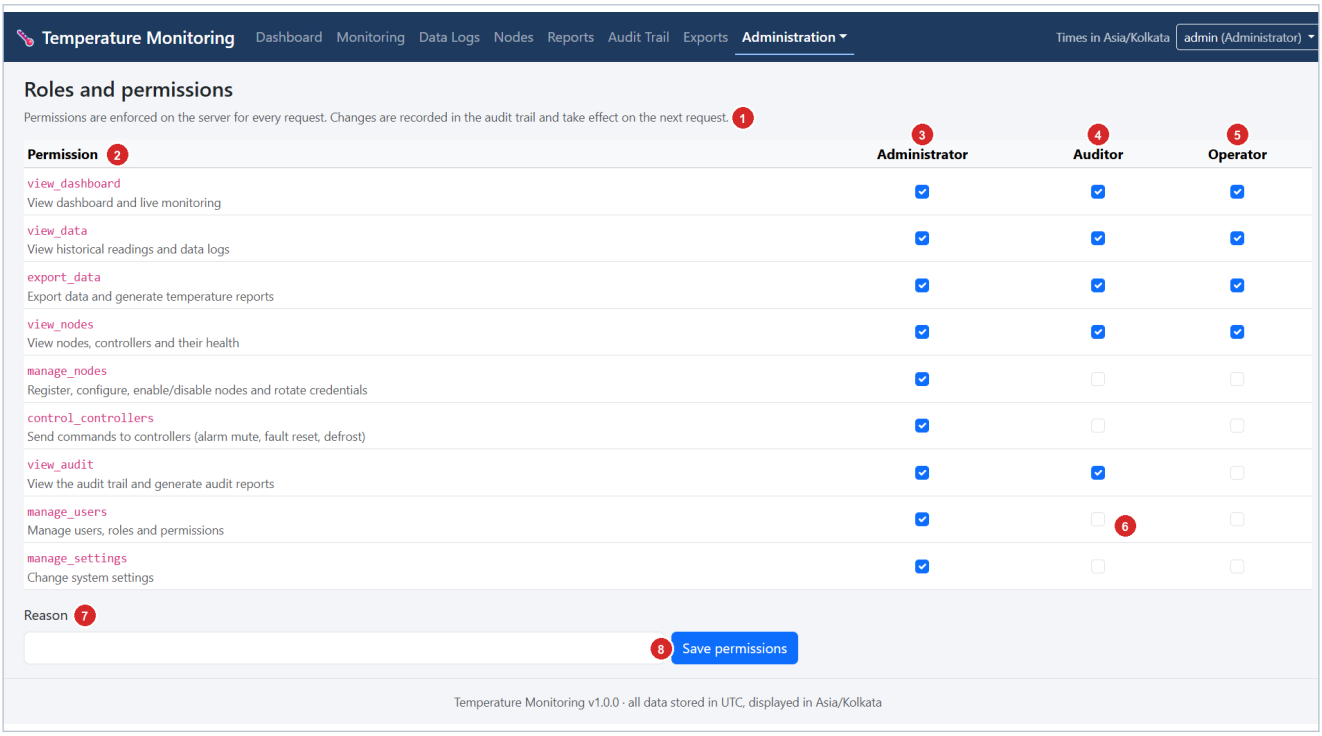


***Figure 12-8** - Roles and permissions (screenshot supplied by the owner, call-outs added).*

No.	Element	Explanation and how to use it
1	Introduction text	"Permissions are enforced on the server for every request. Changes are recorded in the audit trail and take effect on the next request."
2	Permission column	Each permission with its description: `view_dashboard`, `view_data`, `export_data`, `view_nodes`, `manage_nodes`, `control_controllers`, `view_audit`, `manage_users`, `manage_settings`.
3	Administrator column	All nine permissions are ticked.
4	Auditor column	View dashboard, data, exports and nodes, plus `view_audit`.
5	Operator column	View dashboard, data, nodes and export data; no audit, no management.
6	A checkbox	Tick or untick to grant or remove a permission for a role. An empty box means the role does not have it.
7	Reason	Mandatory text explaining the change (recorded in the audit trail).
8	Save permissions	Applies the matrix.

The last active administrator cannot be removed. Keep `manage_users`, `manage_settings` and `manage_nodes` limited to administrators.

12.10 Administration: System settings

Purpose. Site-wide limits that change how data is classified and displayed. Requires `manage_settings`. Every change needs a reason and is audited.

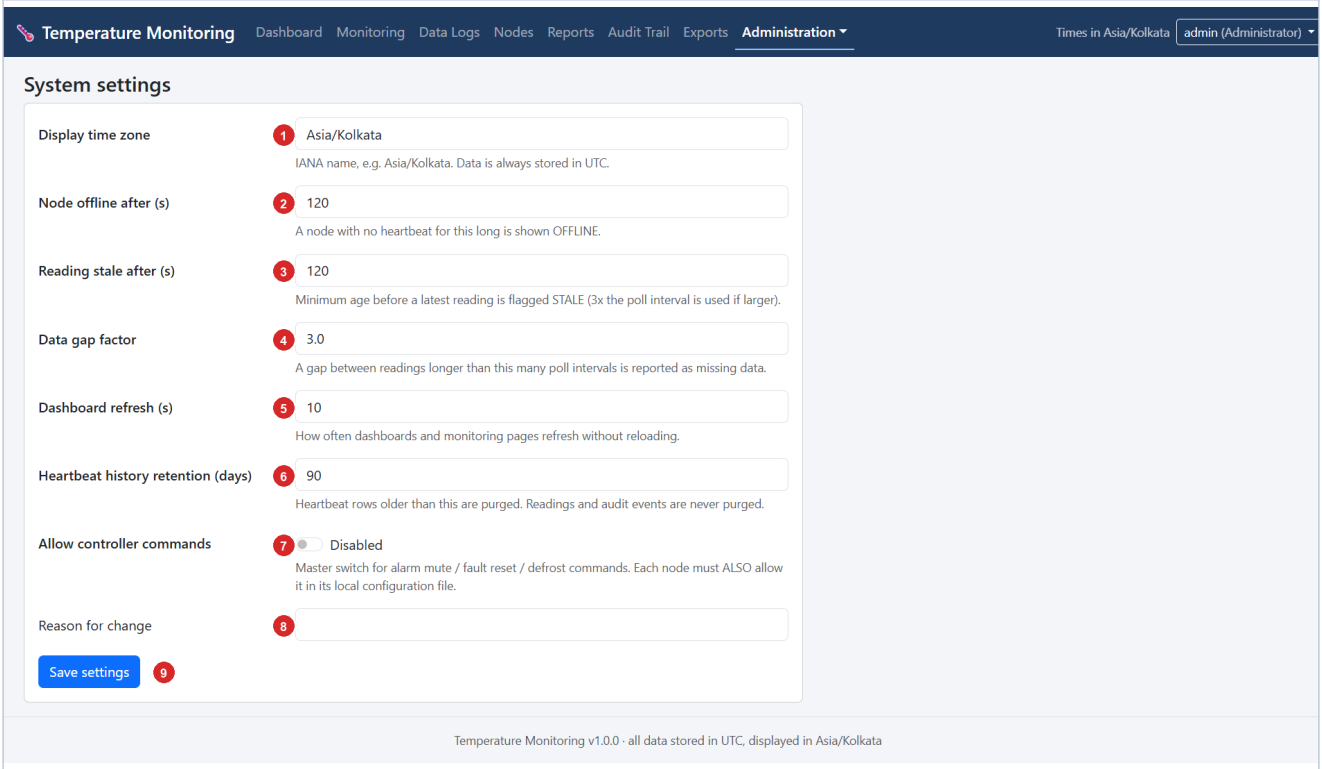


***Figure 12-9** - System settings page (screenshot supplied by the owner, call-outs added).*

No.	Setting	Explanation (value in the screenshot)
1	Display time zone	IANA name such as `Asia/Kolkata` (shown). All data is stored in UTC and converted for display.
2	Node offline after (s)	A node with no heartbeat for this long is shown **OFFLINE** (120). Allowed 30 to 86400.
3	Reading stale after (s)	Minimum age before a latest reading is flagged **STALE** (120). Three times the poll interval is used if larger. Allowed 10 to 86400.
4	Data gap factor	A gap longer than this many poll intervals (minimum 30 s) is reported as missing data (3.0).
5	Dashboard refresh (s)	How often dashboards and monitoring pages refresh without reloading (10). Allowed 2 to 3600.
6	Heartbeat history retention (days)	Heartbeat rows older than this are purged (90). Readings and audit events are never purged.
7	Allow controller commands	Master switch for alarm mute, fault reset and defrost commands (**Disabled**). Each node must also allow it in its own file.
8	Reason for change	Mandatory explanation of the change.
9	Save settings	Stores the changes.

CHAPTER 13

Monitoring Temperature Data

This chapter shows an operator how to read the temperature correctly and how to recognise missing, outdated or failed data.

13.1 Open the monitoring page and identify node and controller

- **Dashboard** for the overview; click **Trend ›** on a controller card, or open **Monitoring** and choose the controller in the drop-down (Figure 12-2, item 1).
- **Which node?** The card shows the node's friendly name (*Temp Control Unit 1*); the Nodes table and the Monitoring drop-down show it too; the Node ID is `temp_control_unit_1` (Figure 12-1).
- **Which controller?** The card title is the controller's name (*Temp Control 1*, *Temp Control 2*). Its controller ID (`Temp_Control_1`) and slave ID (1) are on the node page and the controller page.
- **Several channels.** One node can serve many controllers, and the system can have many nodes. Each controller provides one control temperature (register 40006). The *actual temperature* (register 40008) is stored separately and shown on the controller page and in exports.

13.2 Read the current temperature and its time

- The large number on the card (18.0 °C) and in the Monitoring status strip is the latest **valid control temperature**; the set point (SP 15.0 °C), compressor and alarm state are beside it.
- **"Reading: 6 s ago"** is the age of the newest poll attempt. **"Last reading 2026-11-07 13:14:24"** is its time in the display time zone (Asia/Kolkata). In an Excel export both the local and the UTC time are given.
- India Standard Time is UTC+05:30, so a reading shown as 13:14 IST was taken at 07:44 UTC.

13.3 Statuses: how to recognise missing, outdated and failed data

A latest value is presented as live **only** when it is recent, the node is online and the controller answered. Every other case gets an explicit label (colour is never the only signal). The thresholds are *Node offline after* 120 s and *Reading stale after* the larger of 120 s and three times the poll interval.

Controller status	Colour	Meaning	What to do
Normal	green	Recent valid reading, no fault.	Nothing.
Check data (suspect)	amber	The reading passed but a consistency check flagged it (for example `resolution_mismatch`, or an unsynchronised Pi clock).	See the flag in Data Logs; Chapter 7.
Controller fault	red	The controller reports a fault such as high temperature, low temperature, door open or compressor fault.	Check the controller and the equipment.
Probe fault	red	Temperature probe failure (PFL/PFH); no temperature is shown.	Check the probe.
Invalid measurement	red	Value outside the documented range or failed validation.	Check the controller settings (rS) and wiring.
Controller not responding	red	The node cannot read the controller (timeout and similar). The value shown is the last good one, greyed.	Chapter 7: wiring, slave ID, power.
Stale data	amber	The latest reading is older than the stale limit although the node is online.	Check the node's polling and uploads.
Node offline	red	The node has not been heard from for more than 120 s; the values shown are the last received.	Check the Pi's power and network.
No data yet	grey	No reading has ever been received.	Finish commissioning.
Disabled	grey	The node or controller is disabled.	Enable it if it should run.

Node status	Meaning
Online (green)	Contact within the last 120 s.
Offline (red)	No contact for more than 120 s. "no contact since ..." is shown.
Never connected	Registered but never heard from.
Disabled	Switched off by an administrator; submissions are refused.

13.4 Find communication failures and data gaps

- **Dashboard tiles:** *Controllers not responding*, *Nodes offline*, *Stale / node offline*.
- **Monitoring:** grey crosses for failed polls, line breaks and the *Communication gaps (calculated)* table (Figure 12-2).
- **Data Logs:** filter **Communication = Any failure** or *Timeout*, and look at the *Comm* and *Quality* columns (Figure 12-3).
- **Nodes page:** *Communication events* and the node page's event list (`comm_lost`, `comm_restored`, `data_gap`, `node_offline`).
- **Audit Trail:** the same events with outcome *failure* in red (Figure 8-4).
- **Reports:** *Missing data periods* and *Node downtime and communication interruptions*.

13.5 Verify that the Raspberry Pi is sending data correctly

Where	What to look at	Healthy
Dashboard, NODES table	Status, **Last contact**, **Last upload**, **Buffered**, **Config**, **Clock**	Online; both ages a few seconds; Buffered near 0; Config equals the latest version; Clock *synced*.
Node page, *Health*	Last heartbeat, last data upload, last good poll, readings buffered, upload failures in a row, CPU temperature, free disk, Wi-Fi signal	Failures 0; Wi-Fi better than about -70 dBm; plenty of free disk.
Data Logs	Newest row time	Within about 10-25 s of now, with *Ingestion* = *live*.
Audit Trail / events	`node_started`, `node_offline` entries	Few; each restart is explained.

13.6 The complete workflow from controller to screen

1. The controller measures the temperature and keeps it in register 40006.
2. Every 10 s the Raspberry Pi reads registers 40001-40020 over RS-485 and decodes them.
3. The reading is committed to the Pi's local buffer (status pending).
4. Within about 5 s the uploader sends it to the server, which stores it and confirms; the Pi marks it delivered.
5. The server updates the controller's latest state and any fault periods.
6. The browser asks the server for the dashboard data every 10 s and redraws the cards.
7. Result: a value on the screen is normally **no more than about 25-30 seconds old** (10 s poll + up to 5 s upload + up to 10 s refresh). If it is older, a status label says so.

CHAPTER 14

Exporting Temperature Logs

The website exports stored readings as an Excel workbook (.xlsx) or a PDF report. Both come from exactly the same filtered record set as the Data Logs page, so an Excel and a PDF made from the same view contain the same readings. Every export is recorded in the export history with a SHA-256 hash (Figure 12-6).

14.1 Export the readings shown in Data Logs

1. Open **Data Logs** (Figure 12-3).
2. Select the **Node** and/or **Controller** you need.
3. Set the **From** and **To** date and time (display time zone).
4. Apply any other filters (fault status, communication, quality, temperature limits) and click **Apply filters**. Check the record count in item 7. Note: when filters that remove some readings are used, gap analysis is not included in the export because the series is no longer complete.
5. Click **Export Excel** or **Export PDF**. Tick **raw registers** first if you want the raw register values in the Excel file.
6. The browser downloads the file. If a message says that the export is too large, narrow the period or the filters (limits: 500 000 rows for Excel, 20 000 rows for PDF). Exports are **never silently truncated**.
7. Find the file in the browser's download folder (normally `C:\Users\<you>\Downloads`). The name is `temperature-history-<yyyymmdd-hhmmss>-<8 characters>.xlsx` (or `.pdf`); the time in the name is **UTC**.
8. Open it: double-click the .xlsx file in Microsoft Excel (if Excel shows *Protected View* click **Enable Editing**).

> **NOTE**
> Reports other than the temperature history (excursions, faults, downtime, missing data, audit) are generated from **Reports**; see Section 12.5. The *Temperature history* entry there opens Data Logs.

14.2 Excel workbook layout

Sheet	Content
Summary	Report title, Report ID, time generated (local and UTC), generated by, time zone, period, number of records and every filter used. Then a per-controller table (Node, Controller, Site, Equipment, Readings, Valid temps, Min °C, Max °C, Avg °C, Comm failures, Readings with faults, Gaps, Gap time, Simulated) and notes explaining that statistics are calculated from the exported readings.
Readings	One row per reading, **oldest first**, with a frozen header row and filter buttons. Columns below.
Gaps	Node, Controller, *No data after*, *Next data at*, Duration (periods longer than the gap factor × poll interval, minimum 30 s).
Fault events	Controller, Kind, Code, Description, Started, Cleared, Duration, Status.

Columns of the Readings sheet

Column	Meaning
Timestamp (local)	Acquisition time in the display time zone, as a real Excel date-time.
Timestamp (UTC)	The same instant in UTC (ISO-8601).
Node	Node ID.
Controller	Controller name (or ID).
Temperature °C	Control (display) temperature, register 40006. **Empty = no valid value** (communication failure or probe fault).
Set point °C	Register 40018.
Actual temp °C (40008)	Register 40008 (type not specified by the manufacturer; informational).

Column	Meaning
Compressor, Alarm, Defrost	ON / OFF (empty if unknown).
Faults	Decoded fault codes.
Comm status	`ok`, `timeout`, `exception`, `invalid_response`, `port_error`, `identity_mismatch`, `error`.
Comm error	Error text for failed polls.
Quality	`good`, `suspect`, `probe_fault`, `invalid`, `comm_error`.
Quality flags	Reasons such as `resolution_mismatch`.
Source	`controller` or `simulated`.
Backfilled	`yes` if the reading was delivered late (after an outage), otherwise `no`.
Config version	Node configuration version in force when the reading was taken.
Reading ID	The unique UUID of the reading.
Raw registers	Only when *raw registers* was ticked: the registers as received (JSON).

Sample of the exported data

The following is an **illustrative** extract in the layout of the Readings sheet (selected columns; the *Node* column, always `temp_control_unit_1` here, and the remaining columns are omitted), using values like those in Figure 12-3:

Timestamp (local)	Controller	Temperature °C	Set point °C	Comm status	Quality	Quality flags	Backfilled
07-11-2026 13:13:44	Temp Control 1	18.0	15.0	ok	suspect	resolution_mismatch	no
07-11-2026 13:13:44	Temp Control 2	(empty)	(empty)	timeout	comm_error	(empty)	no
07-11-2026 13:13:54	Temp Control 1	18.0	15.0	ok	suspect	resolution_mismatch	no

14.3 PDF report layout

The PDF contains the title and report metadata (Report ID, generated, by whom, time zone, period, records, filters), the summary statistics per controller, the fault and alarm events (the first 300), the communication gaps, trend charts (for the first six controllers), and the chronological readings table in which **highlighted rows mark periods without data**. Each page carries the Report ID in its footer.

14.4 Verify and keep exports

- The website does not keep the file. Keep your own copy in a controlled folder.
- To prove that a copy is unchanged: `Get-FileHash .\temperature-history-....xlsx -Algorithm SHA256` and compare with the hash in **Exports** (Figure 12-6).
- Excel and PDF show only **recorded** values. Statistics, gaps and durations are **calculated** from them and labelled as such in the workbook notes.

CHAPTER 15

Troubleshooting

Start with the website (Dashboard, node page, Audit Trail) to see **where** the chain is broken, then use the commands below. The chain is: controller - RS-485 - Raspberry Pi - network - server - database - website. Commands marked *Pi* are typed on the Raspberry Pi, *VM* on the Ubuntu server.

15.1 Problem table

Problem	Possible cause	Diagnostic	Solution
Raspberry Pi does not connect to Wi-Fi	Wrong or changed password; SSID out of range; DHCP failure.	Pi: `nmcli device status`, `nmcli device wifi list`, `nmcli -f GENERAL.STATE,IP4.ADDRESS device show wlan0`	Reconnect with `sudo nmcli --ask device wifi connect "AVO_MACHINES"`; see Chapter 4.
Raspberry Pi cannot reach the server	Network connectivity issue; server stopped; firewall; wrong `server.url`.	Pi: `ping -c 4 10.145.15.245`; `curl -s http://10.145.15.245:8000/api/v1/ping`; `ip route`	Fix the route; start the server (`systemctl status temperature-server`); allow port 8000 (`sudo ufw status`); correct `server.url`. The Pi keeps buffering meanwhile.
No temperature readings	RS-485 communication failure: wiring, A/B swapped, TTL-to-RS-485 interface missing or unpowered, wrong slave ID, baud or parity, controller unpowered, `did` = 0.	Pi: `sudo journalctl -u temperature-node -f`; stop the service and run `validate_controller --scan` or `read_temp_from_controllert.py`	Check wiring (swap A/B), set matching `did`/`bd`/`PAr`, check power; Chapter 7.
Website not accessible	Server application stopped; VM down; wrong address; port 8000 blocked.	Windows: `Test-NetConnection 10.145.15.245 -Port 8000`. VM: `systemctl status temperature-server`; `sudo ss -ltnp | grep 8000`	Start the service; check `HOST=0.0.0.0`; open the firewall; Chapter 9.
Data not updating on the website	Client application stopped; Pi off or offline; uploads failing.	Website: node status and *Last upload*. Pi: `systemctl status temperature-node`; `journalctl -u temperature-node -n 50`	Restart: `sudo systemctl restart temperature-node`. If upload errors are shown, see the next rows.
Database not storing readings	Application or database error on the server; disk full; read-only file.	VM: `tail -n 100 ~/temperature-logging-system/logs/server.log`; `df -h`; `sqlite3 instance/temperature.db "PRAGMA integrity_check;"`	Free disk space; fix permissions of `instance/`; restore from backup if corrupt (Chapter 10).
Raspberry Pi program does not start after reboot	Service not enabled; configuration invalid; serial port missing.	Pi: `systemctl is-enabled temperature-node`; `systemctl status temperature-node`; `... --check-config`	`sudo systemctl enable --now temperature-node`; fix the configuration; Chapter 6.
Incorrect temperature value	Register mapping or scaling issue: rS changed; `resolution_mismatch`; wrong controller; offset.	Compare with the controller display; open the controller page and the raw registers; Pi: `validate_controller`	Check rS (40005); run the validation procedure; set `address_offset` only if validation proves it.
One controller shows *Controller not responding*, the other is fine	Slave ID not set, controller off, wiring to that controller. (Seen in the supplied screenshots for *Temp Control 2*.)	Data Logs: Comm column for that controller; Pi log: `poll failed (n in a row)`	Check that controller's power, `did`, `bd`, `PAr` and A/B. Disable the controller on the node page if it is not installed yet, so it stops generating failed polls.
Quality *suspect* with `resolution_mismatch`	rS (40005) and bit 2 of 40010 disagree (known ambiguity A5).	Controller page: resolution rS and decoder warnings	Compare with the display; scaling follows rS. Report to the vendor if values are 10x off.
Quality *suspect* with `clock_unsynchronized`	The Pi clock is not NTP-synchronised.	Pi: `timedatectl`; `systemctl status systemd-timesyncd`	Restore network access to a time server; wait for `System clock synchronized: yes`.
Node shows *Offline* although the Pi is on	No contact for 120 s: network, wrong URL, service stopped, server rejected credentials.	Pi log: `server unreachable` or `server rejected node credentials`	Fix network/URL; install the correct token.

Problem	Possible cause	Diagnostic	Solution
Node log: `server rejected node credentials`	Wrong token or Node ID; token re-issued.	Node page: *Auth failures*	Put the current token in `/etc/temperature-node/node.env`, restart the service.
Node appears under *Unregistered devices*	The Node ID is not registered on the server.	Nodes page	Register it with exactly that Node ID (Chapter 5, Step 3).
Configuration stays *delivered* or *rejected*	The node refused it (for example a serial port that does not exist) or has not reconnected.	Node page: *Configuration*, version page *Node errors*	Fix the error and save a new version.
Buffered count keeps growing	Server unreachable or refusing records.	Pi: `sqlite3 ... select status,count(*) ...`; Pi log	Restore the connection; the backlog drains automatically (Chapter 8).
Gaps in the chart	The node was not running (power, restart) or the buffer overflowed.	Audit Trail: `node_offline`, `data_gap_reported`; Monitoring: gap table	A gap while the Pi was off cannot be recovered. Investigate why the Pi lost power.
`port_error: cannot open serial port`	HAT driver not loaded; wrong port; port in use; `tempnode` not in `dialout`.	Pi: `ls -l /dev/ttySC*`; `id tempnode`; `sudo lsof /dev/ttySC1`	Stop other users of the port; load the HAT driver; Chapter 3.
Times look shifted by hours	Display time zone setting; browsing in another zone.	Footer text; Administration > System settings	Set *Display time zone* to `Asia/Kolkata`. Stored data is always UTC.
Export refused: "N records match, above the limit"	More than 500 000 rows (Excel) or 20 000 (PDF).	The message on the page	Narrow the period or filters, or use Excel for PDF-sized sets.
Account locked	5 failed logins; lock for 15 minutes.	Audit Trail: `login_failed` ... *account locked*	Wait 15 minutes, or ask an administrator to reset the password (Administration > Users).
Page shows "Internal error (reference ...)"	Application error.	VM: search the reference in `logs/server.log`	Fix the cause or contact the developer with the reference.
`database is locked` in the server log	Several heavy exports at once on SQLite.	VM: `server.log`	Retry later (the server waits up to 10 s). Consider PostgreSQL for heavy use.
Pi disk almost full	Large buffer, logs or other files.	Pi: `df -h`; `du -sh /var/lib/temperature-node /var/log/temperature-node`	Free space. Below 100 MB free the node applies its buffer-full policy and reports `data_gap`.
SSH refuses or warns about a changed host key	Device re-imaged; wrong address.	Windows: the warning text	`ssh-keygen -R <address>`, then connect again and verify the address.
Pi's Wi-Fi address changed	DHCP lease changed.	Pi (via Ethernet): `hostname -I`; website node page *Host / IP*	Use the new address for SSH; ask for a DHCP reservation.
Node briefly *Offline* then *Online* after a server restart	Not an error: the server's last-contact time was old when it restarted.	Audit Trail: `node_offline` then `node_online` seconds apart	No action.

15.2 Diagnostic command cheat sheet

Where	Command	Shows
Pi	`systemctl status temperature-node`	Is the node service running, since when.
Pi	`sudo journalctl -u temperature-node --since "1 hour ago"`	Recent node log.
Pi	`sudo tail -f /var/log/temperature-node/node.log`	Live node log file.
Pi	`sudo sqlite3 /var/lib/temperature-node/queue.db "select status,count(*) from readings group by status"`	Pending / delivered / rejected counts.
Pi	`ls -l /dev/ttySC*`	HAT serial ports present.
Pi	`hostname -I`, `ip route`, `nmcli device status`	Addresses, routes, interfaces.
Pi	`timedatectl`	Time zone and clock synchronisation.
Pi	`df -h`, `free -h`, `vcgencmd measure_temp`	Disk, memory, CPU temperature.
VM	`systemctl status temperature-server`	Is the server running.

Where	Command	Shows
VM	`sudo journalctl -u temperature-server -n 100`	Recent server log.
VM	`sudo ss -ltnp | grep 8000`	Is something listening on port 8000.
VM	`sqlite3 ~/temperature-logging-system/instance/temperature.db "PRAGMA integrity_check;"`	Database health (`ok`).
Windows	`Test-NetConnection 10.145.15.245 -Port 8000`	Can this PC reach the website.

CHAPTER 16

Maintenance and Backup

16.1 Restarting (Technician / Administrator)

Task	Commands	Effect on data
Restart the Raspberry Pi	`sudo reboot` (or `sudo shutdown -h now` to power off, then wait for the activity light to stop)	No readings are recorded while it is off; a gap is reported at the next start (Chapter 8). Pending readings are kept.
Restart the client application	`sudo systemctl restart temperature-node`	A few seconds without polling; `node_started` is logged.
Restart the Ubuntu server	`sudo reboot` on the VM; `temperature-server` starts by itself	Website unavailable; the Pi buffers and uploads afterwards. No data loss.
Restart the server application	`sudo systemctl restart temperature-server`	Website unavailable for a few seconds.

IMPORTANT
Prefer a clean shutdown (`sudo shutdown -h now`) to pulling the power. The node survives power cuts by design, but a clean stop avoids storage corruption and a long data gap.

16.2 Check resources (Administrator)

Check	Pi	VM
Disk space	`df -h /`; `du -sh /var/lib/temperature-node /var/log/temperature-node`	`df -h /`; `du -sh ~/temperature-logging-system/instance ~/temperature-logging-system/logs`
RAM	`free -h`	`free -h`
CPU load	`uptime`, `top` (press q to quit)	`uptime`, `top`
CPU temperature	`vcgencmd measure_temp` (also on the node page as *CPU temp*)	--
Application logs	`sudo journalctl -u temperature-node -n 100`; `/var/log/temperature-node/node.log`	`sudo journalctl -u temperature-server -n 100`; `~/temperature-logging-system/logs/server.log`

16.3 Routine schedule

Interval	Task	Who
Every shift / day	Look at the Dashboard: nodes online, no controller in red, *Buffered* near 0.	Operator
Weekly	Check disk space on the Pi and the VM; look at the Audit Trail for unexpected `node_offline` or failed logins.	Administrator
Monthly	Install operating-system updates in a maintenance window (`sudo apt update && sudo apt upgrade -y`, then reboot, Pi and VM one at a time). Test restoring a backup to a scratch location.	Administrator
After any change	Make a fresh backup and record the change.	Administrator
Yearly or per site rules	Review users and roles; change passwords; re-check the controller's display against the system (commissioning check).	Administrator / Technician

16.4 Backing up (Administrator)

What	Where it is	How
Database (the permanent record)	VM: `~/temperature-logging-system/instance/temperature.db`	Nightly `backup_db.py` with cron and a copy to another computer (Chapter 10).
Server settings	VM: `~/temperature-logging-system/.env` and `instance/secret_key`	Copy both into a protected archive (see below). They contain secrets.
Node settings	Pi: `/etc/temperature-node/config.yaml`, `node.env`, `/var/lib/temperature-node/remote_config.json`	Archive (see below). `node.env` holds the node token.
Project source code	The project folder on the development PC (and the copies on the Pi and VM)	Keep the repository (git) and a dated zip of the folder in a second location.
Local buffer	Pi: `/var/lib/temperature-node/queue.db`	Normally not backed up (it is a transient buffer); copy it with `sqlite3 ... ".backup ..."` only for investigations.

Pi: archive the node settings (contains the token - protect the file)

```
sudo tar -czf ~/node-settings-$(date +%Y%m%d).tgz /etc/temperature-node \
  /var/lib/temperature-node/remote_config.json
```

VM: archive the server settings (contains secrets - protect the file)

```
tar -czf ~/server-settings-$(date +%Y%m%d).tgz -C ~/temperature-logging-system .env instance/secret_key
```

VM: archive the application code

```
tar -czf ~/server-app-$(date +%Y%m%d).tgz --exclude=.venv --exclude=__pycache__ --exclude=instance \
    --exclude=logs -C ~ temperature-logging-system
```

16.5 Restoring the system after a failure (Administrator)

Failure	Recovery
Raspberry Pi hardware or card failure	Prepare a new card (Chapter 3), install the node (Chapter 5) with the **same Node ID** and the archived `config.yaml`. Issue a new token on the node page and put it into `node.env`. Data continues in the same series. Readings that were still pending on the failed card are lost.
Server VM failure	Rebuild the server (Chapter 9), restore `.env` and `instance/secret_key`, restore the latest database backup (Chapter 10) and start the service. The Pis keep buffering meanwhile and upload afterwards (limited by the buffer capacity). Data between the last backup and the failure that the Pis had already delivered can only be recovered from their 7-day local copies by a developer.
Corrupt database	Stop the server; restore the latest good backup (Chapter 10); start the server.
Lost administrator password	On the VM: `cd ~/temperature-logging-system && .venv/bin/python -m flask --app run create-admin --username <new-name>` creates another administrator; use it to reset the old password in Administration > Users.
Wrong server address after the VM moved	Change `server.url` on the node page (the node tests the new URL before switching) or in `config.yaml` and restart the node.

CHAPTER 17

System Access Credentials

CONFIDENTIAL - AUTHORIZED PERSONNEL ONLY
CONFIDENTIAL - AUTHORIZED PERSONNEL ONLY. Keep this chapter secure. Remove or redact it before the manual is shared with anyone who does not need the credentials.

SUPPLIED BY OWNER
Supplied by the project owner; not verifiable from the source code. Confirm on site.

17.1 Consolidated credentials

System	Address	Username	Password	Notes
Ubuntu server login (SSH)	10.145.15.245 (host `temp-Virtual-Machine`)	`temp`	`Temp@098`	`temp` = Temperature Monitoring System
Website login	http://10.145.15.245:8000	`admin`	`avo@1234`	Role: Administrator
Raspberry Pi login (SSH)	192.168.0.100 (Ethernet) or current Wi-Fi address; host `temp-logging-system-node-1`	`pi`	`avo1234`	Node 1

17.2 Network configuration

Network	Parameter	Value
Ethernet	Raspberry Pi address	`192.168.0.100 (fixed or reserved: To be confirmed)`
Wi-Fi	SSID	`AVO_MACHINES`
Wi-Fi	Password	`b1n@ry010`
Wi-Fi	Interface	`wlan0`
Wi-Fi	Observed Raspberry Pi address	`10.145.15.166 (DHCP; can change)`
Server	Address and port	`10.145.15.245, TCP 8000 (website and node API), TCP 22 (SSH)`

17.3 Application secrets (not passwords you type)

Secret	Where it lives	Notes
Node token	Pi: `/etc/temperature-node/node.env` (`TEMP_NODE_TOKEN`)	Shown **once** at node registration. The server stores only its SHA-256 hash. Replace it with *Issue new token* on the node page.
Server `SECRET_KEY`	VM: `~/temperature-logging-system/.env` (or `instance/secret_key`)	Signs sessions. Changing it signs everyone out.
Website password hashes	Database table `users`	Only hashes are stored; a password cannot be read back.
Wi-Fi password	Pi: NetworkManager profile `AVO_MACHINES`	Update with `nmcli` if the router password changes.

17.4 Changing the default passwords

Account	How to change it
Raspberry Pi `pi`	Log in and run `passwd`.
Ubuntu `temp`	Log in and run `passwd`.
Website `admin`	User menu > Change password (Section 12.8), or Administration > Users > the user.
Wi-Fi on the Pi	`sudo nmcli --ask device wifi connect "AVO_MACHINES"` after the router password has changed.
Node token	Nodes > the node > **Issue new token**; put the new token into `/etc/temperature-node/node.env`; `sudo systemctl restart temperature-node`.

Change the default passwords
The default passwords `avo1234` and `Temp@098` are short, shared and written in documents. Change them after commissioning, use separate website accounts per person, and prefer SSH keys for `pi` and `temp`. Never type these passwords into e-mail, chat or public sites.

CHAPTER 18

Final System Verification Checklist

Use this checklist at the end of installation and after major changes. Write the result or value in the second column and initial it. Each item points to the chapter that explains it.

18.1 Raspberry Pi and controller

	Verification item	Result / value	Initials / date
☐	Raspberry Pi boots successfully (Chapter 3)		
☐	Raspberry Pi connects to the network: Wi-Fi `wlan0` and/or Ethernet `eth0` have the expected addresses (Chapter 4)		
☐	Ethernet/Wi-Fi communication works: `ping` and SSH succeed		
☐	Raspberry Pi can reach the remote server: `curl http://10.145.15.245:8000/api/v1/ping` returns `"status":"ok"`		
☐	RS-485 module is detected: `ls -l /dev/ttySC*` lists `/dev/ttySC0` and `/dev/ttySC1`		
☐	Controller settings recorded: `did`, `bd`, `PAr` of each controller (Chapter 7)		
☐	Temperature controller communication works: `validate_controller` shows PASS and the printed temperature equals the controller display (including a negative value if possible)		
☐	Temperature readings are received: Data Logs shows rows with Comm `ok` for every installed controller		
☐	Client application runs successfully: `systemctl status temperature-node` is `active (running)`		
☐	Client application starts automatically after reboot: `sudo reboot`, then the service is `active (running)` (Chapter 6)		

18.2 Server and website

	Verification item	Result / value	Initials / date
☐	Server application runs successfully: `systemctl status temperature-server` is `active (running)` (Chapter 9)		
☐	Server application starts automatically after reboot		
☐	Database stores temperature readings: the SQL count in Chapter 10 increases		
☐	Website is accessible at `http://10.145.15.245:8000` from an operator PC		
☐	Website login works with the personal account (Chapter 11)		

	Verification item	Result / value	Initials / date
☐	Temperature readings appear correctly on the Dashboard and agree with the controller display (Chapter 12)		
☐	Historical data can be viewed in Monitoring and Data Logs		
☐	Temperature logs can be exported (Excel and PDF) and opened (Chapter 14)		

18.3 Data protection and operation

	Verification item	Result / value	Initials / date
☐	Database backup is verified: a backup file exists, `PRAGMA integrity_check` returns `ok`, and a copy is stored on another computer (Chapter 10)		
☐	Outage test done: network or server outage of 5 minutes leaves no gap and the backlog is delivered (Chapter 8)		
☐	Controller-loss test done: unplugging RS-485 raises *Controller not responding* and recovery is recorded		
☐	Default passwords changed; separate website users created (Chapter 17)		
☐	System settings reviewed (time zone `Asia/Kolkata`, offline limit, refresh)		
☐	Commissioning record completed (`docs/hardware-validation.md`) and filed		

APPENDIX A

Controller Parameter Registers 40031-40068

The node reads these registers every hour (and on request) and stores a snapshot on the server; they are never written by the software. The table is generated from `raspberry_pi_node/src/registers.py`. Descriptions, units and option labels come from the SZ-7526TX instruction manual as encoded in that file.

Register	Code	Description	Unit / options
40031	SP	Compressor relay set point	degC; temperature, scaled by rS
40032	P1	Heating or cooling mode	0 = Cooling (CoL); 1 = Heating (HEt)
40033	P2	High temperature limit	degC; temperature, scaled by rS
40034	P2d	Differential to reset high temperature fault	degC; temperature, scaled by rS
40035	P3	Low temperature limit	degC; temperature, scaled by rS
40036	P3d	Differential to reset low temperature fault	degC; temperature, scaled by rS
40037	HS	Maximum set point limit	degC; temperature, scaled by rS
40038	LS	Minimum set point limit	degC; temperature, scaled by rS
40039	P4	Differential for compressor relay ON	degC; temperature, scaled by rS
40040	P5	Probe calibration	degC; temperature, scaled by rS
40041	P6	Compressor restart delay	min
40042	ot	Minimum compressor ON time	min
40043	E1	Compressor relay status on probe failure	0 = OFF; 1 = ON; 2 = Duty cycle (CYC)
40044	Cn	Compressor OFF time during probe fault	min
40045	Cy	Compressor ON time during probe fault	min
40046	P7	Defrost duration	min
40047	P8	Defrost frequency	h
40048	AF0	AUX digital input function	0 = Disabled (dIS); 1 = Door open = open contact (doP); 2 = Door open = closed contact (dCL); 3 = Compressor fault = open contact (CoP); 4 = Compressor fault = closed contact (CCL)
40049	CF1/do1	AUX digital input fault sensing delay	s
40050	CF2/do2	Output action on AUX digital input fault	0 = Indication only (Enb); 1 = Indication + alarm (EnA); 2 = Compressor OFF + alarm (CoF)
40051	CF3	Compressor fault reset mode	0 = Auto (AUt); 1 = Manual (nAn)
40052	CF4	Compressor retrials before manual reset	--
40053	do3	Temperature hold delay on door open	s
40054	did	Device ID (0 = communication disabled)	--
40055	bd	Baud rate	1 = 9600; 2 = 19200; 3 = 38400
40056	Par	Parity	0 = None; 1 = Even; 2 = Odd
40057	Ccr	Reset compressor run hours	0 = no; 1 = yes
40058	ddF	Display during defrost	0 = Control probe temperature (Pr1); 1 = Defrost label (dF); 2 = Held temperature (CHt)
40059	nd	Default display	0 = Control probe temperature; 1 = Set point
40060	dd	Display delay (rising temperature)	s
40061	AL	Alarm relay activation	0 = Disabled (dIS); 1 = Ht, Lt and other faults (AHL); 2 = Ht and other faults (AHt); 3 = Lt and other faults (ALt)
40062	Adt	Ht/Lt alarm delay at power ON	min

Register	Code	Description	Unit / options
40063	Add	Alarm relay delay at door open	s
40064	rS	Controller resolution	0 = 0.1 degC; 1 = 1 degC
40065	PA	Keypad password	never transmitted
40066	LP	Keypad lock	0 = Unlocked (UnL); 1 = Locked (LoC)
40067	Ver	Software version (read only)	--
40068	FS	Restore factory defaults	0 = no; 1 = yes

Parameters marked as temperatures (SP, P2, P2d, P3, P3d, HS, LS, P4, P5) are scaled by rS. PA (keypad password) is never transmitted, stored or logged.

A.1 System status register 40010 (bits)

Bit	Key	Name	Note
0	power_off	POWER OFF	Documented as 'Extra'
1	keypad_locked	KEYPAD LOCK	1 = keypad locked
2	resolution_tenths	Resolution_1	1 = 0.1 degC, 0 = 1 degC (inverse polarity of rS)
3	resolution_2	Resolution_2	Documented as 'Extra'
4	temperature_unit	Temperature Unit selection	Documented as 'Extra'
5	defrost	DEFROST	1 = defrost ON
8	restart_delay	Compressor restart time delay (P6)	1 = delay in process
9	min_on_delay	Compressor minimum ON time delay (ot)	1 = delay in process
10	ht_lt_power_on_delay	Ht/Lt power ON delay (Adt)	1 = delay in process

A.2 Coils (function 01 read / 05 write)

Coil	Register-map function	Use in the software
1	Power (documented only as "Extra")	Never written.
2	Alarm relay mute (reads 1 while the alarm relay is ON; write 0 to mute)	Command alarm_mute
3	Manual fault reset (reads 1 while a manual-reset fault is present; write 0 to reset)	Command fault_reset
4	Temperature log reset (always 0; write 1 to reset)	Command temp_log_reset
5	Defrost (write 1 to start, 0 to stop)	Commands defrost_start, defrost_stop

All writes are disabled by default and need four conditions together: safety.allow_remote_control: true in the node's local file, the server setting *Allow controller commands*, the control_controllers permission, and a typed confirmation with a reason. Writes are never retried and are always audited.

APPENDIX B

Event, Status and Quality Glossary

B.1 Node events

Event	Raised when	Audit Trail action
`node_started`	The node software starts.	`node_started`
`data_gap`	The node was not running (`node_not_running`), or the buffer dropped/refused readings.	`data_gap_reported`
`server_connection_restored`	Uploads succeed again after an outage.	`node_reconnected`
`comm_lost` / `comm_restored`	A controller failed 3 polls in a row / answers again.	`controller_comm_lost` / `controller_comm_restored`
`modbus_error`	First failed poll of a failure run.	(event list only)
`parameter_snapshot`	Controller parameters read from the device (stored when changed).	`controller_parameters_changed` (on change)
`controller_command`	A controller command was executed.	`controller_command_executed`

B.2 Server-generated events

Event	Raised when
`node_offline`	No contact for longer than *Node offline after* (120 s).
`node_online`	First contact after being offline.
`api_auth_failure`	A node presented wrong credentials.
`server_started` / `server_stopped`	The server process starts / exits normally (audit trail).

B.3 Quality flags

Flag	Meaning
`resolution_mismatch`	rS (40005) and bit 2 of 40010 disagree.
`device_id_mismatch`	Register 40004 differs from the polled slave ID.
`clock_unsynchronized`	The Pi's clock was not synchronised when the reading was taken.
`out_of_range`	Control temperature outside -45.0 to 100.0 °C.
`unknown_resolution`	rS is neither 0 nor 1; temperatures withheld.
`identity_mismatch`	40001/40003 are not 7507/115.
`undocumented_fault_bits`	Bits 7-15 of the fault register are set.
`unexpected_high_byte`	A one-byte register has a non-zero high byte.
`missing_temperature`	No temperature could be derived.

B.4 Glossary

Term	Meaning
Modbus RTU	A serial protocol in which a master (the Raspberry Pi) asks slaves (the controllers) for register values.
RS-485	A two-wire differential serial bus (A and B) for long cables and several devices.
Slave ID / `did`	The Modbus address of a controller (1-250).
Holding register	A 16-bit value in the controller that is read with function code 03.
Node	A Raspberry Pi running the node software.
Heartbeat	The node's periodic health report to the server (every 30 s).
Pending / delivered / rejected	States of a record in the node's local buffer.
Backfilled	A reading delivered more than 120 s after it was taken.
Gap	A period without readings longer than the gap factor × poll interval (minimum 30 s).
UTC	Coordinated Universal Time; India Standard Time is UTC+05:30.
SHA-256	A fingerprint of a file; any change to the file changes it.

APPENDIX C

Quick Command Reference

C.1 Raspberry Pi

Daily commands

```
systemctl status temperature-node
sudo systemctl restart temperature-node
sudo systemctl stop temperature-node         # before any serial-port test
sudo journalctl -u temperature-node -f
sudo tail -f /var/log/temperature-node/node.log
sudo sqlite3 /var/lib/temperature-node/queue.db "select status,count(*) from readings group by status"
sudo cat /var/lib/temperature-node/remote_config.json
ls -l /dev/ttySC*
hostname -I ; ip route ; nmcli device status
curl -s http://10.145.15.245:8000/api/v1/ping
```

Controller validation (stop the service first)

```
cd /opt/temperature-node/raspberry_pi_node
sudo -u tempnode /opt/temperature-node/venv/bin/python -m src.validate_controller \
  --port /dev/ttySC1 --slave 1 --baud 9600 --parity N
sudo -u tempnode /opt/temperature-node/venv/bin/python -m src.validate_controller \
  --port /dev/ttySC1 --scan --scan-ids 1-20
```

C.2 Ubuntu server

Daily commands

```
systemctl status temperature-server
sudo systemctl restart temperature-server
sudo journalctl -u temperature-server -f
tail -f ~/temperature-logging-system/logs/server.log
sqlite3 ~/temperature-logging-system/instance/temperature.db "PRAGMA integrity_check;"
python3 ~/backup_db.py
```

APPENDIX D

Items To Be Confirmed

Everything the author could not establish from the source code or the supplied material, in one list. Confirm each item on site and then update this manual.

No.	Item	Why it is open	Where used
1	Ubuntu version of the virtual machine	Not supplied (README supports 22.04 and 24.04).	Chapter 9
2	Live node configuration (`config.yaml` plus server overlay): port, controller IDs, slave IDs, poll interval	Only `config.example.yaml` and the dev tool are in the repository; the audit trail shows remote changes.	Chapter 5
3	Exact Waveshare HAT model, revision, required boot-configuration line	Not part of the project source; chip markings not legible in the photographs.	Chapters 2, 3
4	Functions of the HAT jumpers and the 9-pin header; terminal order; termination	Vendor documentation needed.	Chapters 2, 7
5	Pin-out of the controller's 5-pin TTL OUT (RS485) header and the TTL-to-RS-485 interface used in this installation (type, supply, termination)	The instruction manual shows the header but does not name its pins.	Chapter 7
6	Whether `eth0` 192.168.0.100 is fixed in the Pi or reserved in a router; gateway; Wi-Fi subnet, router address	Not supplied.	Chapter 4
7	Whether `.local` name resolution works for the Pi	Depends on the network.	Chapter 3
8	Whether an RTC battery is fitted to the Pi 5	None visible in the photograph.	Chapter 2
9	Why Temp Control 2 does not respond	Seen as timeouts in the screenshots; cause not known.	Chapter 7
10	Cause of `resolution_mismatch` on Temp Control 1	rS and status bit 2 disagree; needs a check against the controller display.	Chapter 7
11	State of the firewall (`ufw`) on the VM; exact folder and service names on the VM	Not supplied (README names are used).	Chapter 9
12	Correctness of all credentials and IP addresses	Supplied by the owner; passwords are stored only as hashes.	Credentials, Chapter 17
13	Completion of the commissioning record	The development log states that the software was not yet validated against a physical controller.	Chapters 7, 18
14	Brief offline/online flicker after a server restart	Derived from the code; not tested.	Chapter 8

APPENDIX E

Source File Map

File	Role
raspberry_pi_node/src/main.py	Supervisor: starts and restarts threads, applies configuration, systemd notify and watchdog, start-up gap detection.
collector.py	One thread per RS-485 bus: polling, reading records, comm-loss events, parameter snapshots, commands.
modbus_client.py	Serial bus with lock, Modbus requests, retries, error classes, guarded writes.
registers.py, decoder.py	Register map and decoding rules (scaling, sentinels, bits, quality).
local_queue.py	SQLite buffer: pending/delivered/rejected, retention, capacity policy.
uploader.py, server_client.py	Upload loop with backoff; HTTP client for the node API.
health_monitor.py	Health metrics, clock check, heartbeat thread.
config_manager.py, config_sync.py	Local plus remote configuration, validation, acknowledgement.
validate_controller.py	Read-only commissioning tool.
server/run.py	Server entry point (Waitress), database preparation, lifecycle audit entries, monitor thread.
server/app/routes/	Pages and the node API (api.py).
server/app/services/ingestion.py	Validation, de-duplication, storage and fault tracking of node data.
server/app/services/monitor.py, status.py	Offline detection, status classification.
server/app/services/exports.py, reports.py	Excel/PDF exports and reports.
server/app/models/	Database tables.
scripts/setup_raspberry_pi.sh, scripts/backup_database.py	Pi installer; database backup.
dev_tools/	read_temp_from_controllert.py (quick controller read), delete_controler_in_server_history.py (destructive clean-up of test controllers).